

Lab4-1. In the first lab, we investigate IP protocol by capturing and studying IP datagrams.

Lab4-2. In the second lab, we capture and study ICMP packets generated by other utility programs such as ping and traceroute programs.

4.9 PROGRAMMING ASSIGNMENT

Write the source code, compile, and test the following programs in one of the programming languages of your choice:

Prg4-1. Write a program to simulate the distance vector algorithm (Table 4.4).

Prg4-2. Write a program to simulate the link state algorithm (Table 4.5).

Prg4-3. Write a program to simulate the path vector algorithm (Table 4.6).

Data-Link Layer: Wired Networks

In Chapters 2 to 4, we discussed the application, transport, and network layers. In this chapter and the next, we discuss the data-link layer. The TCP/IP suite does not define any protocol in the data-link or physical layers. These two layers are territories of networks that when connected make up the Internet. As we discussed in Chapter 1, these networks, wired or wireless, provide services to the upper three layers of the TCP/IP suite. This may give us a clue that there are several standard protocols in the market today. For this reason, we discuss the data-link layer in two chapters: In this chapter we give the general concept of the data-link layer and discuss wired networks; in the next we discuss wireless networks. This chapter is divided into seven sections.

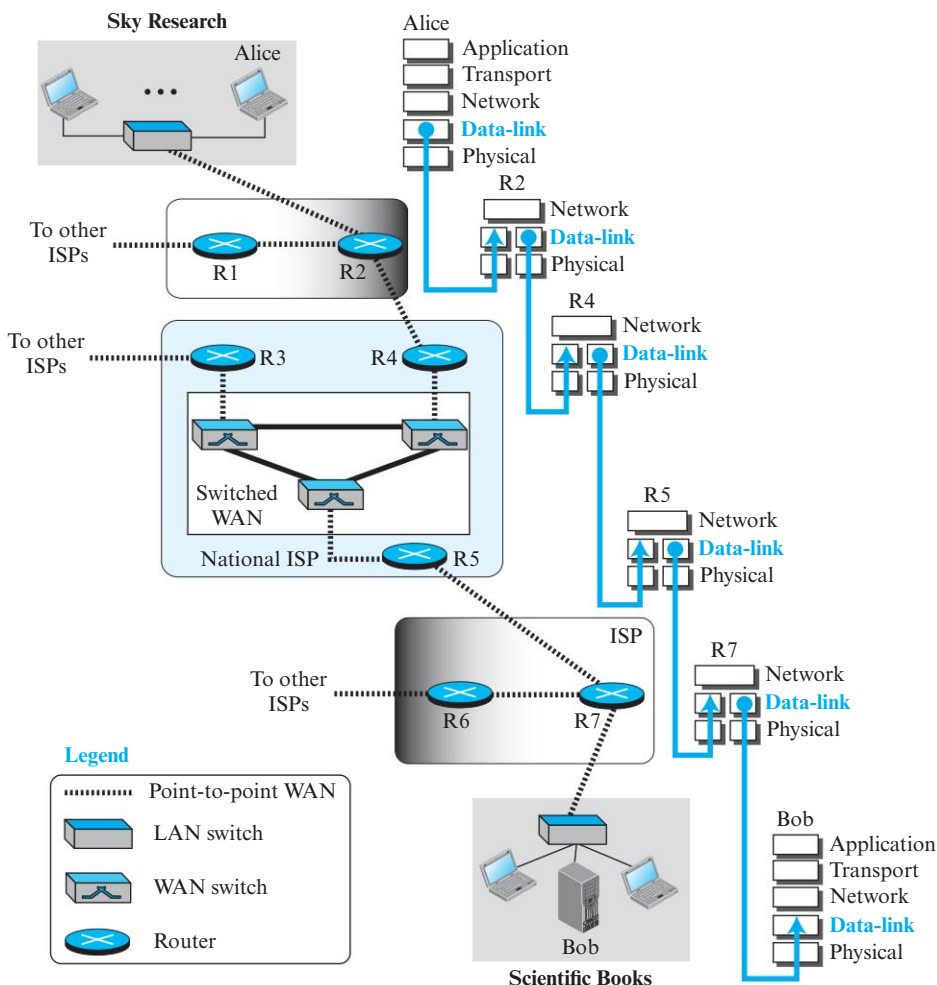
- ❑ In the first section, we introduce the concept of nodes and links and the types of links, and show how the data-link layer is actually divided into two sublayers: data link control and media access control.
- ❑ In the second section, we discuss data link control (DLC) of the data-link layer and explain services provided by this layer, such as framing, flow and error control, and error detection.
- ❑ In the third section, we discuss the media access control (MAC) sublayer of the data-link layer. We explain different approaches such as random access, controlled access, and channelization.
- ❑ In the fourth section, we discuss link-layer addressing and how the link-layer address of a node can be found using the Address Resolution Protocol (ARP).
- ❑ In the fifth section, we introduce the wired LANs and in particular Ethernet, the dominant LAN protocol today. We move through different generations of Ethernet and show how it has evolved.
- ❑ In the sixth section, we discuss other wired networks that we encounter in the Internet today, such as point-to-point networks and switched networks.
- ❑ In the seventh section, we discuss connecting devices used in the lower three layers of the TCP/IP protocol such as hubs, link-layer switches, and routers.

5.1 INTRODUCTION

In Chapter 4, we learned that communication at the network layer is host-to-host. Although it may be fragmented and reassembled, a datagram is a data unit that is sent from a host, somewhere in the world, to another host, somewhere else in the world. The Internet, however, is a combination of networks glued together by connecting devices (routers or switches). If a datagram is to travel from a host to another host, it needs to pass through these networks.

Figure 5.1 shows communication between Alice and Bob, using the same scenario we followed in the last three chapters. Communication at the data-link layer, however, is made up of five separate logical connections between the data-link layers in the path.

Figure 5.1 Communication at the data-link layer

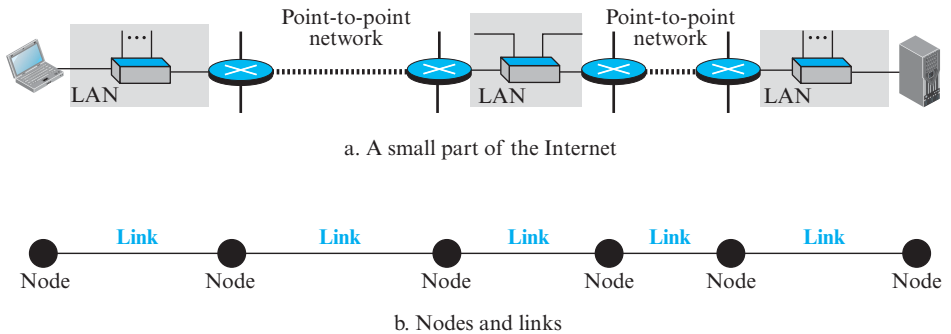


The data-link layer at Alice's computer communicates with the data-link layer at router R2. The data-link layer at router R2 communicates with the data-link layer at router R4, and so on. Finally, the data-link layer at router R7 communicates with the data-link layer at Bob's computer. Only one data-link layer is involved at the source or the destination, but two data-link layers are involved at each router. The reason is that Alice's and Bob's computers are each connected to a single network; each router, however, takes input from one network and sends output to another network.

5.1.1 Nodes and Links

Although communication at the application, transport, and network layers is end-to-end, communication at the data-link layer is node-to-node. As we have learned in the previous chapters, a data unit from one point in the Internet needs to pass through many networks (LANs and WANs) to reach another point. These LANs and WANs are connected by routers. It is customary to refer to the two end hosts and the routers as **nodes** and the networks in between as **links**. The following is a simple representation of links and nodes when the path of the data unit is only six nodes.

Figure 5.2 Nodes and Links



The first node is the source host; the last node is the destination host. The other four nodes are four routers. The first, the third, and the fifth links represent the three LANs; the second and the fourth links represent the two WANs.

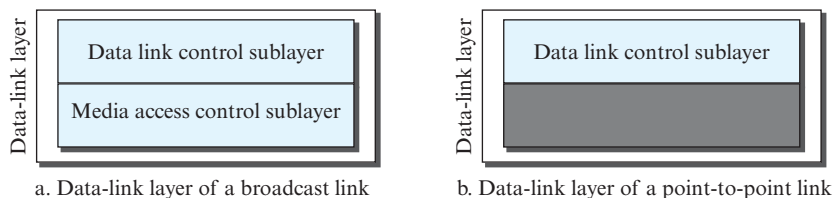
5.1.2 Two Types of Links

Although two nodes are physically connected by a transmission medium such as cable or air, we need to remember that the data-link layer controls how the medium is used. We can have a data-link layer that uses the whole capacity of the medium; we can also have a data-link layer that uses only part of the capacity of the link. In other words, we can have a *point-to-point link* or a *broadcast link*. In a point-to-point link, the link is dedicated to the two devices; in a broadcast link, the link is shared between several pairs of devices. For example, when two friends use the traditional home phones to chat, they are using a point-to-point link; when the same two friends use their cellular phones, they are using a broadcast link (the air is shared among many cell phone users).

5.1.3 Two Sublayers

To better understand the functionality of and the services provided by the link layer, we can divide the data-link layer into two sublayers: *data link control (DLC)* and *media access control (MAC)*. This is not unusual because, as we will see later in this chapter and in the next chapter, LAN protocols actually use the same strategy. The data link control sublayer deals with all issues common to both point-to-point and broadcast links; the media access control sublayer deals only with issues specific to broadcast links. In other words, we separate these two types of links at the data-link layer as shown in Figure 5.3.

Figure 5.3 Dividing the data-link layer into two sublayers



In this chapter, we first discuss the data link control sublayer that is common to both types of links. We then discuss the media access control sublayer that is used only in the broadcast link. After discussing these two sublayers, we discuss a protocol from each category.

5.2 DATA LINK CONTROL (DLC)

The data link control deals with procedures for communication between two adjacent nodes—node-to-node communication—no matter whether the link is dedicated or broadcast. **Data link control (DLC)** functions include *framing*, *flow and error control*, and *error detection and correction*. In this section, we first discuss framing, or how to organize the bits that are carried by the physical layer. We then discuss flow and error control. Techniques for error detection are discussed at the end of this section.

5.2.1 Framing

Data transmission in the physical layer means moving bits in the form of a signal from the source to the destination. The physical layer provides bit synchronization to ensure that the sender and receiver use the same bit durations and timing. We discuss the physical layer in Chapter 7.

The data-link layer, on the other hand, needs to pack bits into frames, so that each frame is distinguishable from another. Our postal system practices a type of framing. The simple act of inserting a letter into an envelope separates one piece of information from another; the envelope serves as the delimiter. In addition, each envelope defines the sender and receiver addresses, which is necessary since the postal system is a many-to-many carrier facility.

Framing in the data-link layer separates a message from one source to a destination by adding a sender address and a destination address. The destination address defines where the packet is to go; the sender address helps the recipient acknowledge the receipt.

Although the whole message could be packed in one frame, that is not normally done. One reason is that a frame can be very large, making flow and error control very inefficient. When a message is carried in one very large frame, even a single-bit error would require the retransmission of the whole frame. When a message is divided into smaller frames, a single-bit error affects only that small frame.

Frame Size

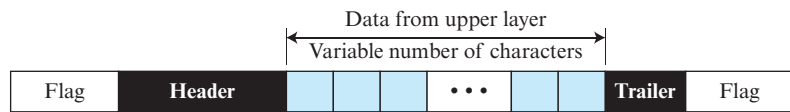
Frames can be of fixed or variable size. In *fixed-size framing*, there is no need for defining the boundaries of the frames; the size itself can be used as a delimiter. An example of this type of framing is the ATM WAN, which uses frames of fixed size called *cells*. We discuss ATM later in the chapter.

Our main discussion in this chapter concerns *variable-size framing*, prevalent in local-area networks. In variable-size framing, we need a way to define the end of one frame and the beginning of the next. Historically, two approaches were used for this purpose: a character-oriented approach and a bit-oriented approach.

Character-Oriented Framing

In *character-oriented (or byte-oriented) framing*, data to be carried are 8-bit characters from a coding system such as ASCII (see Appendix A). The header, which normally carries the source and destination addresses and other control information, and the trailer, which carries error detection redundant bits, are also multiples of 8 bits. To separate one frame from the next, an 8-bit (1-byte) **flag** is added at the beginning and the end of a frame. The flag, composed of protocol-dependent special characters, signals the start or end of a frame. Figure 5.4 shows the format of a frame in a character-oriented protocol.

Figure 5.4 A frame in a character-oriented protocol

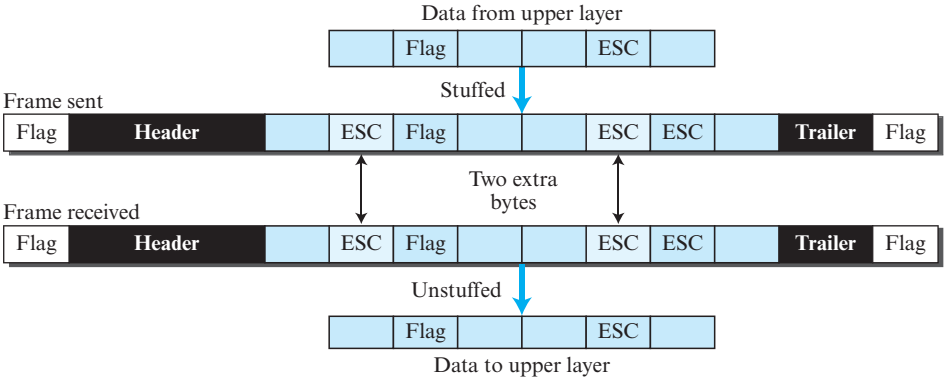


Character-oriented framing was popular when only text was exchanged by the data-link layers. The flag could be selected to be any character not used for text communication. Now, however, we send other types of information such as graphs, audio, and video, any pattern used for the flag could also be part of the information. If this happens, the receiver, when it encounters this pattern in the middle of the data, thinks it has reached the end of the frame. To fix this problem, a **byte-stuffing** strategy was added to character-oriented framing. In byte stuffing (or character stuffing), a special byte is added to the data section of the frame when there is a character with the same pattern as the flag. The data section is stuffed with an extra byte. This byte is usually called the *escape character (ESC)* and has a predefined bit pattern. Whenever the

receiver encounters the ESC character, it removes it from the data section and treats the next character as data, not as a delimiting flag.

Byte stuffing by the escape character allows the presence of the flag in the data section of the frame, but it creates another problem. What happens if the text contains one or more escape characters followed by a byte with the same pattern as the flag? The receiver removes the escape character, but keeps the next byte, which is incorrectly interpreted as the end of the frame. To solve this problem, the escape characters that are part of the text must also be marked by another escape character. In other words, if the escape character is part of the text, an extra one is added to show that the second one is part of the text. Figure 5.5 shows the situation.

Figure 5.5 *Byte stuffing and unstuffing*



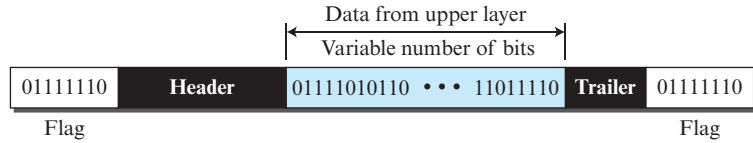
Byte stuffing is the process of adding one extra byte whenever there is a flag or escape character in the text.

Character-oriented protocols present another problem in data communications. The universal coding systems in use today, such as Unicode, have 16-bit and 32-bit characters that conflict with 8-bit characters. We can say that, in general, the tendency is moving toward the bit-oriented protocols that we discuss next.

Bit-Oriented Framing

In *bit-oriented framing*, the data section of a frame is a sequence of bits to be interpreted by the upper layer as text, graphic, audio, video, and so on. However, in addition to headers (and possible trailers), we still need a delimiter to separate one frame from the other. Most protocols use a special 8-bit pattern flag, 01111110, as the delimiter to define the beginning and the end of the frame, as shown in Figure 5.6.

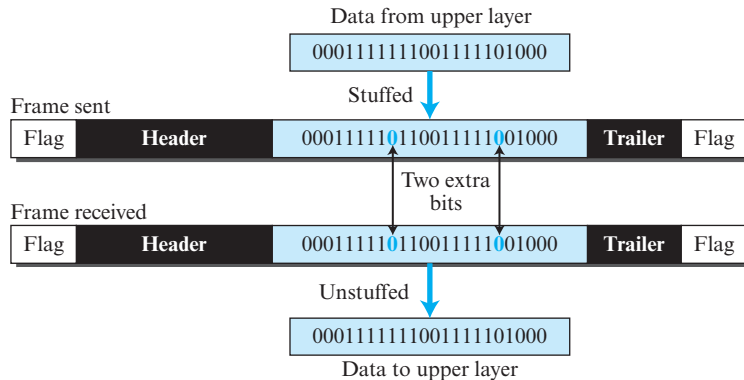
This flag can create the same type of problem we saw in the character-oriented protocols. That is, if the flag pattern appears in the data, we need to somehow inform the receiver that this is not the end of the frame. We do this by stuffing 1 single bit (instead of 1 byte) to prevent the pattern from looking like a flag. The strategy is called **bit stuffing**. In bit stuffing, if a 0 and five consecutive 1 bits are encountered, an extra 0 is added. This extra stuffed bit is eventually removed from the data by the receiver.

Figure 5.6 A frame in a bit-oriented protocol

Note that the extra bit is added after one 0 followed by five 1s regardless of the value of the next bit. This guarantees that the flag field sequence does not inadvertently appear in the frame.

Bit stuffing is the process of adding one extra 0 whenever five consecutive 1s follow a 0 in the data, so that the receiver does not mistake the pattern 0111110 for a flag.

Figure 5.7 shows bit stuffing at the sender and bit removal at the receiver. Note that even if we have a 0 after five 1s, we still stuff a 0. The 0 will be removed by the receiver.

Figure 5.7 Bit stuffing and unstuffing

This means that if the flaglike pattern 01111110 appears in the data, it will change to 011111010 (stuffed) and is not mistaken for a flag by the receiver. The real flag 01111110 is not stuffed by the sender and is recognized by the receiver.

5.2.2 Flow and Error Control

We defined flow and error control in Chapter 3. One of the responsibilities of the data-link control sublayer is flow and error control at the data-link layer.

Flow Control

As we discussed in the transport layer (Chapter 3), flow control coordinates the amount of data that can be sent before receiving an acknowledgment. In the data-link layer, flow control is one of the duties of the data link control sublayer. The idea of flow control in

the data-link layer follows the same principle we discuss for the transport layer. Although at the transport layer flow control is end-to-end (host-to-host), flow control at the data-link layer is node-to-node, across the link.

Error Control

Error control is both error detection and error correction. It allows the receiver to inform the sender of any frames lost or damaged in transmission and coordinates the retransmission of those frames by the sender. In the data-link layer, the term **error control** refers primarily to methods of error detection and retransmission. Error control in the data-link layer is often implemented simply: Any time an error is detected in an exchange, corrupted frames are retransmitted. We need, however, to emphasize that the error control at the transport layer is end-to-end, but the error control at the data-link layer is node-to-node, across the link. In other words, each time a frame passes through a link, we need to be sure that it is not corrupted.

5.2.3 Error Detection and Correction

At the data-link layer, if a frame is corrupted between the two nodes, it needs to be corrected before it continues its journey to other nodes. However, most link-layer protocols simply discard the frame and let the upper-layer protocols handle the retransmission of the frame. Some wireless protocols, however, try to correct the corrupted frame.

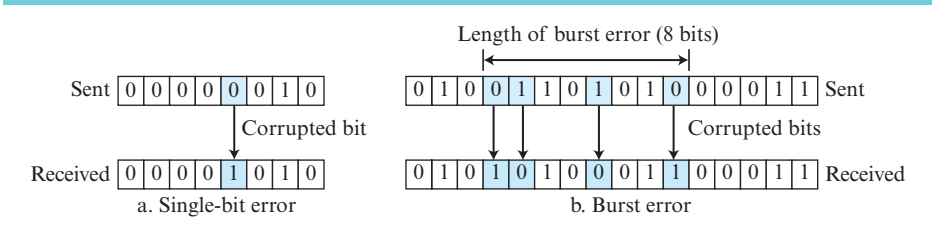
Introduction

Let us first discuss some issues related, directly or indirectly, to error detection and correction.

Types of Errors

Whenever bits flow from one point to another, they are subject to unpredictable changes because of **interference**. This interference can change the shape of the signal. The term **single-bit error** means that only 1 bit of a given data unit (such as a byte, character, or packet) is changed from 1 to 0 or from 0 to 1. The term **burst error** means that 2 or more bits in the data unit have changed from 1 to 0 or from 0 to 1. Figure 5.8 shows the effect of a single-bit and burst error on a data unit.

Figure 5.8 Single-bit and burst error



A burst error is more likely to occur than a single-bit error because the duration of the noise signal is normally longer than the duration of 1 bit, which means that when noise affects data, it affects a set of bits. The number of bits affected depends on the data rate and duration of noise. For example, if we are sending data at 1 kbps, a noise of

1/100 seconds can affect 10 bits; if we are sending data at 1 Mbps, the same noise can affect 10,000 bits.

Redundancy

The central concept in detecting or correcting errors is *redundancy*. To be able to detect or correct errors, we need to send some extra bits with our data. These redundant bits are added by the sender and removed by the receiver. Their presence allows the receiver to detect or correct corrupted bits.

Detection versus Correction

The correction of errors is more difficult than the detection. In *error detection*, we are looking only to see if any error has occurred. The answer is a simple yes or no. We are not even interested in the number of corrupted bits. A single-bit error is the same for us as a burst error. In *error correction*, we need to know the exact number of bits that are corrupted and, more importantly, their location in the message. The number of the errors and the size of the message are important factors. If we need to correct a single error in an 8-bit data unit, we need to consider eight possible error locations; if we need to correct two errors in a data unit of the same size, we need to consider 28 (permutation of 8 by 2) possibilities. You can imagine the receiver's difficulty in finding 10 errors in a data unit of 1000 bits. We concentrate on error detection; error correction is more difficult, but it is discussed briefly in Chapter 8.

Coding

Redundancy is achieved through various coding schemes. The sender adds redundant bits through a process that creates a relationship between the redundant bits and the actual data bits. The receiver checks the relationships between the two sets of bits to detect errors. The ratio of redundant bits to the data bits and the robustness of the process are important factors in any coding scheme.

We can divide coding schemes into two broad categories: *block coding* and *convolution coding*. In this book, we concentrate on block coding; convolution coding is more complex and beyond the scope of this book.

Block Coding

In block coding, we divide our message into blocks, each of k bits, called **datawords**. We add r redundant bits to each block to make the length $n = k + r$. The resulting n -bit blocks are called **codewords**. How the extra r bits are chosen or calculated is something we will discuss later. For the moment, it is important to know that we have a set of datawords, each of size k , and a set of codewords, each of size of n . With k bits, we can create a combination of 2^k datawords; with n bits, we can create a combination of 2^n codewords. Since $n > k$, the number of possible codewords is larger than the number of possible datawords. The block coding process is one-to-one; the same dataword is always encoded as the same codeword. This means that we have $2^n - 2^k$ codewords that are not used. We call these codewords invalid or illegal. The trick in error detection is the existence of these invalid codes, as we discuss next. If the receiver receives an invalid codeword, this indicates that the data was corrupted during transmission.

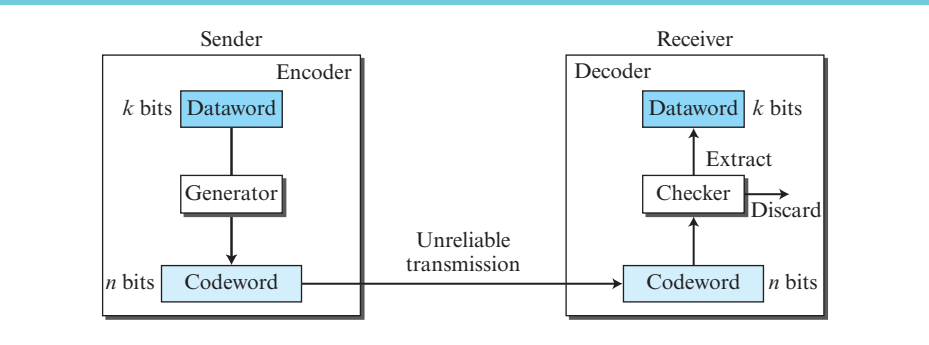
Error Detection

How can errors be detected by using block coding? If the following two conditions are met, the receiver can detect a change in the original codeword.

1. The receiver has (or can find) a list of valid codewords.
2. The original codeword has changed to an invalid one.

Figure 5.9 shows the role of block coding in error detection.

Figure 5.9 *Process of error detection in block coding*



The sender creates codewords out of datawords by using a generator that applies the rules and procedures of encoding (discussed later). Each codeword sent to the receiver may change during transmission. If the received codeword is the same as one of the valid codewords, the word is accepted; the corresponding dataword is extracted for use. If the received codeword is not valid, it is discarded. However, if the codeword is corrupted during transmission but the received word still matches a valid codeword, the error remains undetected.

Example 5.1

Let us assume that $k = 2$ and $n = 3$. Table 5.1 shows the list of datawords and codewords. Later, we will see how to derive a codeword from a dataword.

Table 5.1 *A code for error detection in Example 5.1*

<i>Datawords</i>	<i>Codewords</i>	<i>Datawords</i>	<i>Codewords</i>
00	000	10	101
01	011	11	110

Assume the sender encodes the dataword 01 as 011 and sends it to the receiver. Consider the following cases:

1. The receiver receives 011. It is a valid codeword. The receiver extracts the dataword 01 from it.
2. The codeword is corrupted during transmission, and 111 is received (the leftmost bit is corrupted). This is not a valid codeword and is discarded.
3. The codeword is corrupted during transmission, and 000 is received (the right two bits are corrupted). This is a valid codeword. The receiver incorrectly extracts the dataword 00. Two corrupted bits have made the error undetectable.

An error-detecting code can detect only the types of errors for which it is designed; other types of errors may remain undetected.

Hamming Distance

One of the central concepts in coding for error control is the idea of the Hamming distance. The **Hamming distance** between two words (of the same size) is the number of differences between the corresponding bits. We show the Hamming distance between two words x and y as $d(x, y)$. We may wonder why Hamming distance is important for error detection. The reason is that the Hamming distance between the received codeword and the sent codeword is the number of bits that are corrupted during transmission. For example, if the codeword 00000 is sent and 01101 is received, 3 bits are in error and the Hamming distance between the two is $d(00000, 01101) = 3$. In other words, if the Hamming distance between the sent and the received codeword is not zero, the codeword has been corrupted during transmission.

The Hamming distance can easily be found if we apply the XOR operation ($\oplus$) on the two words and count the number of 1s in the result. Note that the Hamming distance is a value greater than or equal to zero.

The Hamming distance between two words is the number of differences between corresponding bits.

Example 5.2

Let us find the Hamming distance between two pairs of words.

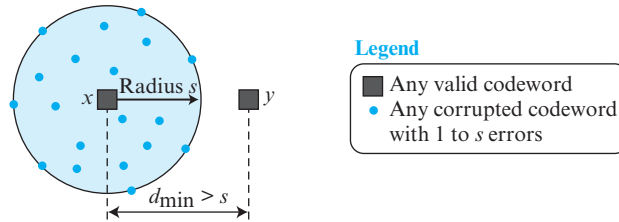
1. The Hamming distance $d(000, 011)$ is 2 because $(000 \oplus 011)$ is 011 (two 1s).
2. The Hamming distance $d(10101, 11110)$ is 3 because $(10101 \oplus 11110)$ is 01011 (three 1s).

Minimum Hamming Distance for Error Detection

In a set of codewords, the minimum Hamming distance is the smallest Hamming distance between all possible pairs of codewords. Now let us find the minimum Hamming distance in a code if we want to be able to detect up to s errors. If s errors occur during transmission, the Hamming distance between the sent codeword and received codeword is s . If our system is to detect up to s errors, the minimum distance between the valid codes must be $(s + 1)$, so that the received codeword does not match a valid codeword. In other words, if the minimum distance between all valid codewords is $(s + 1)$, the received codeword cannot be erroneously mistaken for another codeword. The error will be detected. We need to clarify a point here: Although a code with $d_{\min} = s + 1$ may be able to detect more than s errors in some special cases, only s or fewer errors are guaranteed to be detected.

To guarantee the detection of up to s errors in all cases, the minimum Hamming distance in a block code must be $d_{\min} = s + 1$.

We can look at this criteria geometrically. Let us assume that the sent codeword x is at the center of a circle with radius s . All received codewords that are created by 0 to s errors are points inside the circle or on the perimeter of the circle. All other valid codewords must be outside the circle, as shown in Figure 5.10. This means that $d_{\min}$ must be an integer greater than s or $d_{\min} = s + 1$.

Figure 5.10 Geometric concept explaining $d_{\min}$ in error detection**Example 5.3**

The minimum Hamming distance for our first code scheme (Table 5.1) is 2. This code guarantees detection of only a single error. For example, if the third codeword (101) is sent and one error occurs, the received codeword does not match any valid codeword. If two errors occur, however, the received codeword may match a valid codeword and the errors are not detected.

Example 5.4

A code scheme has a Hamming distance $d_{\min} = 4$. This code guarantees the detection of up to three errors ($d = s + 1$ or $s = 3$).

Linear Block Codes

Almost all block codes used today belong to a subset of block codes called *linear block codes*. The use of nonlinear block codes for error detection and correction is not as widespread because their structure makes theoretical analysis and implementation difficult. We therefore concentrate on linear block codes. The formal definition of linear block codes requires the knowledge of abstract algebra (particularly Galois fields), which is beyond the scope of this book. We therefore give an informal definition. For our purposes, a linear block code is a code in which the exclusive OR (addition modulo-2) of two valid codewords creates another valid codeword.

Example 5.5

The code in Table 5.1 is a linear block code because the result of XORing any codeword with any other codeword is a valid codeword. For example, the XORing of the second and third codewords creates the fourth one.

Minimum Distance for Linear Block Codes

It is simple to find the minimum Hamming distance for a linear block code. The minimum Hamming distance is the number of 1s in the nonzero valid codeword with the smallest number of 1s.

Example 5.6

In our first code (Table 5.1), the numbers of 1s in the nonzero codewords are 2, 2, and 2. So the minimum Hamming distance is $d_{\min} = 2$.

Parity-Check Code

Perhaps the most familiar error-detecting code is the **parity-check code**. This code is a linear block code. In this code, a k -bit dataword is changed to an n -bit codeword where

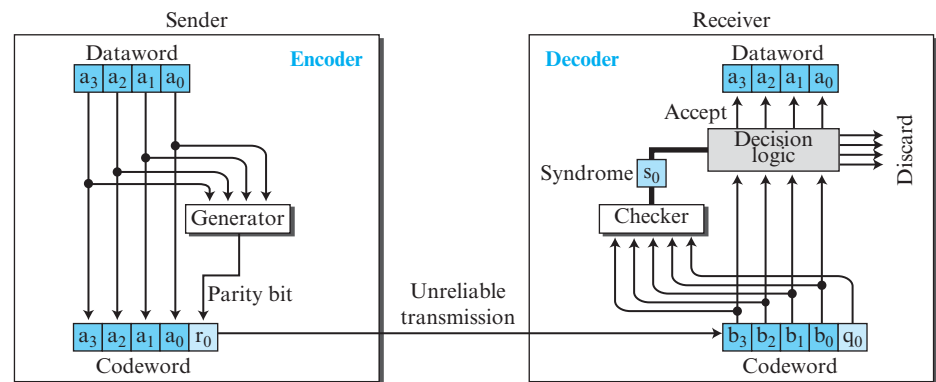
$n = k + 1$. The extra bit, called the *parity bit*, is selected to make the total number of 1s in the codeword even. Although some implementations specify an odd number of 1s, we discuss the even case. The minimum Hamming distance for this category is $d_{\min} = 2$, which means that the code is a single-bit error-detecting code. Our first code (Table 5.1) is a parity-check code ($k = 2$ and $n = 3$). The code in Table 5.2 is also a parity-check code with $k = 4$ and $n = 5$.

Table 5.2 Simple parity-check code $C(5, 4)$

Datawords	Codewords	Datawords	Codewords
0000	00000	1000	10001
0001	00011	1001	10010
0010	00101	1010	10100
0011	00110	1011	10111
0100	01001	1100	11000
0101	01010	1101	11011
0110	01100	1110	11101
0111	01111	1111	11110

Figure 5.11 shows a possible structure of an encoder (at the sender) and a decoder (at the receiver).

Figure 5.11 Encoder and decoder for simple parity-check code



The encoder uses a generator that takes a copy of a 4-bit dataword (a_0 , a_1 , a_2 , and a_3) and generates a parity bit r_0 . The dataword bits and the parity bit create the 5-bit codeword. The parity bit that is added makes the number of 1s in the codeword even. This is normally done by adding the 4 bits of the dataword (modulo-2); the result is the parity bit. In other words,

$$r_0 = a_3 + a_2 + a_1 + a_0 \quad (\text{modulo-2})$$

If the number of 1s is even, the result is 0; if the number of 1s is odd, the result is 1. In both cases, the total number of 1s in the codeword is even.

The sender sends the codeword, which may be corrupted during transmission. The receiver receives a 5-bit word. The checker at the receiver does the same thing as the generator in the sender with one exception: The addition is done over all 5 bits. The result, which is called the **syndrome**, is just 1 bit. The syndrome is 0 when the number of 1s in the received codeword is even; otherwise, it is 1.

$$s_0 = b_3 + b_2 + b_1 + b_0 + q_0 \quad (\text{modulo-2})$$

The syndrome is passed to the decision logic analyzer. If the syndrome is 0, there is no detectable error in the received codeword; the data portion of the received codeword is accepted as the dataword; if the syndrome is 1, the data portion of the received codeword is discarded. The dataword is not created.

Example 5.7

Let us look at some transmission scenarios. Assume the sender sends the dataword 1011. The codeword created from this dataword is 10111, which is sent to the receiver. We examine five cases:

1. No error occurs; the received codeword is 10111. The syndrome is 0. The dataword 1011 is created.
2. One single-bit error changes a_1 . The received codeword is 10**0**11. The syndrome is 1. No dataword is created.
3. One single-bit error changes r_0 . The received codeword is 1011**0**. The syndrome is 1. No dataword is created. Note that although none of the dataword bits are corrupted, no dataword is created because the code is not sophisticated enough to show the position of the corrupted bit.
4. An error changes r_0 and a second error changes a_3 . The received codeword is **0**011**0**. The syndrome is 0. The dataword 0011 is created at the receiver. Note that here the dataword is wrongly created due to the syndrome value. The simple parity-check decoder cannot detect an even number of errors. The errors cancel each other out and give the syndrome a value of 0.
5. Three bits— a_3 , a_2 , and a_1 —are changed by errors. The received codeword is **0**1**0**11. The syndrome is 1. The dataword is not created. This shows that the simple parity check, guaranteed to detect one single error, can also find any odd number of errors.

A parity-check code can detect an odd number of errors.

Cyclic Codes

Cyclic codes are special linear block codes with one extra property. In a **cyclic code**, if a codeword is cyclically shifted (rotated), the result is another codeword. For example, if 1011000 is a codeword and we cyclically left-shift, then 0110001 is also a codeword. In this case, if we call the bits in the first word a_0 to a_6 , and the bits in the second word b_0 to b_6 , we can shift the bits by using the following:

$$b_1 = a_0 \quad b_2 = a_1 \quad b_3 = a_2 \quad b_4 = a_3 \quad b_5 = a_4 \quad b_6 = a_5 \quad b_0 = a_6$$

In the rightmost equation, the last bit of the first word is wrapped around and becomes the first bit of the second word.

Cyclic Redundancy Check

We can create cyclic codes to correct errors. However, the theoretical background required is beyond the scope of this book. In this section, we simply discuss a subset of

cyclic codes called the **cyclic redundancy check (CRC)** that is used in networks such as LANs and WANs.

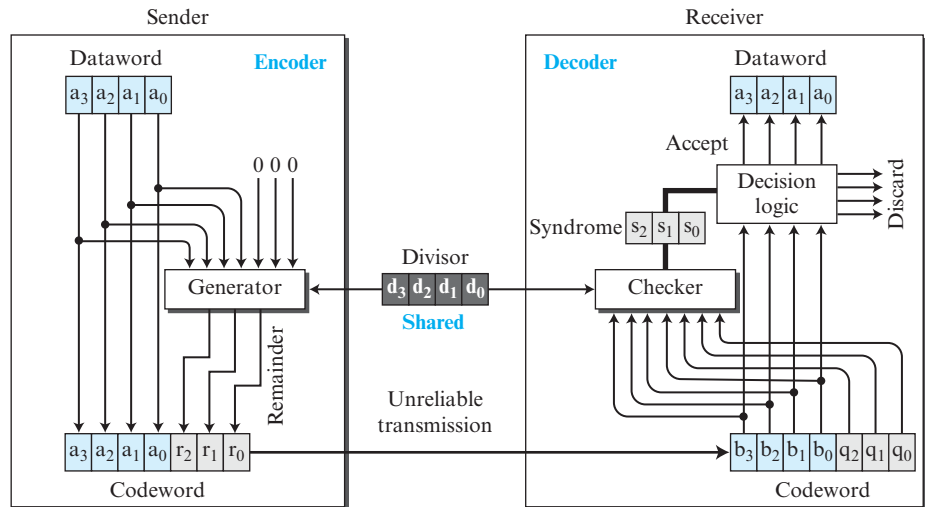
Table 5.3 shows an example of a CRC code. We can see both the linear and cyclic properties of this code.

Table 5.3 A CRC code with $C(7, 4)$

Dataword	Codeword	Dataword	Codeword
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

Figure 5.12 shows one possible design for the encoder and decoder.

Figure 5.12 CRC encoder and decoder



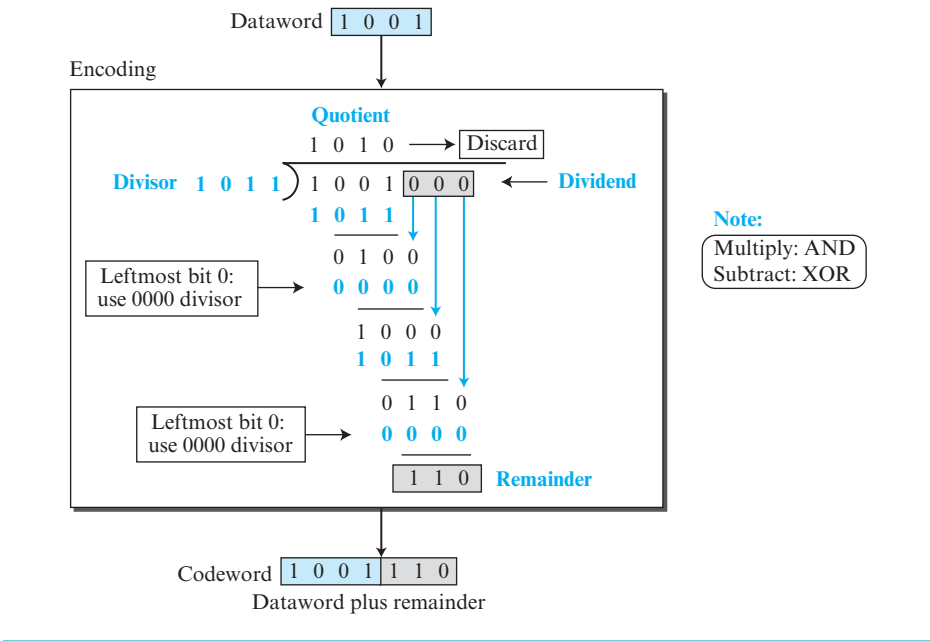
In the encoder, the dataword has k bits (4 here); the codeword has n bits (7 here). The size of the dataword is augmented by adding $n - k$ (3 here) 0s to the right-hand side of the word. The n -bit result is fed into the generator. The generator uses a divisor of size $n - k + 1$ (4 here), predefined and agreed upon. The generator divides the augmented dataword by the divisor (modulo-2 division). The quotient of the division is discarded; the remainder ($r_2r_1r_0$) is appended to the dataword to create the codeword.

The decoder receives the codeword (possibly corrupted in transition). A copy of all n bits is fed to the checker, which is a replica of the generator. The remainder produced by the checker is a syndrome of $n - k$ (3 here) bits, which is fed to the decision logic

analyzer. The analyzer has a simple function. If the syndrome bits are all 0s, the 4 leftmost bits of the codeword are accepted as the dataword (interpreted as no error); otherwise, the 4 bits are discarded (error).

Encoder Let us take a closer look at the encoder. The encoder takes a dataword and augments it with $n - k$ number of 0s. It then divides the augmented dataword by the divisor, as shown in Figure 5.13.

Figure 5.13 Division in CRC encoder



The process of modulo-2 binary division is the same as the familiar division process we use for decimal numbers. However, addition and subtraction in this case are the same; we use the XOR operation to do both.

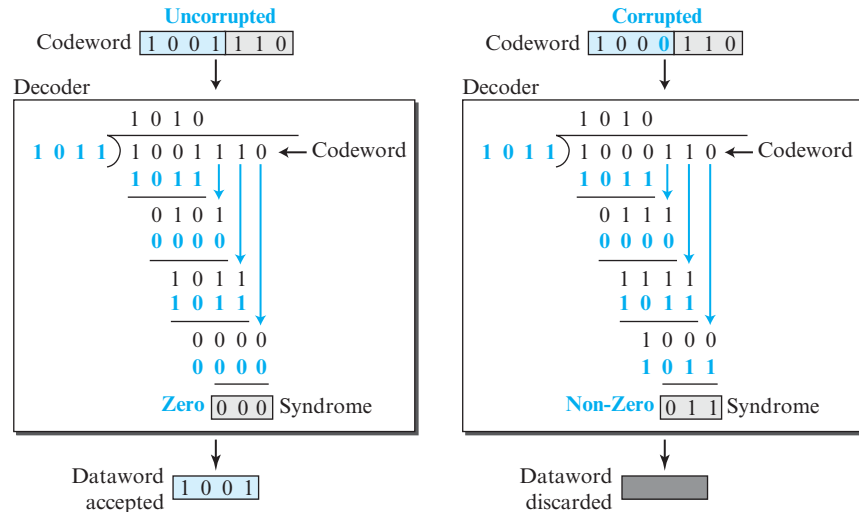
As in decimal division, the process is done step by step. In each step, a copy of the divisor is XORed with the 4 bits of the dividend. The result of the XOR operation (remainder) is 3 bits (in this case), which is used for the next step after 1 extra bit is pulled down to make it 4 bits long. There is one important point we need to remember in this type of division. If the leftmost bit of the dividend (or the part used in each step) is 0, the step cannot use the regular divisor; we need to use an all-0s divisor.

When there are no bits left to pull down, we have a result. The 3-bit remainder forms the check bits (r_2 , r_1 , and r_0). They are appended to the dataword to create the codeword.

Decoder The codeword can change during transmission. The decoder does the same division process as the encoder. The remainder of the division is the syndrome. If the syndrome is all 0s, there is no error with a high probability; the dataword is separated from the received codeword and accepted. Otherwise, everything is discarded.

Figure 5.14 shows two cases: The left-hand figure shows the value of the syndrome when no error has occurred; the syndrome is 000. The right-hand part of the figure shows the case in which there is a single error. The syndrome is not all 0s (it is 011).

Figure 5.14 Division in the CRC decoder for two cases



Divisor We may be wondering how the divisor 1011 is chosen. This depends on the expectation we have from the code. We discuss the criteria on the book website. Some of the standard divisors used in networking are shown in Table 5.4. The number in the name of the divisor (for example, CRC-32) refers to the degree of polynomial (the highest power) representing the divisor. The number of bits is always one more than the degree of polynomial. For example, CRC-8 has 9 bits and CRC-32 has 33 bits.

Table 5.4 Standard polynomials

Name	Binary	Application
CRC-8	100000111	ATM header
CRC-10	11000110101	ATM AAL
CRC-16	10001000000100001	HDLC
CRC-32	100000100110000010001110110110111	LANs

Polynomials

A better way to understand cyclic codes and how they can be analyzed is to represent them as polynomials. We discuss polynomials on the book website for interested readers.

Requirement

We can mathematically prove that a bit pattern needs to have at least two properties to be considered as a generator (divisor):

1. The pattern should have at least two bits.
2. The rightmost and the leftmost bits should both be 1s.

Performance

We can mathematically prove the performance of the CRC as shown below.

- ❑ **Single Errors.** All qualified generators (see above) can detect any single-bit error.
- ❑ **Odd Number of Errors.** All qualified generators can detect any odd number of errors if the generator can be evenly divided by $(11)_2$ using binary division in modulo 2 arithmetic; otherwise, only some odd number errors will be detected.
- ❑ **Burst Errors.** If we assume the length of the burst error is L bits and r is the length of the remainder (r is the length of the generator minus 1; it is also the value of the highest power in the polynomial representing the generator):
 - a. All burst errors of the size $L \leq r$ are detected.
 - b. All burst errors of the size $L = r + 1$ are detected with probability $1 - (0.5)^{r-1}$.
 - c. All burst errors of the size $L > r + 1$ are detected with probability $1 - (0.5)^r$.

Advantages of Cyclic Codes

Cyclic codes can easily be implemented in hardware and software. They are especially fast when implemented in hardware. This has made cyclic codes a good candidate for many networks. In the book website, we show how division can be done by a shift register that is included in the hardware of the node.

Checksum

Checksum is an error-detecting technique that can be applied to a message of any length. In the Internet, the checksum technique is mostly used at the network and transport layer rather than the data-link layer. However, to make our discussion of error-detecting techniques complete, we discuss the checksum in this chapter.

At the source, the message is first divided into m -bit units. The generator then creates an extra m -bit unit called the *checksum*, which is sent with the message. At the destination, the checker creates a new checksum from the combination of the message and sent checksum. If the new checksum is all 0s, the message is accepted; otherwise, the message is discarded (Figure 5.15). Note that in the real implementation, the checksum unit is not necessarily added at the end of the message; it can be inserted in the middle of the message.

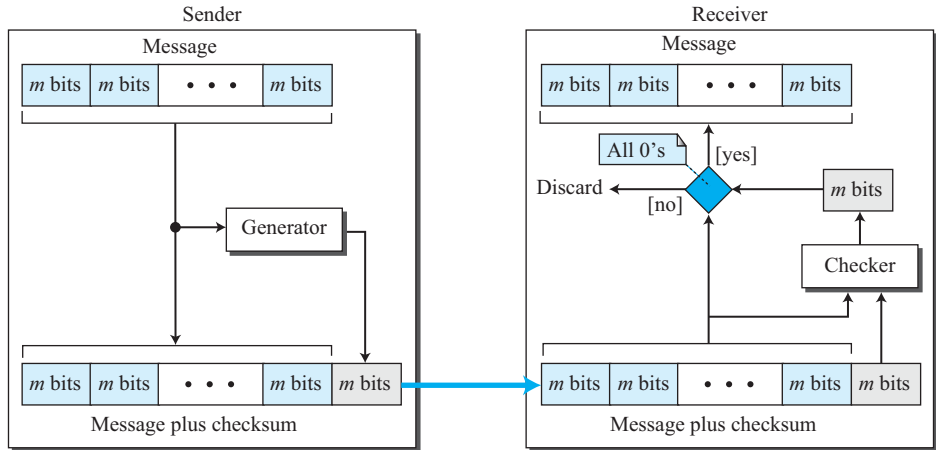
Concept

The idea of the traditional checksum is simple. We show this using a simple example.

Example 5.8

Suppose the message is a list of five 4-bit numbers that we want to send to a destination. In addition to sending these numbers, we send the sum of the numbers. For example, if the set of numbers is (7, 11, 12, 0, 6), we send (7, 11, 12, 0, 6, **36**), where 36 is the sum of the original numbers. The receiver adds the five numbers and compares the result with the sum. If the two are the same, the receiver assumes no error, accepts the five numbers, and discards the sum. Otherwise, there is an error somewhere and the message not accepted.

One's Complement Addition The previous example has one major drawback. Each number can be written as a 4-bit word (each is less than 15) except for the sum. One solution is to use **one's complement** arithmetic. In this arithmetic, we can represent unsigned numbers between 0 and $2^m - 1$ using only m bits. If the number has

Figure 5.15 Checksum

more than m bits, the extra leftmost bits need to be added to the m rightmost bits (wrapping).

Example 5.9

In the previous example, the decimal number 36 in binary is $(100100)_2$. To change it to a 4-bit number we add the extra leftmost bit to the right four bits as shown below.

$$(10)_2 + (0100)_2 = (0110)_2 \rightarrow (6)_{10}$$

Instead of sending 36 as the sum, we can send 6 as the sum (7, 11, 12, 0, 6, 6). The receiver can add the first five numbers in one's complement arithmetic. If the result is 6, the numbers are accepted; otherwise, they are rejected.

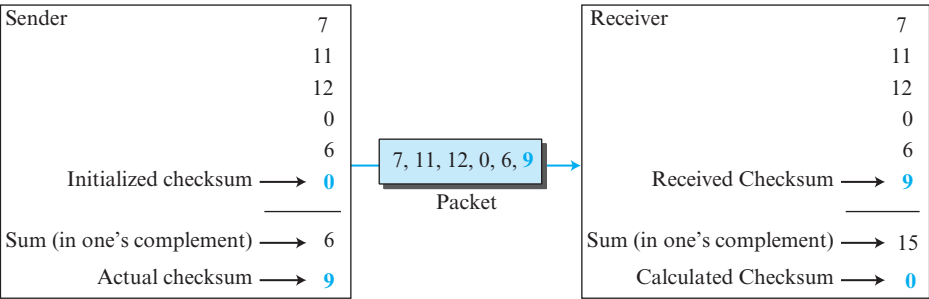
Checksum We can make the job of the receiver easier if we send the complement of the sum, called the *checksum*. In one's complement arithmetic, the complement of a number is found by completing all bits (changing all 1s to 0s and all 0s to 1s). This is the same as subtracting the number from $2^m - 1$. In one's complement arithmetic, we have two 0s: one positive and one negative, which are complements of each other. The positive zero has all m bits set to 0; the negative zero has all bits set to 1 (it is $2^m - 1$). If we add a number with its complement, we get a negative zero (a number with all bits set to 1). When the receiver adds all five numbers (including the checksum), it gets a negative zero. The receiver can complement the result again to get a positive zero.

Example 5.10

Let us use the idea of the checksum in Example 5.9. The sender adds all five numbers in one's complement to get the sum = 6. The sender then complements the result to get the checksum = 9, which is $15 - 6$. Note that $6 = (0110)_2$ and $9 = (1001)_2$; they are complements of each other. The sender sends the five data numbers and the checksum (7, 11, 12, 0, 6, 9). If there is no corruption in transmission, the receiver receives (7, 11, 12, 0, 6, 9) and adds them in one's complement to get 15.

The sender complements 15 to get 0. This shows that data have not been corrupted. Figure 5.16 shows the process.

Figure 5.16 Example 5.10



Internet Checksum

Traditionally, the Internet has used a 16-bit checksum. The sender and the receiver follow the steps depicted in Table 5.5. The sender or the receiver uses five steps.

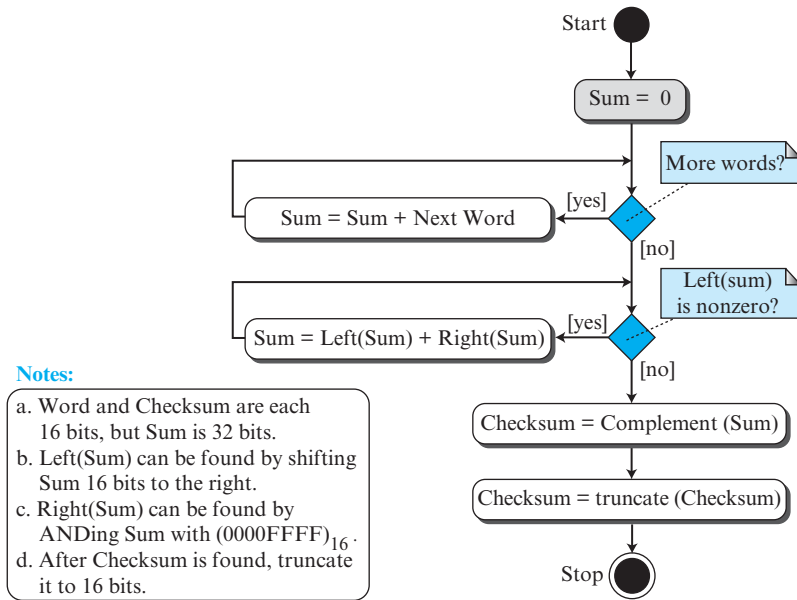
Table 5.5 Procedure to calculate the traditional checksum

Sender	Receiver
1. The message is divided into 16-bit words.	1. The message and the checksum is received.
2. The value of the checksum word is initially set to zero.	2. The message is divided into 16-bit words.
3. All words including the checksum are added using one's complement addition.	3. All words are added using one's complement addition.
4. The sum is complemented and becomes the checksum.	4. The sum is complemented and becomes the new checksum.
5. The checksum is sent with the data.	5. If the value of the checksum is 0, the message is accepted; otherwise, it is rejected.

Algorithm

We can use the flow diagram of Figure 5.17 to show the algorithm for calculation of the checksum. A program in any language can easily be written based on the algorithm. Note that the first loop just calculates the sum of the data units in two's complement; the second loop wraps the extra bits created from the two's complement calculation to simulate the calculations in one's complement. This is needed because almost all computers today do calculation in two's complement.

Performance The traditional checksum uses a small number of bits (16) to detect errors in a message of any size (sometimes thousands of bits). However, it is not as strong as the CRC in error-checking capability. For example, if the value of one word is incremented and the value of another word is decremented by the same amount, the two errors cannot be detected because the sum and checksum remain the same. Also, if the values of several words are incremented but the sum and the checksum do not change,

Figure 5.17 Algorithm to calculate a traditional checksum

the errors are not detected. Fletcher and Adler have proposed some weighted checksums that eliminate the first problem. However, the tendency in the Internet, particularly in designing new protocols, is to replace the checksum with a CRC.

Other Approaches to the Checksum

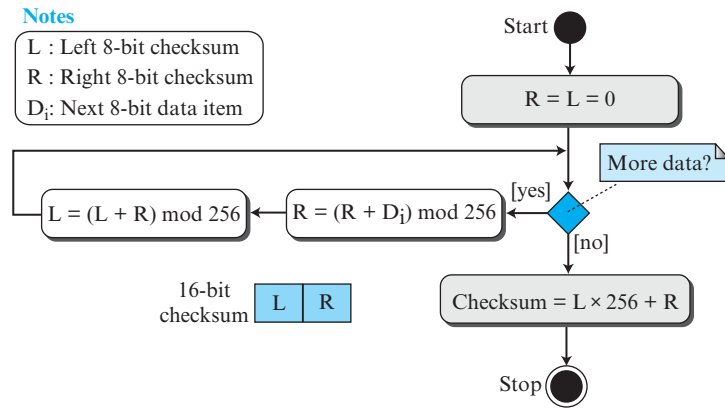
As mentioned before, there is one major problem with the traditional checksum calculation. If two 16-bit items are transposed in transmission, the checksum cannot catch this error. The reason is that the traditional checksum is not weighted: it treats each data item equally. In other words, the order of data items is immaterial to the calculation. Several approaches have been used to prevent this problem. We mention two of them here: Fletcher and Adler.

Fletcher Checksum The Fletcher checksum was devised to weight each data item according to its position. Fletcher has proposed two algorithms: 8-bit and 16-bit. The first, 8-bit Fletcher, calculates on 8-bit data items and creates a 16-bit checksum. The second, 16-bit Fletcher, calculates on 16-bit data items and creates a 32-bit checksum.

The 8-bit Fletcher is calculated over data octets (bytes) and creates a 16-bit checksum. The calculation is done modulo 256 (2^8), which means the intermediate results are divided by 256 and the remainder is kept. The algorithm uses two accumulators, L and R. The first simply adds data items together; the second adds a weight to the calculation. There are many variations of the 8-bit Fletcher algorithm; we show a simple one in Figure 5.18.

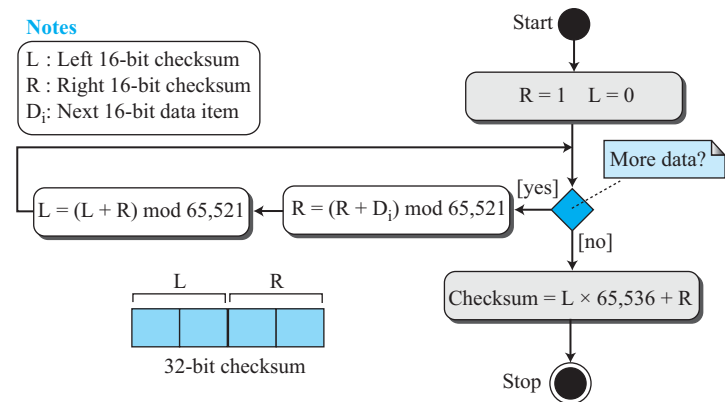
The 16-bit Fletcher checksum is similar to the 8-bit Fletcher checksum, but it is calculated over 16-bit data items and creates a 32-bit checksum. The calculation is done modulo 65,536.

Figure 5.18 Algorithm to calculate an 8-bit Fletcher checksum



Adler Checksum The Adler checksum is a 32-bit checksum. Figure 5.19 shows a simple algorithm in flowchart form. It is similar to the 16-bit Fletcher with three differences. First, calculation is done on single bytes instead of 2 bytes at a time. Second, the modulus is a prime number (65,521) instead of 65,536. Third, L is initialized to 1 instead of 0. It has been proved that a prime modulo has a better detecting capability in some combinations of data.

Figure 5.19 Algorithm to calculate an Adler checksum



5.2.4 Two DLC Protocols

After finishing all issues related to the DLC sublayer, we discuss two DLC protocols that actually implemented those concepts. The first, HDLC, is the base of many protocols that have been designed for LANs. The second, Point-to-Point, is a protocol derived from HDLC and is used for point-to-point links.

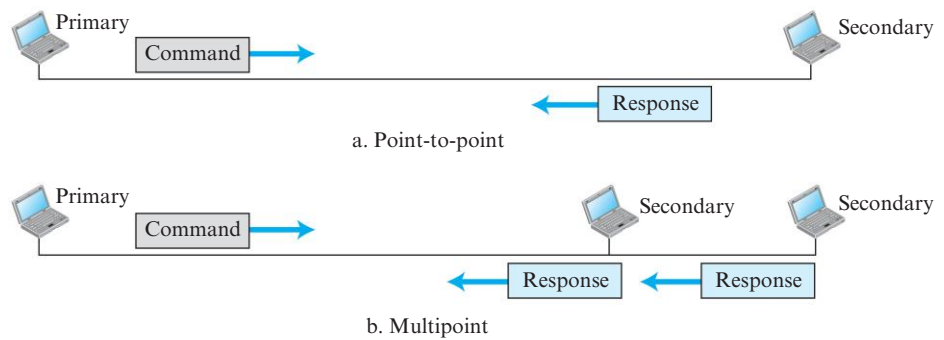
HDLC

High-level Data Link Control (HDLC) is a bit-oriented protocol for communication over point-to-point and multipoint links. It implements the Stop-and-Wait protocol we discussed in Chapter 3.

Configurations and Transfer Modes

HDLC provides two common transfer modes that can be used in different configurations: *normal response mode (NRM)* and *asynchronous balanced mode (ABM)*. In *normal response mode (NRM)*, the station configuration is unbalanced. We have one primary station and multiple secondary stations. A *primary station* can send commands; a *secondary station* can only respond. The NRM is used for both point-to-point and multipoint links, as shown in Figure 5.20.

Figure 5.20 Normal response mode



In ABM, the configuration is balanced. The link is point-to-point, and each station can function as a primary and a secondary (acting as peers), as shown in Figure 5.21. This is the common mode today.

Figure 5.21 Asynchronous balanced mode

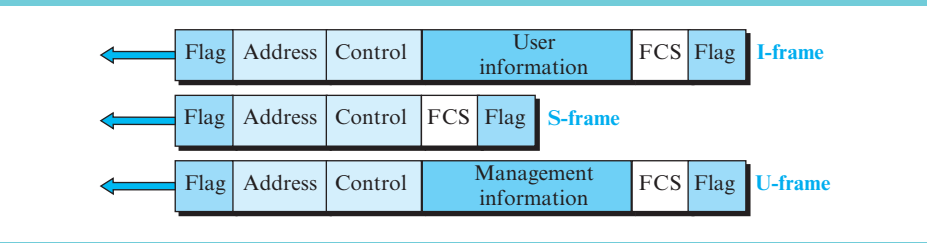


Frames

To provide the flexibility necessary to support all the options possible in the modes and configurations just described, HDLC defines three types of frames: *information frames (I-frames)*, *supervisory frames (S-frames)*, and *unnumbered frames (U-frames)*. Each type of frame serves as an envelope for the transmission of a different type of message. I-frames are used to transport user data and control information relating to user data (piggy-backing). S-frames are used only to transport control information. U-frames are reserved for system management. Information carried by U-frames is intended for managing the

link itself. Each frame in HDLC may contain up to six fields, as shown in Figure 5.22: a beginning flag field, an address field, a control field, an information field, a frame check sequence (FCS) field, and an ending flag field. In multiple-frame transmissions, the ending flag of one frame can serve as the beginning flag of the next frame.

Figure 5.22 HDLC frames

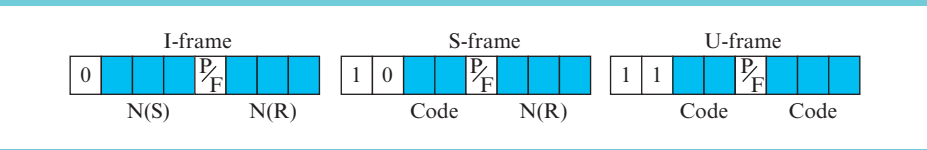


Let us now discuss the fields and their use in different frame types.

- ❑ **Flag field.** This field contains synchronization pattern 01111110, which identifies both the beginning and the end of a frame.
- ❑ **Address field.** This field contains the address of the secondary station. If a primary station created the frame, it contains a *to* address. If a secondary station creates the frame, it contains a *from* address. The address field can be one byte or several bytes long, depending on the needs of the network.
- ❑ **Control field.** The control field is one or two bytes used for flow and error control. The interpretation of bits are discussed later.
- ❑ **Information field.** The information field contains the user's data from the network layer or management information. Its length can vary from one network to another.
- ❑ **FCS field.** The frame check sequence (FCS) is the HDLC error detection field. It can contain either a 2- or 4-byte CRC.

The control field determines the type of frame and defines its functionality. So let us discuss the format of this field in detail. The format is specific for the type of frame, as shown in Figure 5.23.

Figure 5.23 Control field format for the different frame types



Control Field for I-Frames I-frames are designed to carry user data from the network layer. In addition, they can include flow- and error-control information (piggybacking). The subfields in the control field are used to define these functions. The first bit defines the type. If the first bit of the control field is 0, this means the frame is an I-frame. The next 3 bits, called *N(S)*, define the sequence number of the frame. Note that with 3 bits, we can define a sequence number between 0 and 7. The last 3

bits, called $N(R)$, correspond to the acknowledgment number when piggybacking is used. The single bit between $N(S)$ and $N(R)$ is called the P/F bit. The P/F field is a single bit with a dual purpose. It has meaning only when it is set (bit = 1) and can mean poll or final. It means *poll* when the frame is sent by a primary station to a secondary (when the address field contains the address of the receiver). It means *final* when the frame is sent by a secondary to a primary (when the address field contains the address of the sender).

Control Field for S-Frames Supervisory frames are used for flow and error control whenever piggybacking is either impossible or inappropriate. S-frames do not have information fields. If the first 2 bits of the control field are 10, this means the frame is an S-frame. The last 3 bits, called $N(R)$, corresponds to the acknowledgment number (ACK) or negative acknowledgment number (NAK) depending on the type of S-frame. The 2 bits called *code* are used to define the type of S-frame itself. With 2 bits, we can have four types of S-frames, as described below:

- ❑ **Receive ready (RR).** If the value of the code subfield is 00, it is an RR S-frame. This kind of frame acknowledges the receipt of a safe and sound frame or group of frames. In this case, the value of the $N(R)$ field defines the acknowledgment number.
- ❑ **Receive not ready (RNR).** If the value of the code subfield is 10, it is an RNR S-frame. This kind of frame is an RR frame with additional functions. It acknowledges the receipt of a frame or group of frames, and it announces that the receiver is busy and cannot receive more frames. It acts as a kind of congestion-control mechanism by asking the sender to slow down. The value of $N(R)$ is the acknowledgment number.
- ❑ **Reject (REJ).** If the value of the code subfield is 01, it is an REJ S-frame. This is a NAK frame, but not like the one used for Selective Repeat ARQ. It is a NAK that can be used in Go-Back- N ARQ to improve the efficiency of the process by informing the sender, before the sender timer expires, that the last frame is lost or damaged. The value of $N(R)$ is the negative acknowledgment number.
- ❑ **Selective reject (SREJ).** If the value of the code subfield is 11, it is an SREJ S-frame. This is a NAK frame used in Selective Repeat ARQ. Note that the HDLC Protocol uses the term *selective reject* instead of *selective repeat*. The value of $N(R)$ is the negative acknowledgment number.

Control Field for U-Frames Unnumbered frames are used to exchange session management and control information between connected devices. Unlike S-frames, U-frames contain an information field, but one used for system management information, not user data. As with S-frames, however, much of the information carried by U-frames is contained in codes included in the control field. U-frame codes are divided into two sections: a 2-bit prefix before the P/F bit and a 3-bit suffix after the P/F bit. Together, these two segments (5 bits) can be used to create up to 32 different types of U-frames.

Point-to-Point Protocol (PPP)

One of the most common protocols for point-to-point access is the **Point-to-Point Protocol (PPP)**. Today, millions of Internet users who need to connect their home

computers to the server of an Internet service provider use PPP. The majority of these users have a traditional modem; they are connected to the Internet through a telephone line, which provides the services of the physical layer. But to control and manage the transfer of data, there is a need for a point-to-point protocol at the data-link layer. PPP is by far the most common.

Services

The designers of PPP have included several services to make it suitable for a point-to-point protocol, but have ignored some traditional services to make it simple.

Services Provided by PPP PPP defines the format of the frame to be exchanged between devices. It also defines how two devices can negotiate the establishment of the link and the exchange of data. PPP is designed to accept payloads from several network layers (not only IP). Authentication is also provided in the protocol, but it is optional. The new version of PPP, called *Multilink PPP*, provides connections over multiple links. One interesting feature of PPP is that it provides network address configuration. This is particularly useful when a home user needs a temporary network address to connect to the Internet.

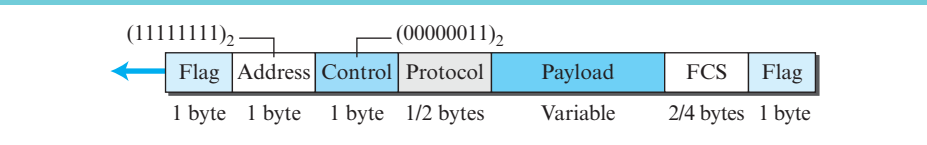
Services Not Provided by PPP PPP does not provide flow control. A sender can send several frames one after another with no concern about overwhelming the receiver. PPP has a very simple mechanism for error control. A CRC field is used to detect errors. If the frame is corrupted, it is silently discarded; the upper-layer protocol needs to take care of the problem. Lack of error control and sequence numbering may cause a packet to be received out of order. PPP does not provide a sophisticated addressing mechanism to handle frames in a multipoint configuration.

Framing

PPP uses a character-oriented (or byte-oriented) frame. Figure 5.24 shows the format of a PPP frame. The description of each field follows:

- ❑ **Flag.** A PPP frame starts and ends with a 1-byte flag with the bit pattern 01111110.
- ❑ **Address.** The address field in this protocol is a constant value and set to 11111111 (broadcast address).

Figure 5.24 PPP frame format



- ❑ **Control.** This field is set to the constant value 00000011 (imitating unnumbered frames in HDLC). As we will discuss later, PPP does not provide any flow control. Error control is also limited to error detection.
- ❑ **Protocol.** The protocol field defines what is being carried in the data field: either user data or other information. This field is by default 2 bytes long, but the two parties can agree to use only 1 byte.
- ❑ **Payload field.** This field carries either the user data or other information that we will discuss shortly. The data field is a sequence of bytes with the default of a

maximum of 1500 bytes; but this can be changed during negotiation. The data field is byte-stuffed if the flag byte pattern appears in this field. Because there is no field defining the size of the data field, padding is needed if the size is less than the maximum default value or the maximum negotiated value.

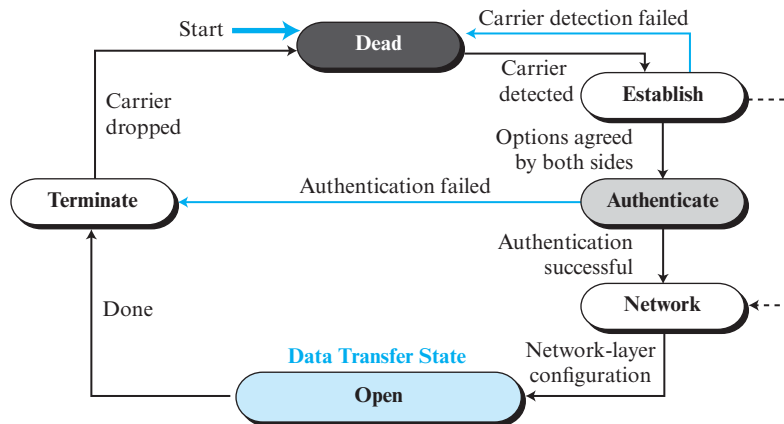
❑ **FCS.** The frame check sequence (FCS) is simply a 2-byte or 4-byte standard CRC.

Byte Stuffing Since PPP is a byte-oriented protocol, the flag in PPP is a byte that needs to be escaped whenever it appears in the data section of the frame. The escape byte is 01111101, which means that every time the flaglike pattern appears in the data, this extra byte is stuffed to tell the receiver that the next byte is not a flag. Obviously, the escape byte itself should be stuffed with another escape byte.

Transition Phases

A PPP connection goes through phases which can be shown in a *transition phase* diagram (see Figure 5.25). The transition diagram starts with the *dead* state. In this state, there is no active carrier (at the physical layer) and the line is quiet. When one of the two nodes starts the communication, the connection goes into the *establish* state. In this state, options are negotiated between the two parties. If the two ends agree with authentication, the system goes to the *authenticate* state; otherwise, the system goes to the *network* state. The link-control protocol packets, discussed shortly, are used for this purpose. Several packets may be exchanged here. Data transfer takes place in the *open* state. When a connection reaches this state, the exchange of data packets can be started. The connection remains in this state until one of the endpoints wants to terminate the connection. In this case, the system goes to the *terminate* state. The system remains in this state until the carrier (physical-layer signal) is dropped, which moves the system to the *dead* state again.

Figure 5.25 Transition phases

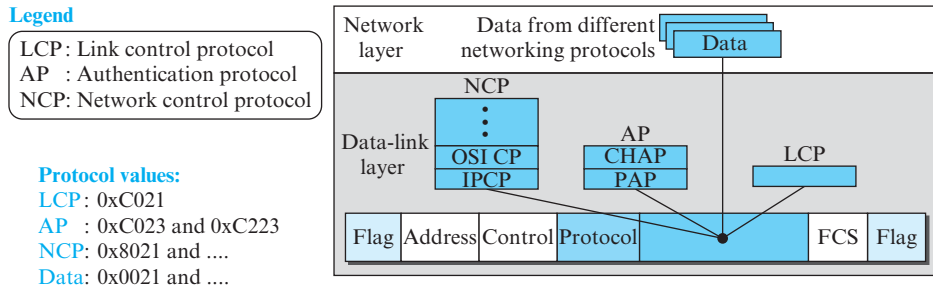


Multiplexing

Although PPP is a link-layer protocol, it uses another set of protocols to establish the link, authenticate the parties involved, and carry the network-layer data. Three sets of protocols are defined to make PPP powerful: the Link Control Protocol (LCP), two Authentication Protocols (APs), and several Network Control Protocols (NCPs). At any

moment, a PPP packet can carry data from one of these protocols in its data field, as shown in Figure 5.26. Note that there is one LCP, two APs, and several NCPs. Data may also come from several different network layers.

Figure 5.26 Multiplexing in PPP



Link Control Protocol The **Link Control Protocol (LCP)** is responsible for establishing, maintaining, configuring, and terminating links. It also provides negotiation mechanisms to set options between the two endpoints. Both endpoints of the link must reach an agreement about the options before the link can be established.

Authentication Protocols Authentication plays a very important role in PPP because PPP is designed for use over dial-up links where verification of user identity is necessary. *Authentication* means validating the identity of a user who needs to access a set of resources. PPP has created two protocols for authentication: Password Authentication Protocol and Challenge Handshake Authentication Protocol. Note that these protocols are used during the authentication phase.

- ❑ **PAP.** The *Password Authentication Protocol (PAP)* is a simple authentication procedure with a two-step process:
 - a. The user who wants to access a system sends an authentication identification (usually the user name) and a password.
 - b. The system checks the validity of the identification and password and either accepts or denies connection.
- ❑ **CHAP.** The *Challenge Handshake Authentication Protocol (CHAP)* is a three-way hand-shaking authentication protocol that provides greater security than PAP. In this method, the password is kept secret; it is never sent online.
 - a. The system sends the user a challenge packet containing a challenge value, usually a few bytes.
 - b. The user applies a predefined function that takes the challenge value and the user's own password and creates a result. The user sends the result in the response packet to the system.
 - c. The system does the same. It applies the same function to the password of the user (known to the system) and the challenge value to create a result. If the result created is the same as the result sent in the response packet, access is

granted; otherwise, it is denied. CHAP is more secure than PAP, especially if the system continuously changes the challenge value. Even if the intruder learns the challenge value and the result, the password is still secret.

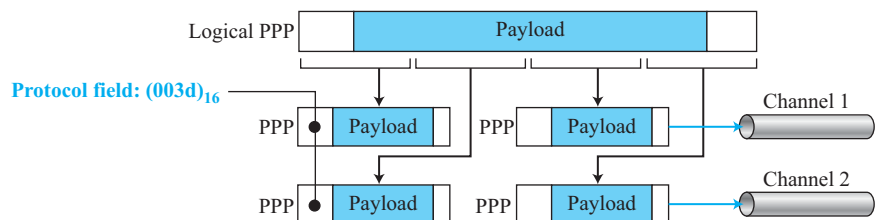
Network Control Protocols PPP is a multiple-network-layer protocol. It can carry a network-layer data packet from protocols defined by the Internet, OSI, Xerox, DECnet, AppleTalk, Novel, and so on. To do this, PPP has defined a specific Network Control Protocol for each network protocol. For example, **Internet Protocol Control Protocol (IPCP)** configures the link for carrying IP data packets. Xerox CP does the same for the Xerox protocol data packets, and so on. Note that none of the NCP packets carry network-layer data; they just configure the link at the network layer for the incoming data. One NCP protocol is the Internet Protocol Control Protocol (IPCP). This protocol configures the link used to carry IP packets in the Internet. IPCP is especially of interest to us.

Data from the Network Layer After the network-layer configuration is completed by one of the NCP protocols, the users can exchange data packets from the network layer. Here again, there are different protocol fields for different network layers. For example, if PPP is carrying data from the IP network layer, the field value is $(0021)_{16}$. If PPP is carrying data from the OSI network layer, the value of the protocol field is $(0023)_{16}$, and so on.

Multilink PPP

PPP was originally designed for a single-channel point-to-point physical link. The availability of multiple channels in a single point-to-point link motivated the development of Multilink PPP. In this case, a logical PPP frame is divided into several actual PPP frames. A segment of the logical frame is carried in the payload of an actual PPP frame, as shown in Figure 5.27. To show that the actual PPP frame is carrying a fragment of a logical PPP frame, the protocol field is set to $(003d)_{16}$. This new development adds complexity. For example, a sequence number needs to be added to the actual PPP frame to show a fragment's position in the logical frame.

Figure 5.27 Multilink PPP



5.3 MULTIPLE ACCESS PROTOCOLS

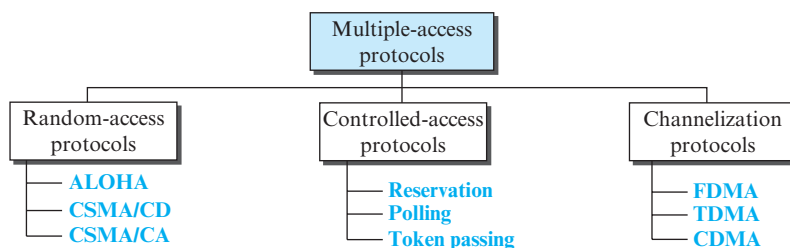
We said that the data-link layer is divided into two sublayers: data link control (DLC) and media access control (MAC). We discussed DLC in the previous section; we talk about MAC in this section. When we are using a dedicated link, such as a dial-up

telephone line, we need only a data-link-control protocol, such as Point-to-Point Protocol (PPP), that manages the data transfer between the two ends. On the other hand, if we are sharing the media, wire or air, with other users, we need to have a protocol to first manage the sharing process and then to do data transfer. For example, if we use our cellular phone to connect to another cellular phone, the channel (the band allocated to the vendor company) is not dedicated. A person a few feet away from us may be using the same band to talk to her friend.

When nodes or stations are connected and use a common link, called a multi-point or broadcast link, we need a multiple-access protocol to coordinate access to the link. The problem of controlling the access to the medium is similar to the rules of speaking in an assembly. The procedures guarantee that the right to speak is upheld and ensure that two people do not speak at the same time, do not interrupt each other, do not monopolize the discussion, and so on. The situation is similar for multipoint networks. We need to be sure that each node gets access to the link. The first goal is to prevent any collision between nodes. If somehow a collision occurs, the second goal is to handle the collision.

Many protocols have been devised to handle access to a shared link. We categorize them into three groups. Protocols belonging to each group are shown in Figure 5.28.

Figure 5.28 Taxonomy of multiple-access protocols discussed in this chapter



5.3.1 Random Access

In **random-access** or **contention** methods, no station is superior to another station and none is assigned the control over another. At each instance, a station that has data to send uses a procedure defined by the protocol to make a decision on whether or not to send. This decision depends on the state of the medium (idle or busy). In other words, each station can transmit when it desires on the condition that it follows the predefined procedure, including the testing of the state of the medium.

Two features give this method its name. First, there is no scheduled time for a station to transmit. Transmission is random among the stations. That is why these methods are called *random access*. Second, no rules specify which station should send next. Stations compete with one another to access the medium. That is why these methods are also called *contention* methods.

In a random-access method, each station has the right to the medium without being controlled by any other station. However, if more than one station tries to send, there is an access conflict—*collision*—and the frames will be either destroyed or modified. To

avoid access conflict or to resolve it when it happens, each station follows a procedure that answers the following questions:

- ❑ When can the station access the medium?
- ❑ What can the station do if the medium is busy?
- ❑ How can the station determine the success or failure of the transmission?
- ❑ What can the station do if there is an access conflict?

The random-access methods we study in this chapter have evolved from a very interesting protocol known as *ALOHA*, which used a very simple procedure called *multiple access (MA)*. The method was improved with the addition of a procedure that forces the station to sense the medium before transmitting. This was called *carrier sense multiple access (CSMA)*. This method later evolved into two parallel methods: *carrier sense multiple access with collision detection (CSMA/CD)*, which tells the station what to do when a collision is detected, and *carrier sense multiple access with collision avoidance (CSMA/CA)*, which tries to avoid the collision.

ALOHA

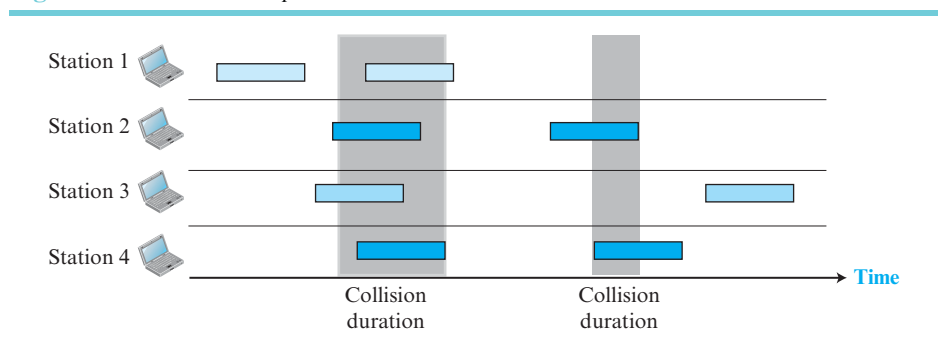
ALOHA, the earliest random access method, was developed at the University of Hawaii in early 1970. It was designed for a radio (wireless) LAN, but it can be used on any shared medium.

It is obvious that there are potential collisions in this arrangement. The medium is shared between the stations. When a station sends data, another station may attempt to do so at the same time. The data from the two stations collide and become garbled.

Pure ALOHA

The original ALOHA protocol is called **pure ALOHA**. This is a simple but elegant protocol. The idea is that each station sends a frame whenever it has a frame to send (multiple access). However, since there is only one channel to share, there is the possibility of collision between frames from different stations. Figure 5.29 shows an example of frame collisions in pure ALOHA.

Figure 5.29 Frames in a pure ALOHA network



There are four stations (unrealistic assumption) that contend with one another for access to the shared channel. The figure shows that each station sends two frames; there

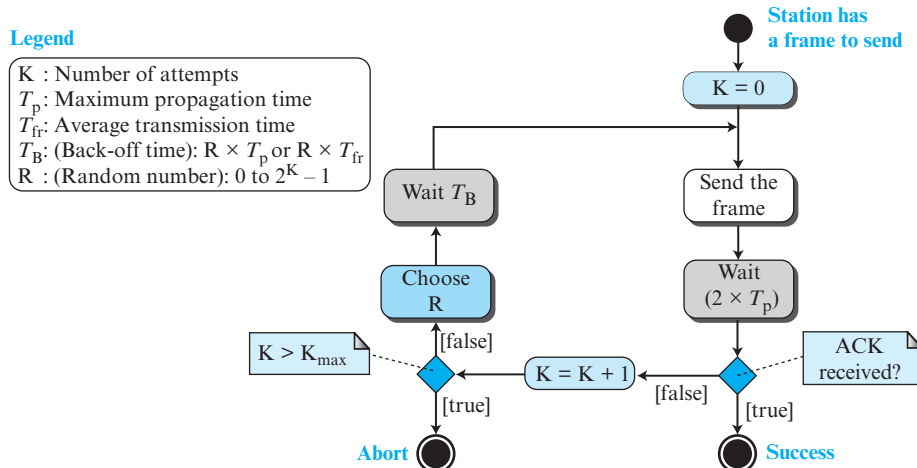
are a total of eight frames on the shared medium. Some of these frames collide because multiple frames are in contention for the shared channel. Figure 5.29 shows that only two frames survive: one frame from station 1 and one frame from station 3. We need to mention that even if one bit of a frame coexists on the channel with one bit from another frame, there is a collision and both will be destroyed. It is obvious that we need to resend the frames that have been destroyed during transmission.

The pure ALOHA protocol relies on acknowledgments from the receiver. When a station sends a frame, it expects the receiver to send an acknowledgment. If the acknowledgment does not arrive after a time-out period, the station assumes that the frame (or the acknowledgment) has been destroyed and resends the frame.

A collision involves two or more stations. If all these stations try to resend their frames after the time-out, the frames will collide again. Pure ALOHA dictates that when the time-out period passes, each station waits a random amount of time before resending its frame. The randomness will help avoid more collisions. We call this time the *back-off time* T_B .

Pure ALOHA has a second method to prevent congesting the channel with retransmitted frames. After a maximum number of retransmission attempts $K_{\max}$, a station must give up and try later. Figure 5.30 shows the procedure for pure ALOHA based on the above strategy.

Figure 5.30 Procedure for pure ALOHA protocol



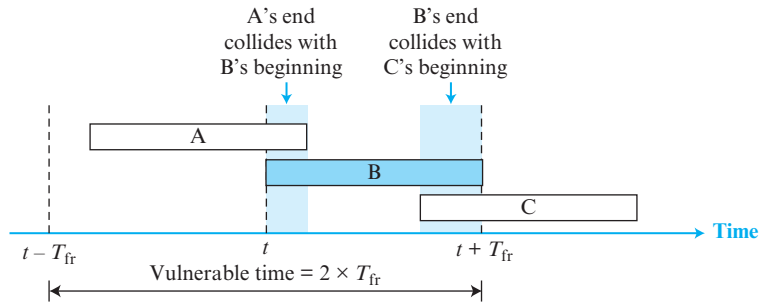
The time-out period is equal to the maximum possible round-trip propagation delay, which is twice the amount of time required to send a frame between the two most widely separated stations ($2 \times T_p$). The back-off time T_B is a random value that normally depends on K (the number of attempted unsuccessful transmissions). The formula for T_B depends on the implementation. One common formula is the *binary exponential back-off*. In this method, for each retransmission, a multiplier $R = 0$ to $2^K - 1$ is randomly chosen and multiplied by T_p (maximum propagation time) or T_{fr} (the average time required to send out a frame) to find T_B . Note that in this procedure, the range of the random numbers increases after each collision. The value of $K_{\max}$ is usually chosen as 15.

Example 5.11

The stations on a wireless ALOHA network are a maximum of 600 km apart. If we assume that signals propagate at 3×10^8 m/s, we find $T_p = (600 \times 10^3) / (3 \times 10^8) = 2$ ms. For $K = 2$, the range of R is $\{0, 1, 2, 3\}$. This means that T_B can be 0, 2, 4, or 6 ms, based on the outcome of the random variable R .

Vulnerable time Let us find the length of time, the *vulnerable time*, in which there is a possibility of collision. We assume that the stations send fixed-length frames with each frame taking T_{fr} seconds to send. Figure 5.31 shows the vulnerable time for station B.

Figure 5.31 Vulnerable time for pure ALOHA protocol



Station B starts to send a frame at time t . Now imagine station A has started to send its frame after $t - T_{fr}$. This leads to a collision between the frames from station B and station A. On the other hand, suppose that station C starts to send a frame before time $t + T_{fr}$. Here, there is also a collision between frames from station B and station C.

Looking at Figure 5.31, we see that the vulnerable time, during which a collision may occur in pure ALOHA, is 2 times the frame transmission time.

$$\text{Pure ALOHA vulnerable time} = 2 \times T_{fr}$$

Example 5.12

A pure ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. What is the requirement to make this frame collision-free?

Solution

Average frame transmission time T_{fr} is 200 bits/200 kbps or 1 ms. The vulnerable time is 2×1 ms = 2 ms. This means no station should send later than 1 ms before this station starts transmission and no station should start sending during the period (1 ms) that this station is sending.

Throughput Let us call G the average number of frames generated by the system during one frame transmission time. Then it can be proven that the average number of successfully transmitted frames for pure ALOHA is $S = G \times e^{-2G}$. The maximum throughput S_{max} is 0.184, for $G = 1/2$. (We can find it by setting the derivative of S with respect to G to 0, see Exercises.) In other words, if one-half a frame is generated during one frame transmission time (one frame during two frame transmission times), then 18.4 percent of these frames reach their destination successfully. We expect $G = 1/2$ to produce the maximum throughput because the vulnerable time is 2 times the frame transmission time.

Therefore, if a station generates only one frame in this vulnerable time (and no other stations generate a frame during this time), the frame will reach its destination successfully.

The throughput for pure ALOHA is $S = G \times e^{-2G}$.
The maximum throughput $S_{\max} = 1/(2e) = 0.184$ when $G = (1/2)$.

Example 5.13

A pure ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. What is the throughput if the system (all stations together) produces

- a. 1000 frames per second?
- b. 500 frames per second?
- c. 250 frames per second?

Solution

The frame transmission time is 200/200 kbps or 1 ms.

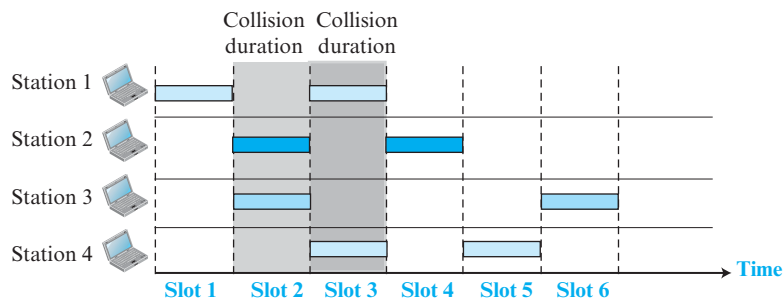
- a. If the system creates 1000 frames per second, or 1 frame per millisecond, then $G = 1$. In this case $S = G \times e^{-2G} = 0.135$ (13.5 percent). This means that the throughput is $1000 \times 0.135 = 135$ frames. Only 135 frames out of 1000 will probably survive.
- b. If the system creates 500 frames per second, or 1/2 frames per millisecond, then $G = 1/2$. In this case $S = G \times e^{-2G} = 0.184$ (18.4 percent). This means that the throughput is $500 \times 0.184 = 92$ and that only 92 frames out of 500 will probably survive. Note that this is the *maximum* throughput case, percentage-wise.
- c. If the system creates 250 frames per second, or 1/4 frames per millisecond, then $G = 1/4$. In this case $S = G \times e^{-2G} = 0.152$ (15.2 percent). This means that the throughput is $250 \times 0.152 = 38$. Only 38 frames out of 250 will probably survive.

Slotted ALOHA

Pure ALOHA has a vulnerable time of $2 \times T_{fr}$. This is so because there is no rule that defines when the station can send. A station may send soon after another station has started or just before another station has finished. Slotted ALOHA was invented to improve the efficiency of pure ALOHA.

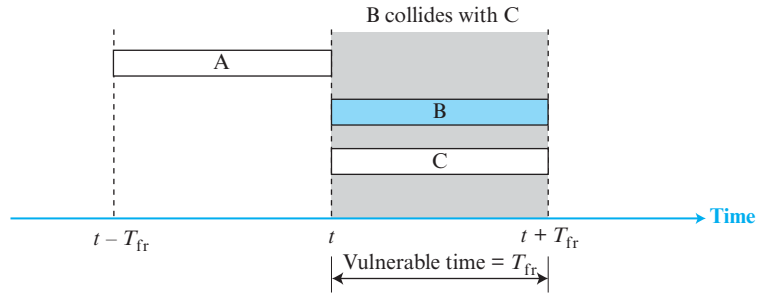
In **slotted ALOHA** we divide the time into slots of T_{fr} seconds and force the station to send only at the beginning of the time slot. Figure 5.32 shows an example of frame collisions in slotted ALOHA.

Figure 5.32 Frames in a slotted ALOHA network



Because a station is allowed to send only at the beginning of the synchronized time slot; if a station misses this moment, it must wait until the beginning of the next time slot. This means that the station which started at the beginning of this slot has already finished sending its frame. Of course, there is still the possibility of collision if two stations try to send at the beginning of the same time slot. However, the vulnerable time is now reduced to one-half, equal to T_{fr} . Figure 5.33 shows the situation.

Figure 5.33 Vulnerable time for slotted ALOHA protocol



Slotted ALOHA vulnerable time = T_{fr}

Throughput It can be proven that the average number of successful transmissions for slotted ALOHA is $S = G \times e^{-G}$. The maximum throughput S_{max} is 0.368, when $G = 1$. In other words, if one frame is generated during one frame transmission time, then 36.8 percent of these frames reach their destination successfully. We expect $G = 1$ to produce maximum throughput because the vulnerable time is equal to the frame transmission time. Therefore, if a station generates only one frame in this vulnerable time (and no other station generates a frame during this time), the frame will reach its destination successfully.

**The throughput for slotted ALOHA is $S = G \times e^{-G}$.
The maximum throughput $S_{max} = 0.368$ when $G = 1$.**

Example 5.14

A slotted ALOHA network transmits 200-bit frames using a shared channel with a 200-kbps bandwidth. Find the throughput if the system (all stations together) produces

- a. 1000 frames per second.
- b. 500 frames per second.
- c. 250 frames per second.

Solution

This situation is similar to the previous exercise except that the network is using slotted ALOHA instead of pure ALOHA. The frame transmission time is $200/200$ kbps or 1 ms.

- a. In this case G is 1. So $S = G \times e^{-G} = 0.368$ (36.8 percent). This means that the throughput is $1000 \times 0.368 = 368$ frames. Only 368 out of 1000 frames will probably survive. Note that this is the maximum throughput case, percentage-wise.

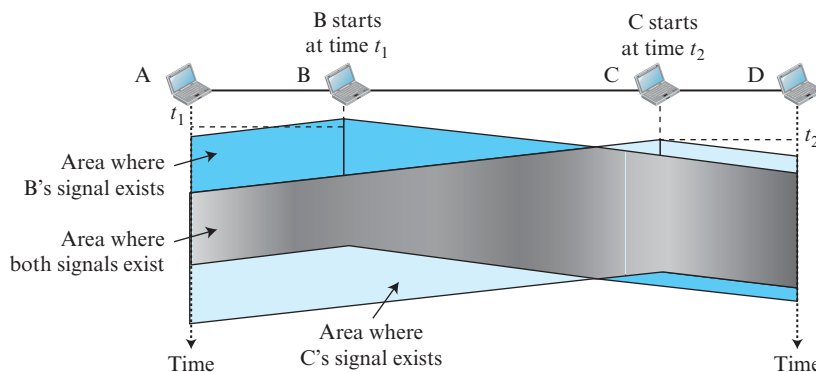
- b. Here G is $1/2$. In this case $S = G \times e^{-G} = 0.303$ (30.3 percent). This means that the throughput is $500 \times 0.303 = 151$. Only 151 frames out of 500 will probably survive.
- c. Now G is $1/4$. In this case $S = G \times e^{-G} = 0.195$ (19.5 percent). This means that the throughput is $250 \times 0.195 = 49$. Only 49 frames out of 250 will probably survive.

Carrier Sense Multiple Access (CSMA)

To minimize the chance of collision and, therefore, increase the performance, the CSMA method was developed. The chance of collision can be reduced if a station senses the medium before trying to use it. **Carrier sense multiple access (CSMA)** requires that each station first listen to the medium (or check the state of the medium) before sending. In other words, CSMA is based on the principle “sense before transmit” or “listen before talk.”

CSMA can reduce the possibility of collision, but it cannot eliminate it. The reason for this is shown in Figure 5.34, a space and time model of a CSMA network. Stations are connected to a shared channel (usually a dedicated medium).

Figure 5.34 Space/time model of a collision in CSMA

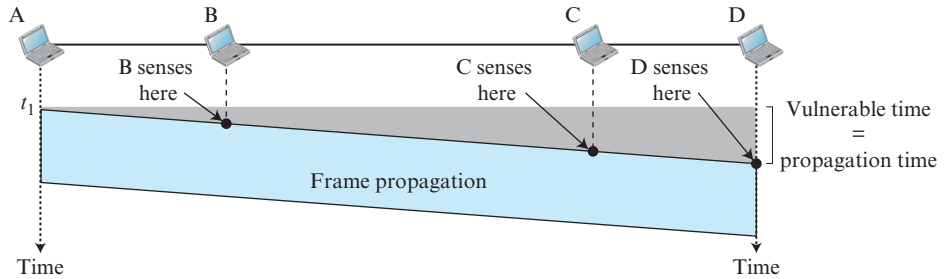


The possibility of collision still exists because of propagation delay; when a station sends a frame, it still takes time (although very short) for the first bit to reach every station and for every station to sense it. In other words, a station may sense the medium and find it idle, only because the first bit sent by another station has not yet been received.

At time t_1 , station B senses the medium and finds it idle, so it sends a frame. At time t_2 ($t_2 > t_1$), station C senses the medium and finds it idle because, at this time, the first bits from station B have not reached station C. Station C also sends a frame. The two signals collide and both frames are destroyed.

Vulnerable Time

The vulnerable time for CSMA is the *propagation time* T_p . This is the time needed for a signal to propagate from one end of the medium to the other. When a station sends a frame and any other station tries to send a frame during this time, a collision will result. But if the first bit of the frame reaches the end of the medium, every station will already have heard the bit and will refrain from sending. Figure 5.35 shows the worst case. The leftmost station A sends a frame at time t_1 , which reaches the rightmost station D at time $t_1 + T_p$. The gray area shows the vulnerable area in time and space.

Figure 5.35 Vulnerable time in CSMA**Persistence Methods**

What should a station do if the channel is busy? What should a station do if the channel is idle? Three methods have been devised to answer these questions: the **1-persistent method**, the **nonpersistent method**, and the ***p*-persistent method**. Figure 5.36 shows the behavior of three persistence methods when a station finds a channel busy.

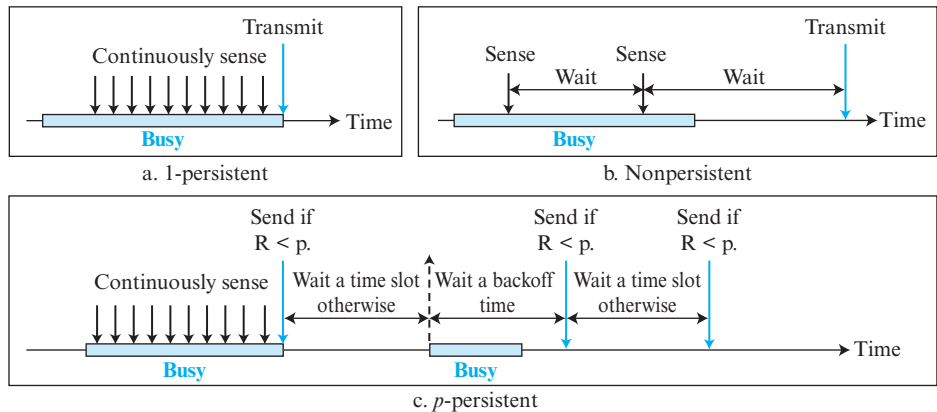
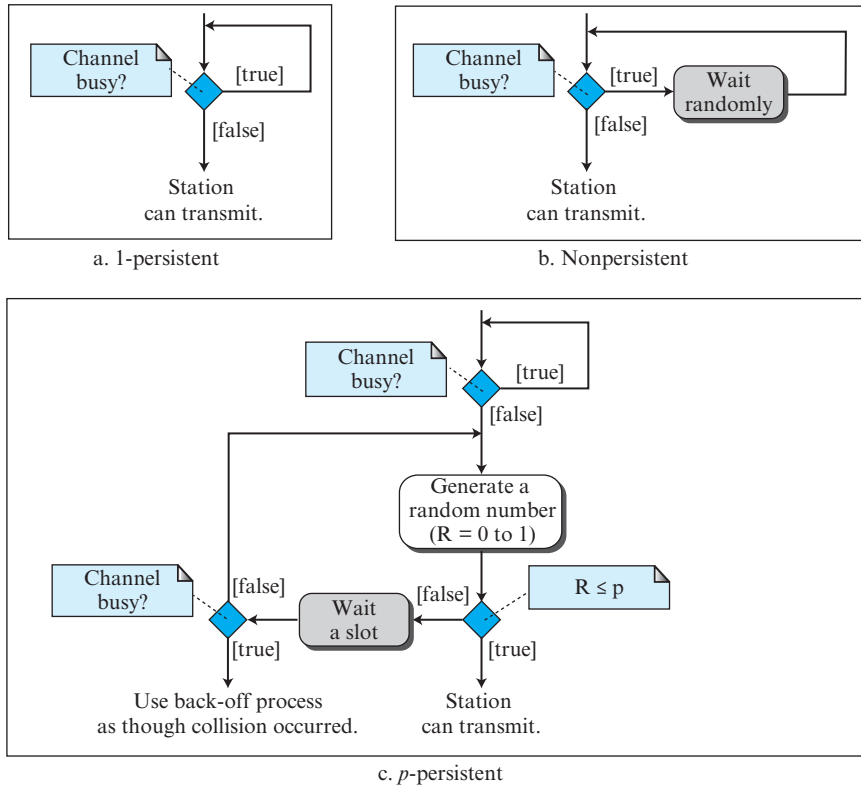
Figure 5.36 Behavior of three persistence methods

Figure 5.37 shows the flow diagrams for these methods.

1-Persistent The *1-persistent method* is simple and straightforward. In this method, after the station finds the line idle, it sends its frame immediately (with probability 1). This method has the highest chance of collision because two or more stations may find the line idle and send their frames immediately. We will see later that Ethernet uses this method.

Nonpersistent In the *nonpersistent method*, a station that has a frame to send senses the line. If the line is idle, it sends immediately. If the line is not idle, it waits a random amount of time and then senses the line again. The nonpersistent approach reduces the chance of collision because it is unlikely that two or more stations will wait the same amount of time and retry to send simultaneously. However, this method reduces the

Figure 5.37 Flow diagram for three persistence methods

efficiency of the network because the medium remains idle when there may be stations with frames to send.

p-Persistent The *p-persistent method* is used if the channel has time slots with a slot duration equal to or greater than the maximum propagation time. The *p-persistent* approach combines the advantages of the other two strategies. It reduces the chance of collision and improves efficiency. In this method, after the station finds the line idle it follows these steps:

1. With probability p , the station sends its frame.
2. With probability $q = 1 - p$, the station waits for the beginning of the next time slot and checks the line again.
 - a. If the line is idle, it goes to step 1.
 - b. If the line is busy, it acts as though a collision has occurred and uses the back-off procedure.

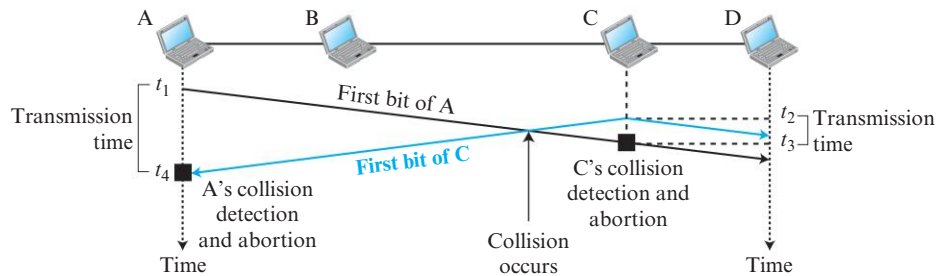
CSMA/CD

The CSMA method does not specify the procedure following a collision. **Carrier sense multiple access with collision detection (CSMA/CD)** augments the algorithm to handle the collision.

In this method, a station monitors the medium after it sends a frame to see if the transmission was successful. If so, the station is finished. If, however, there is a collision, the frame is sent again.

To better understand CSMA/CD, let us look at the first bits transmitted by the two stations involved in the collision. Although each station continues to send bits in the frame until it detects the collision, we show what happens as the first bits collide. In Figure 5.38, stations A and C are involved in the collision.

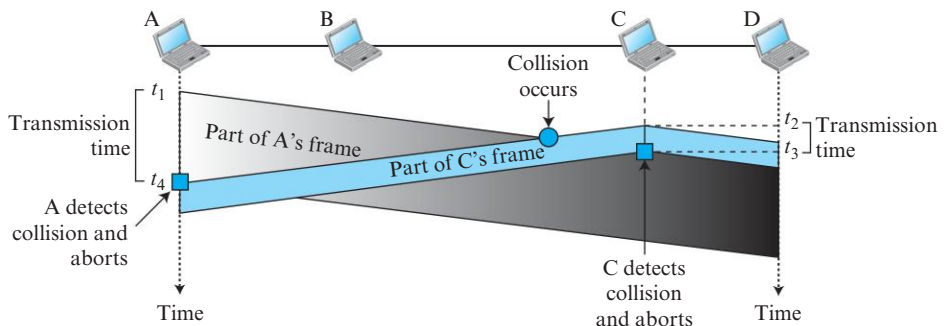
Figure 5.38 Collision of the first bits in CSMA/CD



At time t_1 , station A has executed its persistence procedure and starts sending the bits of its frame. At time t_2 , station C has not yet sensed the first bit sent by A. Station C executes its persistence procedure and starts sending the bits in its frame, which propagate both to the left and to the right. The collision occurs sometime after time t_2 . Station C detects a collision at time t_3 when it receives the first bit of A's frame. Station C immediately (or after a short time, but we assume immediately) aborts transmission. Station A detects collision at time t_4 when it receives the first bit of C's frame; it also immediately aborts transmission. Looking at the figure, we see that A transmits for the duration $t_4 - t_1$; C transmits for the duration $t_3 - t_2$.

Now that we know the time durations for the two transmissions, we can show a more complete graph in Figure 5.39.

Figure 5.39 Collision and abortion in CSMA/CD



Minimum Frame Size

For CSMA/CD to work, we need a restriction on the frame size. Before sending the last bit of the frame, the sending station must detect a collision, if any, and abort the transmission. This is so because the station, once the entire frame is sent, does not keep a copy of the frame and does not monitor the line for collision detection. Therefore, the frame transmission time T_{fr} must be at least two times the maximum propagation time T_p . To understand the reason, let us think about the worst-case scenario. If the two stations involved in a collision are the maximum distance apart, the signal from the first takes time T_p to reach the second, and the effect of the collision takes another time T_p to reach the first. So the requirement is that the first station must still be transmitting after $2T_p$.

Example 5.15

A network using CSMA/CD has a bandwidth of 10 Mbps. If the maximum propagation time (including the delays in the devices and ignoring the time needed to send a jamming signal, as we see later) is $25.6 \mu s$, what is the minimum size of the frame?

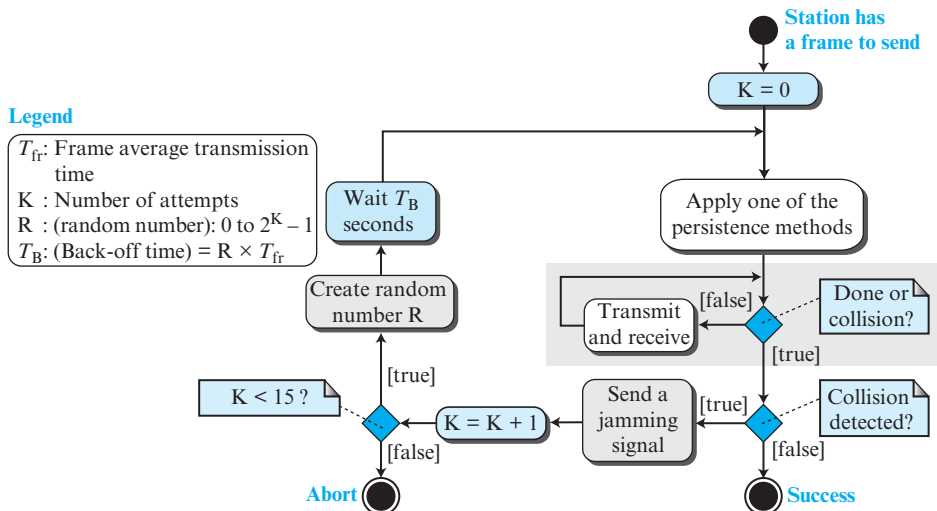
Solution

The minimum frame transmission time is $T_{fr} = 2 \times T_p = 51.2 \mu s$. This means, in the worst case, a station needs to transmit for a period of $51.2 \mu s$ to detect the collision. The minimum size of the frame is $10 \text{ Mbps} \times 51.2 \mu s = 512 \text{ bits}$ or 64 bytes. This is actually the minimum size of the frame for Standard Ethernet, as we will see later in the chapter.

Procedure

Now let us look at the flow diagram for CSMA/CD in Figure 5.40. It is similar to the one for the ALOHA protocol, but there are differences.

Figure 5.40 Flow diagram for the CSMA/CD



The first difference is the addition of the persistence process. We need to sense the channel before we start sending the frame by using one of the persistence processes we

discussed previously (nonpersistent, 1-persistent, or p -persistent). The corresponding box can be replaced by one of the persistence processes shown in Figure 5.37.

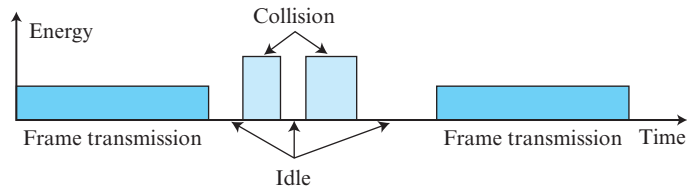
The second difference is the frame transmission. In ALOHA, we first transmit the entire frame and then wait for an acknowledgment. In CSMA/CD, transmission and collision detection is a continuous process. We do not send the entire frame and then look for a collision. The station transmits and receives continuously and simultaneously (using two different ports or a bidirectional port). We use a loop to show that transmission is a continuous process. We constantly monitor in order to detect one of two conditions: either transmission is finished or a collision is detected. Either event stops transmission. When we come out of the loop, if a collision has not been detected, it means that transmission is complete; the entire frame is transmitted. Otherwise, a collision has occurred.

The third difference is the sending of a short *jamming signal* to make sure that all other stations become aware of the collision.

Energy Level

We can say that the level of energy in a channel can have three values: zero, normal, and abnormal. At the zero level, the channel is idle. At the normal level, a station has successfully captured the channel and is sending its frame. At the abnormal level, there is a collision and the level of the energy is twice the normal level. A station that has a frame to send or is sending a frame needs to monitor the energy level to determine if the channel is idle, busy, or in collision mode. Figure 5.41 shows the situation.

Figure 5.41 Energy level during transmission, idleness, or collision



Throughput

The throughput of CSMA/CD is greater than that of pure or slotted ALOHA. The maximum throughput occurs at a different value of G and is based on the persistence method and the value of p in the p -persistent approach. For the 1-persistent method, the maximum throughput is around 50 percent when $G = 1$. For the nonpersistent method, the maximum throughput can go up to 90 percent when G is between 3 and 8.

Traditional Ethernet

One of the LAN protocols that used CSMA/CD is the traditional Ethernet with the data rate of 10 Mbps. We discuss the Ethernet LANs later in the chapter, but it is good to know that the traditional Ethernet was a broadcast LAN that used 1-persistence method to control access to the common media. Later versions of Ethernet try to move from CSMA/CD access methods for the reason that we discuss in the next section when we discuss wired LANs.

CSMA/CA

A variation of the CSMA method is **Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)**, which is used in wireless LANs. We'll postpone the discussion until Chapter 6, which deals with that topic.

5.3.2 Controlled Access

In **controlled access**, the stations consult one another to find which station has the right to send. A station cannot send unless it has been authorized by other stations. We discuss three controlled-access methods.

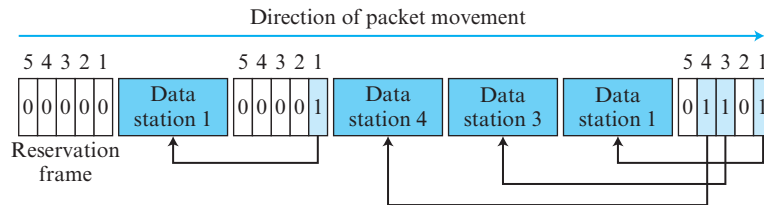
Reservation

In the **reservation** method, a station needs to make a reservation before sending data. Time is divided into intervals. In each interval, a reservation frame precedes the data frames sent in that interval.

If there are N stations in the system, there are exactly N reservation minislots in the reservation frame. Each minislot belongs to a station. When a station needs to send a data frame, it makes a reservation in its own minislot. The stations that have made reservations can send their data frames after the reservation frame.

Figure 5.42 shows a situation with five stations and a five-minislot reservation frame. In the first interval, only stations 1, 3, and 4 have made reservations. In the second interval, only station 1 has made a reservation.

Figure 5.42 Reservation access method

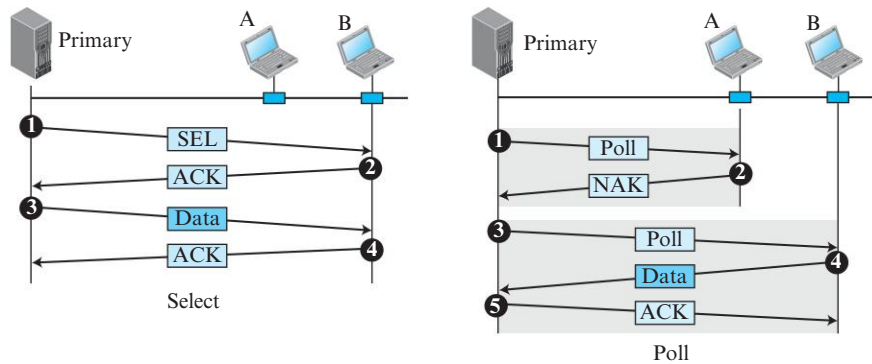


Polling

Polling works with topologies in which one device is designated as a *primary station* and the other devices are *secondary stations*. All data exchanges must be made through the primary device even when the ultimate destination is a secondary device. The primary device controls the link; the secondary devices follow its instructions. It is up to the primary device to determine which device is allowed to use the channel at a given time. The primary device, therefore, is always the initiator of a session (see Figure 5.43). This method uses poll and select functions to prevent collisions. However, the drawback is if the primary station fails, the system goes down.

Select

The *select* function is used whenever the primary device has something to send. Remember that the primary controls the link. If the primary is neither sending nor receiving data, it knows the link is available. If it has something to send, the primary

Figure 5.43 Select and poll functions in polling-access method

device sends it. What it does not know, however, is whether the target device is prepared to receive. So the primary must alert the secondary to the upcoming transmission and wait for an acknowledgment of the secondary's ready status. Before sending data, the primary creates and transmits a select (SEL) frame, one field of which includes the address of the intended secondary.

Poll

The *poll* function is used by the primary device to solicit transmissions from the secondary devices. When the primary is ready to receive data, it must ask (poll) each device in turn if it has anything to send. When the first secondary is approached, it responds either with a NAK frame if it has nothing to send or with data (in the form of a data frame) if it does. If the response is negative (a NAK frame), then the primary polls the next secondary in the same manner until it finds one with data to send. When the response is positive (a data frame), the primary reads the frame and returns an acknowledgment (ACK frame), verifying its receipt.

Token Passing

In the **token-passing** method, the stations in a network are organized in a logical ring. In other words, for each station, there is a *predecessor* and a *successor*. The predecessor is the station which is logically before the station in the ring; the successor is the station which is after the station in the ring. The current station is the one that is accessing the channel now. The right to this access has been passed from the predecessor to the current station. The right will be passed to the successor when the current station has no more data to send.

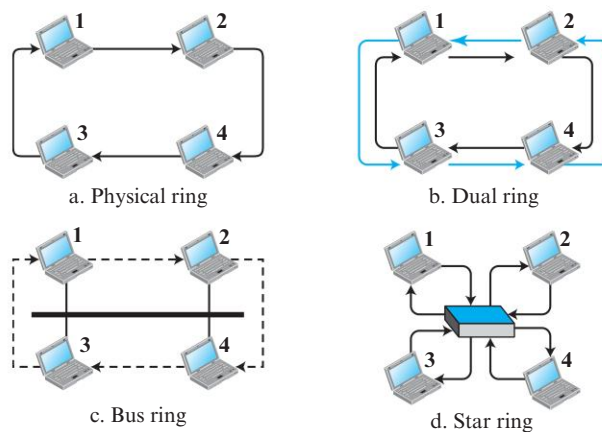
But how is the right to access the channel passed from one station to another? In this method, a special packet called a *token* circulates through the ring. The possession of the token gives the station the right to access the channel and send its data. When a station has some data to send, it waits until it receives the token from its predecessor. It then holds the token and sends its data. When the station has no more data to send, it releases the token, passing it to the next logical station in the ring. The station cannot send data until it receives the token again in the next round. In this process, when a station receives the token and has no data to send, it just passes the data to the next station.

Token management is needed for this access method. Stations must be limited in the time they can have possession of the token. The token must be monitored to ensure it has not been lost or destroyed. For example, if a station that is holding the token fails, the token will disappear from the network. Another function of token management is to assign priorities to the stations and to the types of data being transmitted. And finally, token management is needed to make low-priority stations release the token to high-priority stations.

Logical Ring

In a token-passing network, stations do not have to be physically connected in a ring; the ring can be a logical one. Figure 5.44 shows four different physical topologies that can create a logical ring.

Figure 5.44 Logical ring and physical topology in token-passing access method



In the physical ring topology, when a station sends the token to its successor, the token cannot be seen by other stations; the successor is the next one in line. This means that the token does not have to have the address of the next successor. The problem with this topology is that if one of the links—the medium between two adjacent stations—fails, the whole system fails.

The dual ring topology uses a second (auxiliary) ring which operates in the reverse direction compared with the main ring. The second ring is for emergencies only (such as a spare tire for a car). If one of the links in the main ring fails, the system automatically combines the two rings to form a temporary ring. After the failed link is restored, the auxiliary ring becomes idle again. Note that for this topology to work, each station needs to have two transmitter ports and two receiver ports. The high-speed Token Ring networks called FDDI (Fiber Distributed Data Interface) and CDDI (Copper Distributed Data Interface) use this topology.

In the bus ring topology, also called a token bus, the stations are connected to a single cable called a bus. They, however, make a logical ring, because each station knows the address of its successor (and also predecessor for token management purposes). When a station has finished sending its data, it releases the token and inserts the address of its successor in the token. Only the station with the address matching the destination

address of the token gets the token to access the shared media. The Token Bus LAN, standardized by IEEE, uses this topology.

In a star ring topology, the physical topology is a star. There is a hub, however, that acts as the connector. The wiring inside the hub makes the ring; the stations are connected to this ring through the two wire connections. This topology makes the network less prone to failure because if a link goes down, it will be bypassed by the hub and the rest of the stations can operate. Also adding and removing stations from the ring is easier. This topology is still used in the Token Ring LAN designed by IBM.

5.3.3 Channelization

Channelization (or *channel partition*, as it is sometimes called) is a multiple-access method in which the available bandwidth of a link is shared in time, frequency, or through code, among different stations. Since these methods are normally used in wireless networks, we postpone their discussion until the next chapter.

5.4 LINK-LAYER ADDRESSING

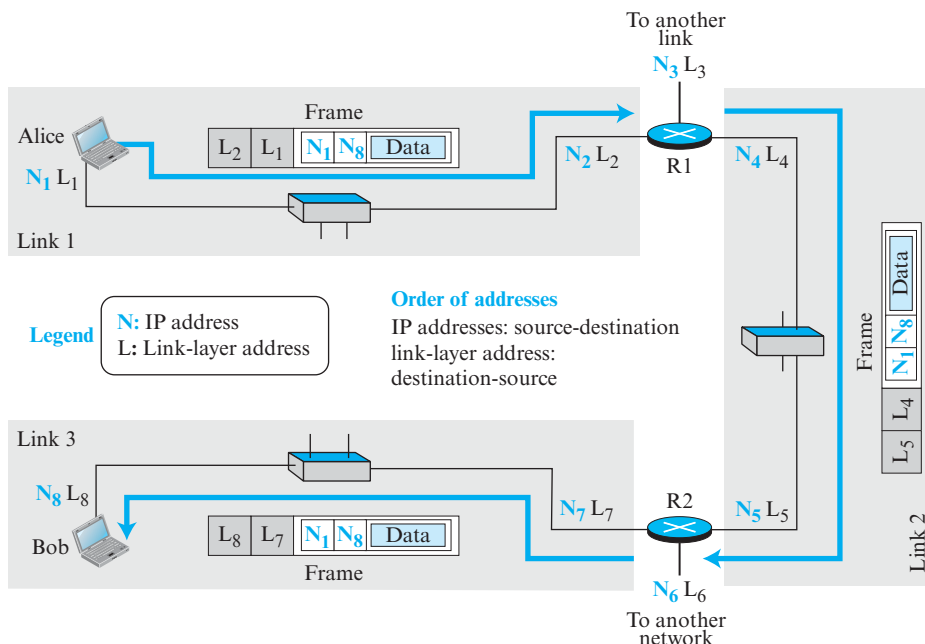
The next issue we need to discuss about the data-link layer is the link-layer addresses. In Chapter 4, we discussed IP addresses as the identifiers at the network layer that define the exact points in the Internet where the source and destination hosts are connected. However, in a connectionless internetwork such as the Internet we cannot make a datagram reach its destination using only IP addresses. The reason is that each datagram in the Internet, from the same source host to the same destination host, may take a different path. The source and destination IP addresses define the two ends but cannot define which links the datagram should pass through.

We need to remember that the IP addresses in a datagram should not be changed. If the destination IP address in a datagram changes, the packet never reaches its destination; if the source IP address in a datagram changes, the destination host or a router can never communicate with the source if a response needs to be sent back or an error needs to be reported back to the source (see ICMP in Chapter 4).

The above discussion shows that we need another addressing mechanism in a connectionless internetwork: the link-layer addresses of the two nodes. A *link-layer address* is sometimes called a *link address*, sometimes a *physical address*, and sometimes a *MAC address*. We use these terms interchangeably in this book.

Since a link is controlled at the data-link layer, the addresses need to belong to the data-link layer. When a datagram passes from the network layer to the data-link layer, the datagram will be encapsulated in a frame and two data-link addresses are added to the frame header. These two addresses are changed every time the frame moves from one link to another. Figure 5.45 demonstrates the concept in a small internet.

In the internet in Figure 5.45, we have three links and two routers. We also have shown only two hosts: Alice (source) and Bob (destination). For each host, we have shown two addresses, the IP addresses (N) and the link-layer addresses (L). Note that a router has as many pairs of addresses as the number of links the router is connected to. We have shown three frames, one in each link. Each frame carries the same datagram with the same source and destination addresses (N1 and N8), but the link-layer addresses of the frame change from link to link. In link1, the link-layer addresses are

Figure 5.45 IP addresses and link-layer addresses in a small internet

L₁ and L₂. In link2, they are L₄ and L₅. In link3, they are L₇ and L₈. Note that the IP addresses and the link-layer addresses are not in the same order. For IP addresses, the source address comes before the destination address; for link-layer addresses, the destination address comes before the source. The datagrams and frames are designed in this way and we follow the design. We may raise several questions:

- ❑ If the IP address of a router does not appear in any datagram sent from a source to a destination, why do we need to assign IP addresses to routers? The answer is that in some protocols a router may act as a sender or receiver of a datagram. For example, in routing protocols we discussed in Chapter 4, a router is a sender or a receiver of a message. The communications in these protocols are between routers.
- ❑ Why do we need more than one IP address in a router, one for each interface? The answer is that an interface is a connection of a router to a link. We said that an IP address defines a point in the Internet at which a device is connected. A router with n interfaces is connected to the Internet at n points. This is the situation of a house at the corner of a street with two gates; each gate has the address related to the corresponding street.
- ❑ How are the source and destination IP addresses in a packet determined? The answer is that the host should know its own IP address, which becomes the source IP address in the packet. As we discussed in Chapter 2, the application layer uses the services of DNS to find the destination address of the packet and passes it to the network layer to be inserted in the packet.

- ❑ How are the source and destination link-layer addresses determined for each link? Again each hop (router or host) should know its own link-layer address, as we discuss later in the chapter. The destination link-layer address is determined by using the Address Resolution Protocol, which we discuss shortly.
- ❑ What is the size of link-layer addresses? The answer is that it depends on the protocol used by the link. Although we have only one IP protocol for the whole Internet, we may be using different data-link protocols in different links. This means that we can define the size of the address when we discuss different link-layer protocols. This is done for wired networks in this chapter and wireless networks in the next chapter.

Example 5.16

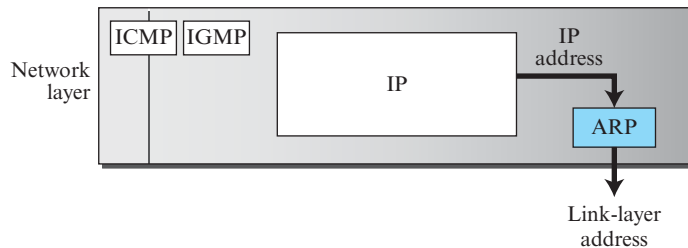
As we discuss later in the chapter, the link-layer addresses in the most common LAN, Ethernet, are 48 bits (six bytes) that are presented as 12 hexadecimal digits separated by colons; for example, the following is a link-layer address of a computer.

A2:34:45:11:92:F1

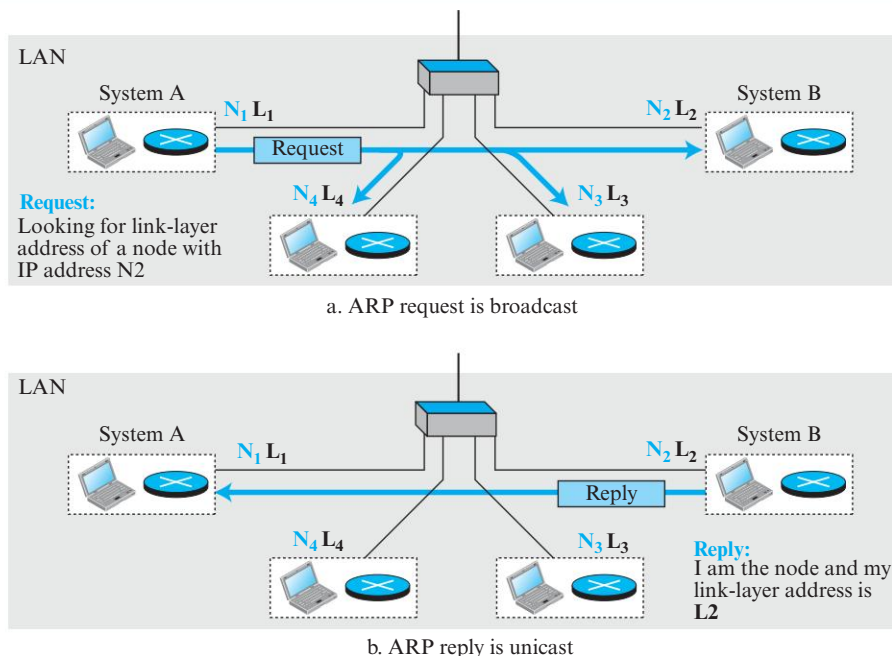
Address Resolution Protocol (ARP)

Anytime a node has an IP datagram to send to another node in a link, it has the IP address of the receiving node. The source host knows the IP address of the default router. Each router except the last one in the path gets the IP address of the next router by using its forwarding table (in the next hop column). The last router knows the IP address of the destination host. However, the IP address of the next node is not helpful in moving a frame through a link; we need the link-layer address of the next node. This is the time when the **Address Resolution Protocol (ARP)** becomes helpful. The ARP protocol is one of the auxiliary protocols defined in the network layer, as shown in Figure 5.46. It belongs to the network layer, but we postponed its discussion until now because it maps an IP address to a logical-link address. ARP accepts an IP address from the IP protocol, maps the address to the corresponding link-layer address, and passes it to the data-link layer.

Figure 5.46 Position of ARP in TCP/IP protocol suite



Anytime a host or a router needs to find the link-layer address of another host or router in its network, it sends an ARP request packet. The packet includes the link-layer and IP addresses of the sender and the IP address of the receiver. Because the sender does not know the link-layer address of the receiver, the query is broadcast over the link using the link-layer broadcast address, which we discuss for each protocol later (see Figure 5.47).

Figure 5.47 ARP operation

Every host or router on the network receives and processes the ARP request packet, but only the intended recipient recognizes its IP address and sends back an ARP response packet. The response packet contains the recipient's IP and link-layer addresses. The packet is unicast directly to the node that sent the request packet.

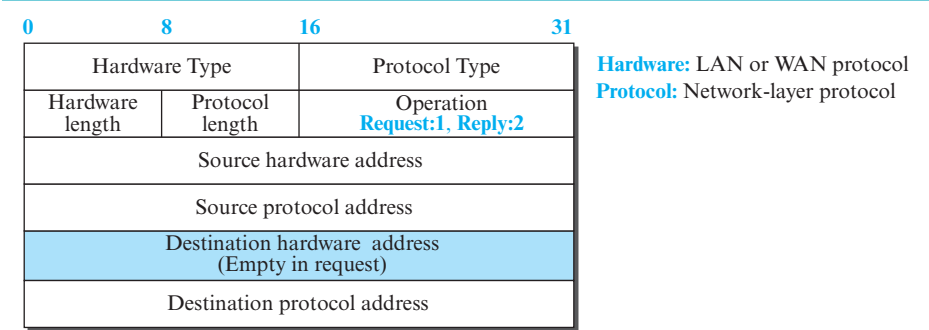
In Figure 5.47a, the system on the left (A) has a packet that needs to be delivered to another system (B) with IP address **N₂**. System A needs to pass the packet to its data-link layer for the actual delivery, but it does not know the physical address of the recipient. It uses the services of ARP by asking the ARP protocol to send a broadcast ARP request packet to ask for the physical address of a system with an IP address of **N₂**.

This packet is received by every system on the physical network, but only system B will answer it, as shown in Figure 5.47b. System B sends an ARP reply packet that includes its physical address. Now system A can send all the packets it has for this destination using the physical address it received.

Packet Format

Figure 5.48 shows the format of an ARP packet. The names of the fields are self-explanatory. The *hardware type* field defines the type of the link-layer protocol; Ethernet is given the type 1. The *protocol type* field defines the network-layer protocol: IPv4 protocol is (0800)₁₆. The source hardware and source protocol addresses are variable-length fields defining the link-layer and network-layer addresses of the sender. The destination hardware address and destination protocol address fields define the receiver link-layer and network-layer addresses. An ARP packet is encapsulated directly into a data-link frame. The frame needs to have a field to show that the payload belongs to the ARP and not to the network-layer datagram.

Figure 5.48 ARP packet



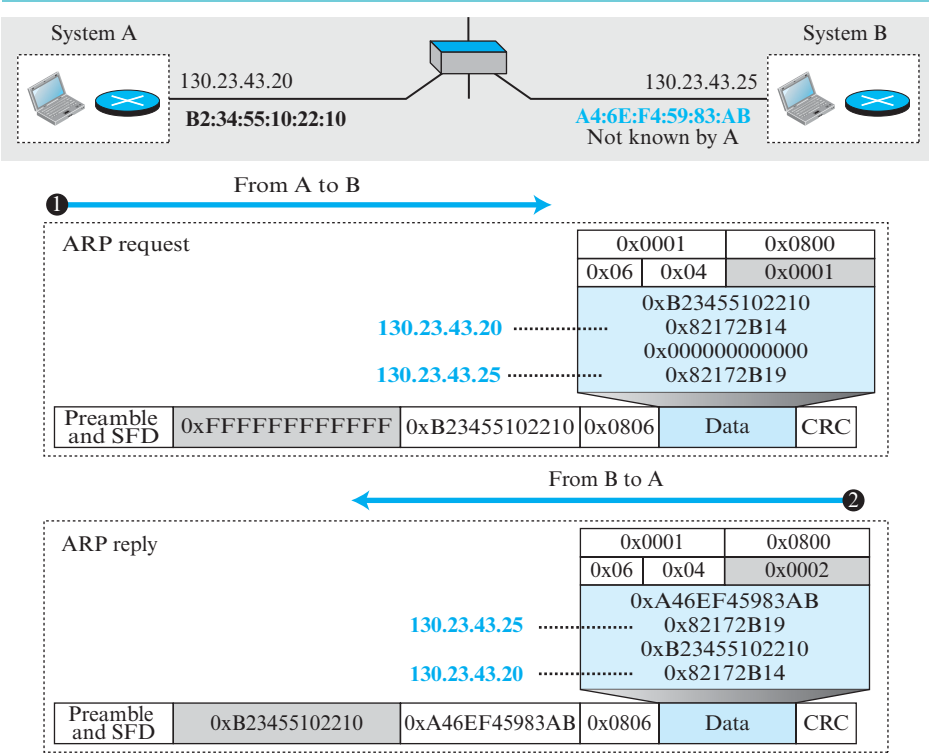
Example 5.17

A host with IP address **N1** and MAC address **L1** has a packet to send to another host with IP address **N2** and physical address **L2** (which is unknown to the first host). The two hosts are on the same network. Show the ARP request and reply packets encapsulated in Ethernet frames (see Figure 5.55).

Solution

Figure 5.49 shows the ARP request and reply packets. Note that the ARP data field in this case is 28 bytes, and that the individual addresses do not fit in the 4-byte boundary. That is

Figure 5.49 Example 5.17

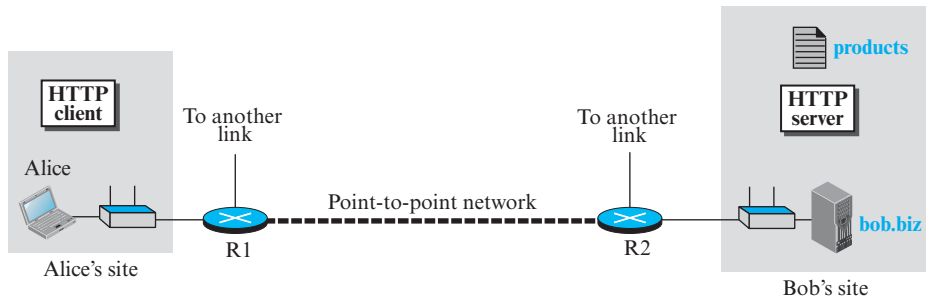


why we do not show the regular 4-byte boundaries for these addresses. Also note that the IP addresses are shown in hexadecimal.

An Example

We discussed addressing in the four upper layers of the TCP/IP protocol suite. There is no addressing in the physical layer because the unit of communication in this layer is a bit, which cannot hold an address; at the physical layer bits are sent out and received by any recipient connected to the sender (broadcast). We now think it is time to show all of these addresses in a single transaction, emphasizing their relationships. We also show how all these addresses can actually be created by the system. What is needed for the transaction is only the name of the site that a user needs to contact. Figure 5.50 shows an internet to which Alice and Bob are connected. Alice is a user that wants to access the list of the products offered by Bob. Bob has a small business called **Bob.biz**, in which the list of the products and their specifications is stored in a document called **products**.

Figure 5.50 The internet for our example

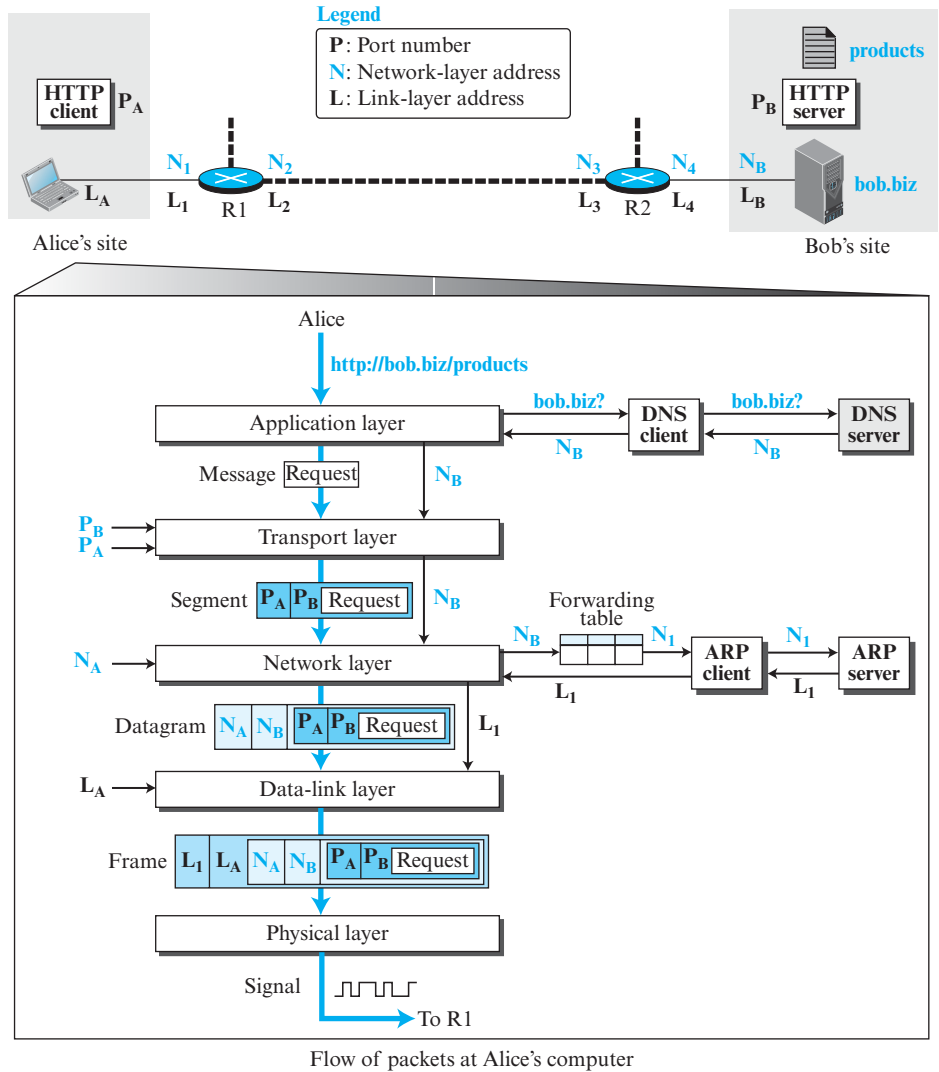


The only identifier Alice needs to know is the URL of the site: ***http://bob.biz/product***. The rest of addresses, source and destination, are generated by the system, as we show in this example.

Activities at the Alice Site

We assume that Alice is connected to a LAN and that her computer knows its IP address and its link-layer address. Bob's server is also connected to a LAN and his computer knows its IP address and its link-layer address. The routers in the internet also know their IP addresses and their link-layer addresses. We will use symbolic addresses to make the figures more readable. However, Alice does not need to know any of these addresses: they are created automatically as the transaction goes on.

Figure 5.51 shows what happens at Alice's site. Alice, using a browser, types the URL of Bob's site (***http://bob.biz/product***). Alice's computer uses the HTTP client to answer Alice's request. The HTTP client, however, does not know Bob's IP address. The client calls the DNS client for help. The DNS client sends a request to a DNS server and finds the IP address of Bob's computer (***N_B***), which is passed to the transport layer.

Figure 5.51 *Flow of packets at Alice's computer*

The transport layer now creates a segment using the well-known port number of the HTTP server ($P_B = 80$) and a temporary port number received from the operating system for the HTTP client (P_A). The transport layer can now make a segment and pass it to the network layer with the IP address of Bob's computer (N_B) obtained in the previous step. Of course, we do not show all hand-shaking processes that occurs; we assume that the connection is made here and only one segment is exchanged.

The network layer receives the segment and N_B , but it needs to find the link-layer address. The network layer consults its routing table and tries to find which router is

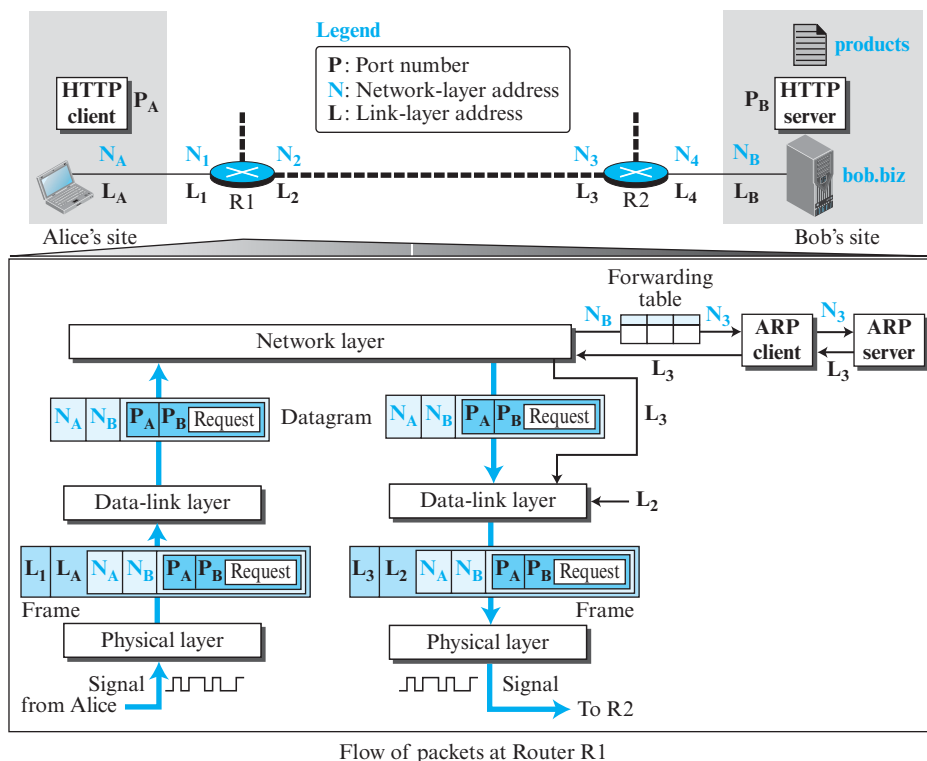
next (the default router in this case) for the destination N_B . The routing table gives N_1 , but the network layer needs to find the link-layer address of router R1. It uses its ARP client to find the link-layer address L_1 . The network layer can now pass the datagram with the link-layer address to the data-link layer.

The data-link layer knows its own link-layer address, L_A . It creates the frame and passes it to the physical layer, where the address is converted to signals (as we will see in Chapter 7) and sends it through the media.

Activities at Router R1

Now let us see what happens at Router R1. Router R1, as we know, has only three lower layers. The packet received needs to go up through these three layers and come down. Figure 5.52 shows the activities. At arrival, the physical layer of the left link creates the frame and passes it to the data-link layer. The data-link layer decapsulates the datagram and passes it to the network layer. The network layer examines the network-layer address of the datagram and finds that the datagram needs to be delivered to the device with IP address N_B . The network layer consults its routing table to find out which is the next node (router) in the path to N_B . The forwarding table returns N_3 . The IP address of router R2 is in the same link with R1. The network layer now uses the ARP client and server to find the link-layer address of this router, which

Figure 5.52 Flow of activities at router R1



comes up as L_3 . The network layer passes the datagram and L_3 to the data-link layer belonging to the link at the left side. The link layer encapsulates the datagram, adds L_3 and L_2 (its own link-layer address), and passes the frame to the physical layer. The physical layer encodes the bits to signals and sends them through the medium to R2.

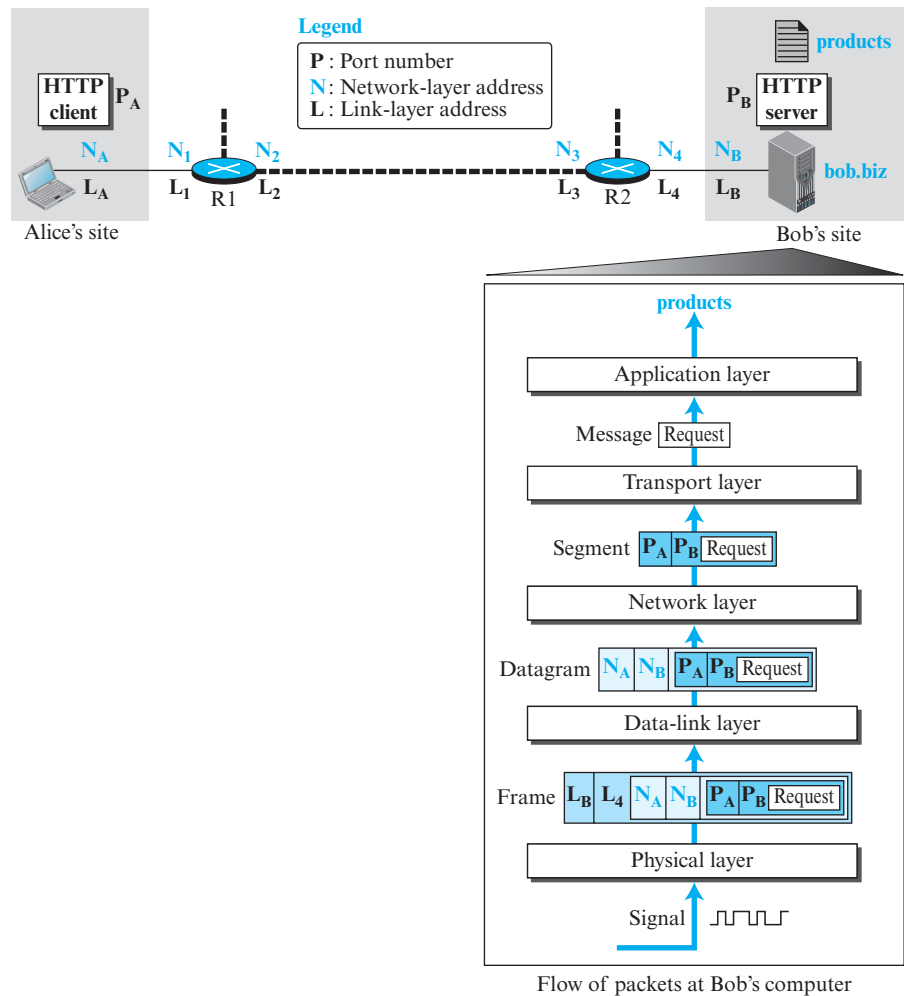
Activities at Router R2

Activities at this router are the same as those at router R1, except that some addresses are changed. So we do not repeat those here.

Activities at Bob's Site

Now let us see what happens at Bob's site. Figure 5.53 shows how the signals at Bob's site are changed to a message for the HTTP server program. At Bob's site there are no

Figure 5.53 Activities at Bob's site



more addresses or mapping needed. The signal received from the link is changed to a frame. The frame is passed to the data-link layer, which decapsulates the datagram and passes it to the network layer. The network layer decapsulates the message and passes it to the transport layer. The transport layer decapsulates the segment and passes to the port **P_B**(80). The HTTP server receives the message, prepares a copy of the file products, formats it as an HTML document, and sends it back to Alice using the same flow, but in the reverse order. The return message will probably go through the same routers, R2 and R1, to reach Alice.

5.5 WIRED LANS: ETHERNET PROTOCOL

In Chapter 1, we mentioned that the TCP/IP protocol suite does not define any protocol for data-link or physical layer. In other words, TCP/IP accepts any protocol at these two layers that can provide services to the network layer. The data-link layer and the physical layer are actually the territory of the local and wide area networks. This means that when we discuss these two layers, we are talking about networks that are using them. As we see in this chapter and the next, we can have wired or wireless networks. We discuss wired networks in this chapter and postpone the discussion of wireless networks to the next chapter.

In Chapter 1, we learned that a local area network (LAN) is a computer network that is designed for a limited geographic area such as a building or a campus. Although a LAN can be used as an isolated network to connect computers in an organization for the sole purpose of sharing resources, most LANs today are also linked to a wide area network (WAN) or the Internet.

In the 1980s and 1990s several different types of LANs were used. All of these LANs used a media-access method to solve the problem of sharing the media. The Ethernet used the CSMA/CD approach. The Token Ring, Token Bus, and FDDI (Fiber Distribution Data Interface) used the token-passing approach. During this period, another LAN technology, ATM LAN, which deployed the high speed WAN technology (ATM), appeared in the market.

Almost every LAN except Ethernet has disappeared from the marketplace because Ethernet was able to update itself to meet the needs of the time. Several reasons for this success have been mentioned in the literature, but we believe that the Ethernet protocol was designed so that it could evolve with the demand for higher transmission rates. It is natural that an organization that has used an Ethernet LAN in the past and now needs a higher data rate would update to the new generation instead of switching to another technology, which might cost more. This means that we confine our discussion of wired LANs to the discussion of Ethernet.

The Ethernet LAN was developed in the 1970s by Robert Metcalfe and David Boggs. Since then, it has gone through four generations: **Standard Ethernet** (10 Mbps), **Fast Ethernet** (100 Mbps), **Gigabit Ethernet** (1 Gbps), and **10 Gigabit Ethernet** (10 Gbps).

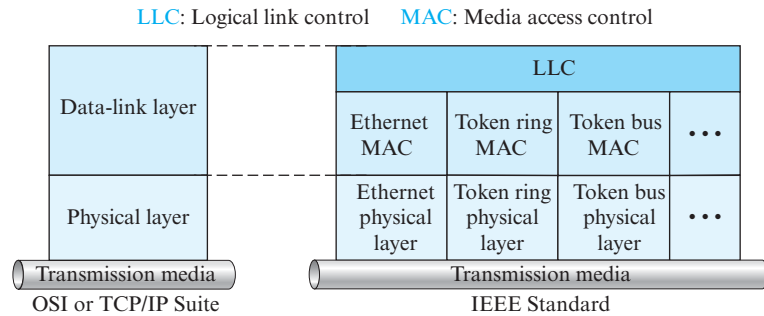
5.5.1 IEEE Project 802

Before we discuss the Ethernet protocol and all its generations, we need to briefly discuss the IEEE standard that we often encounter in text or real life. In 1985, the Computer Society of the IEEE started a project, called **Project 802**, to set standards to

enable intercommunication among equipment from a variety of manufacturers. Project 802 does not seek to replace any part of the OSI model or TCP/IP protocol suite. Instead, it is a way of specifying functions of the physical layer and the data-link layer of major LAN protocols.

The relationship of the 802 Standard to the TCP/IP protocol suite is shown in Figure 5.54. The IEEE has subdivided the data-link layer into two sublayers: **logical link control (LLC)** and **media access control (MAC)**. IEEE has also created several physical-layer standards for different LAN protocols.

Figure 5.54 IEEE standard for LANs



Logical Link Control (LLC)

Earlier we discussed *data link control*. We said that data link control handles framing, flow control, and error control. In IEEE Project 802, flow control, error control, and part of the framing duties are collected into one sublayer called the *logical link control* (LLC). Framing is handled in both the LLC sublayer and the MAC sublayer.

The LLC provides a single link-layer control protocol for all IEEE LANs. This means LLC protocol can provide interconnectivity between different LANs because it makes the MAC sublayer transparent.

Media Access Control (MAC)

Earlier we discussed multiple access methods including random access, controlled access, and channelization. IEEE Project 802 has created a sublayer called media access control that defines the specific access method for each LAN. For example, it defines CSMA/CD as the media access method for Ethernet LANs and defines the token-passing method for Token Ring and Token Bus LANs. As we mentioned in the previous section, part of the framing function is also handled by the MAC layer.

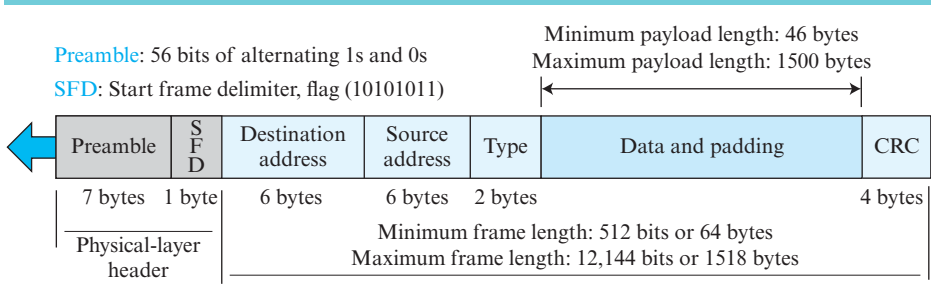
5.5.2 Standard Ethernet

We refer to the original Ethernet technology with the data rate of 10 Mbps as the Standard Ethernet. Although most implementations have moved to other technologies in the Ethernet evolution, there are some features of the Standard Ethernet that have not changed during the evolution. We discuss this standard version to pave the way for understanding the other three technologies.

Frame Format

The Ethernet frame contains seven fields, as shown in Figure 5.55.

Figure 5.55 Ethernet frame



- ❑ **Preamble.** This field contains 7 bytes (56 bits) of alternating 0s and 1s that alert the receiving system to the coming frame and enable it to synchronize its clock if it's out of synchronization. The pattern provides only an alert and a timing pulse. The 56-bit pattern allows the stations to miss some bits at the beginning of the frame. The *preamble* is actually added at the physical layer and is not (formally) part of the frame.
- ❑ **Start frame delimiter (SFD).** This field (1 byte: 10101011) signals the beginning of the frame. The SFD warns the station or stations that this is the last chance for synchronization. The last 2 bits are $(11)_2$ and alert the receiver that the next field is the destination address. This field is actually a flag that defines the beginning of the frame. We need to remember that an Ethernet frame is a variable-length frame. It needs a flag to define the beginning of the frame. The SFD field is also added at the physical layer.
- ❑ **Destination address (DA).** This field is six bytes (48 bits) and contains the link-layer address of the destination station or stations to receive the packet. We will discuss addressing shortly. When the receiver sees its own link-layer address, or a multicast address for a group that the receiver is a member of, or a broadcast address, it decapsulates the data from the frame and passes the data to the upper-layer protocol defined by the value of the type field.
- ❑ **Source address (SA).** This field is also six bytes and contains the link-layer address of the sender of the packet. We will discuss addressing shortly.
- ❑ **Type.** This field defines the upper-layer protocol whose packet is encapsulated in the frame. This protocol can be IP, ARP, OSPF, and so on. In other words, it serves the same purpose as the protocol field in a datagram and the port number in a segment or user datagram. It is used for multiplexing and demultiplexing.
- ❑ **Data.** This field carries data encapsulated from the upper-layer protocols. It is a minimum of 46 and a maximum of 1500 bytes. We discuss the reason for these minimum and maximum values shortly. If the data coming from the upper layer is more than 1500 bytes, it should be fragmented and encapsulated in more than one frame. If it is less than 46 bytes, it needs to be padded with extra 0s. A padded data frame is delivered to the upper-layer protocol as it is (without removing the

padding), which means that it is the responsibility of the upper layer to remove or add the padding. The upper-layer protocol needs to know the length of its data. For example, a datagram has a field that defines the length of the data.

- ❑ **CRC.** The last field contains error detection information, in this case a CRC-32. The CRC is calculated over the addresses, types, and data field. If the receiver calculates the CRC and finds that it is not zero (corruption in transmission), it discards the frame.

Connectionless and Unreliable Service

Ethernet provides a connectionless service, which means each frame sent is independent of the previous or next frame. Ethernet has no connection establishment or connection termination phases. The sender sends a frame whenever it has it; the receiver may or may not be ready for it. The sender may overwhelm the receiver with frames, which may result in dropping frames. If a frame drops, the sender will not know about it. Since IP, which is using the service of the Ethernet, is also connectionless, it will not know about it either. If the transport layer is also a connectionless protocol, such as UDP, the frame is lost and salvation may only come from the application layer. However, if the transport layer is TCP, the sender TCP does not receive acknowledgment for its segment and sends it again.

Ethernet is also unreliable like IP and UDP. If a frame is corrupted during transmission and the receiver finds out about the corruption, which has a high level of probability of happening because of the CRC-32, the receiver drops the frame silently. It is the duty of high-level protocols to find about it.

Frame Length

Ethernet has imposed restrictions on both the minimum and maximum lengths of a frame. The minimum length restriction is required for the correct operation of CSMA/CD, as we will see shortly. An Ethernet frame needs to have a minimum length of 512 bits or 64 bytes. Part of this length is the header and the trailer. If we count 18 bytes of header and trailer (6 bytes of source address, 6 bytes of destination address, 2 bytes of length or type, and 4 bytes of CRC), then the minimum length of data from the upper layer is $64 - 18 = 46$ bytes. If the upper-layer packet is less than 46 bytes, padding is added to make up the difference.

The standard defines the maximum length of a frame (without preamble and SFD field) as 1518 bytes. If we subtract the 18 bytes of header and trailer, the maximum length of the payload is 1500 bytes. The maximum length restriction has two historical reasons. First, memory was very expensive when Ethernet was designed; a maximum length restriction helped to reduce the size of the buffer. Second, the maximum length restriction prevents one station from monopolizing the shared medium, blocking other stations that have data to send.

Minimum frame length: 64 bytes	Minimum data length: 46 bytes
Maximum frame length: 1518 bytes	Maximum data length: 1500 bytes

Addressing

Each station on an Ethernet network (such as a PC, workstation, or printer) has its own **network interface card (NIC)**. The NIC fits inside the station and provides the station with a link-layer address. The Ethernet address is 6 bytes (48 bits), normally written in

hexadecimal notation, with a colon between the bytes. For example, the following shows an Ethernet MAC address:

4A:30:10:21:10:1A

Transmission of Address Bits

The way the addresses are sent out online is different from the way they are written in hexadecimal notation. The transmission is left to right, byte by byte; however, for each byte, the least significant bit is sent first and the most significant bit is sent last. This means that the bit that defines an address as unicast or multicast arrives first at the receiver. This helps the receiver to immediately know if the packet is unicast or multicast.

Example 5.18

Show how the address 47:20:1B:2E:08:EE is sent out online.

Solution

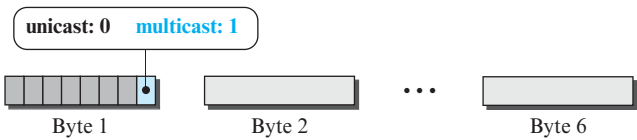
The address is sent left to right, byte by byte; for each byte, it is sent right to left, bit by bit, as shown below:

Hexadecimal	47	20	1B	2E	08	EE
Binarys	01000111	00100000	00011011	00101110	00001000	11101110
Transmitted ←	11100010	00000100	11011000	01110100	00010000	01110111

Unicast, Multicast, and Broadcast Addresses

A source address is always a *unicast address*—the frame comes from only one station. The destination address, however, can be *unicast*, *multicast*, or *broadcast*. Figure 5.56 shows how to distinguish a unicast address from a multicast address. If the least significant bit of the first byte in a destination address is 0, the address is unicast; otherwise, it is multicast.

Figure 5.56 Unicast and multicast addresses



Note that with the way the bits are transmitted, the unicast/multicast bit is the first bit which is transmitted or received. The broadcast address is a special case of the multicast address: the recipients are all the stations on the LAN. A broadcast destination address is forty-eight 1s.

Example 5.19

Define the type of the following destination addresses:

- a. 4A:30:10:21:10:1A
- b. 47:20:1B:2E:08:EE
- c. FF:FF:FF:FF:FF:FF

Solution

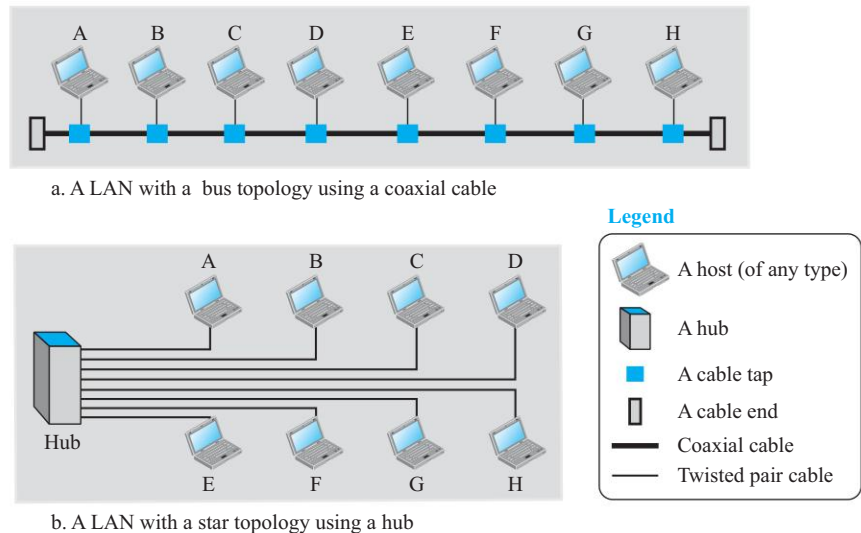
To find the type of the address, we need to look at the second hexadecimal digit from the left. If it is even, the address is unicast. If it is odd, the address is multicast. If all digits are Fs, the address is broadcast. Therefore, we have the following:

- a. This is a unicast address because A in binary is 1010 (even).
- b. This is a multicast address because 7 in binary is 0111 (odd).
- c. This is a broadcast address because all digits are Fs in hexadecimal.

Distinguish Between Unicast, Multicast, and Broadcast Transmission

Standard Ethernet uses a coaxial cable (bus topology) or a set of twisted-pair cables with a hub (star topology) as shown in Figure 5.57.

Figure 5.57 Implementation of standard Ethernet



We need to know that transmission in the standard Ethernet is always broadcast, no matter if the intention is unicast, multicast, or broadcast. In the bus topology, when station A sends a frame to station B, all stations will receive it. In the star topology, when station A sends a frame to station B, the hub will receive it. Since the hub is a passive element, it does not check the destination address of the frame; it regenerates the bits (if they have been weakened) and sends them to all stations except station A. In fact, it floods the network with the frame.

The question is, then, how the actual unicast, multicast, and broadcast transmissions are distinguished from each other. The answer is in the way the frames are kept or dropped.

- ❑ In a unicast transmission, all stations will receive the frame, the intended recipient keeps and handles the frame; the rest discard it.
- ❑ In a multicast transmission, all stations will receive the frame, the stations that are members of the group keep and handle it; the rest discard it.

- ❑ In a broadcast transmission, all stations (except the sender) will receive the frame and all stations (except the sender) keep and handle it.

Access Method

Since the network that uses the standard Ethernet protocol is a broadcast network, we need to use an access method to control the access to the sharing medium. The standard Ethernet chose CSMA/CD with 1-persistent method discussed earlier in Figures 5.37 to 5.40. Let us use a scenario to see how this method work for the Ethernet protocol.

- ❑ Assume station A in Figure 5.57 has a frame to send to station D. Station A first should check whether any other station is sending (carrier sense). Station A measures the level of energy on the medium (for a short period of time, normally less than 100 μ s). If there is no signal energy on the medium, it means that no station is sending (or the signal has not reached station A). Station A interprets this situation as idle medium. It starts sending its frame. On the other hand, if the signal energy level is not zero, it means that the medium is being used by another station. Station A continuously monitors the medium until it becomes idle for 100 μ s. It then starts sending the frame. However, station A needs to keep a copy of the frame in its buffer until it is sure that there is no collision. When station A is sure of this, is the subject we discuss next.
- ❑ The medium sensing does not stop after station A has started sending the frame. Station A needs to send and receive continuously. Two cases may occur:
 - a. Station A has sent 512 bits and no collision is sensed (the energy level did not go above the regular energy level), the station then is sure that the frame will go through and stops sensing the medium. Where does the number 512 bits come from? If we consider the transmission rate of the Ethernet as 10 Mbps, this means that it takes the station $512/(10 \text{ Mbps}) = 51.2 \mu\text{s}$ to send out 512 bits. With the speed of propagation in a cable (2×10^8 meters), the first bit could have gone 10,240 meters (one way) or only 5,120 meters (round trip), have collided with a bit from the last station on the cable, and have gone back. In other words, if a collision were to occur, it should occur by the time the sender has sent out 512 bits (worst case) and the first bit has made a round trip of 5,120 meters. We should know that if the collision happens in the middle of the cable, not at the end, station A hears the collision earlier and aborts the transmission. We also need to mention another issue. The above assumption is that the length of the cable is 5120 meters. The designer of the standard Ethernet actually put a restriction of 2500 meters because we need to consider the delays encountered throughout the journey. It means that they considered the worse case. The whole idea is that if station A does not sense the collision before sending 512 bits, there must have been no collision, because during this time, the first bit has reached the end of the line and all other stations know that a station is sending and refrain from sending. In other words, the problem occurs when another station (for example, the last station) starts sending before the first bit of station A has reached it. The other station mistakenly thinks that the line is free because the first bit has not yet reached it. The reader should notice that the restriction of

512 bits actually helps the sending station: The sending station is certain that no collision will occur if it is not heard during the first 512 bits, so it can discard the copy of the frame in its buffer.

- b. Station A has sensed a collision before sending 512 bits. This means that one of the previous bits has collided with a bit sent by another station. In this case both stations should refrain from sending and keep the frame in their buffer for resending when the line becomes available. However, to inform other stations that there is a collision in the network, the station sends a 48-bit jam signal. The jam signal is to create enough signal (even if the collision happens after a few bits) to alert other stations about the collision. After sending the jam signal, the stations need to increment the value of K (number of attempts). If after increment $K = 15$, the experience has shown that the network is too busy; the station needs to abort its effort and try again. If $K < 15$, the station can wait a back-off time (T_B in Figure 5.40) and restart the process. As Figure 5.40 shows, the station creates a random number between 0 and $2^K - 1$, which means each time the collision occurs, the range of the random number increases exponentially. After the first collision ($K = 1$) the random number is in the range (0,1). After the second collision ($K = 2$), it is in the range (0, 1, 2, 3). After the third collision ($K = 3$) it is in the range (0, 1, 2, 3, 4, 5, 6, 7). So after each collision, the probability increases that the back-off time becomes longer. This is due to the fact that if the collision happens even after the third or fourth attempt, it means that the network is really busy; a longer back-off time is needed.

Efficiency of Standard Ethernet

The efficiency of the Ethernet is defined as the ratio of the time used by a station to send data to the time the medium is occupied by this station. The practical efficiency of standard Ethernet has been measured to be

$$\text{Efficiency} = 1 / (1 + 6.4 \times a)$$

in which the parameter “ a ” is the number of the frames that can fit on the medium. It can be calculated as $a = (\text{propagation delay})/(\text{transmission delay})$ because the transmission delay is the time it takes a frame of average size to be sent out and the propagation delay is the time it takes to reach the end of the medium. Note that as the value of parameter a decreases, the efficiency increases. This means that if the length of the media is shorter or the frame size longer, the efficiency increases. In the ideal case, $a = 0$ and the efficiency is 1. We ask to calculate this efficiency in problems at the end of the chapter.

Example 5.20

In the Standard Ethernet with the transmission rate of 10 Mbps, we assume that the length of the medium is 2500 m and the size of the frame is 512 bits. The propagation speed of a signal in a cable is normally 2×10^8 m/s.

Propagation delay = $2500/(2 \times 10^8) = 12.5 \mu\text{s}$	Transmission delay = $512/(10^7) = 51.2 \mu\text{s}$
$a = 12.5/51.2 = 0.24$	Efficiency = 39%

The example shows that $a = 0.24$, which means only 0.24 of a frame occupies the whole medium in this case. The efficiency is 39 percent, which is considered moderate; it means that only 61 percent of the time the medium is occupied but not used by a station.

Implementation

The Standard Ethernet defined several implementations, but only four of them became popular during the 1980s. Table 5.6 shows a summary of Standard Ethernet implementations.

Table 5.6 Summary of Standard Ethernet implementations

Implementation	Medium	Medium Length	Encoding
10Base5	Thick coax	500 m	Manchester
10Base2	Thin coax	185 m	Manchester
10Base-T	2 UTP	100 m	Manchester
10Base-F	2 Fiber	2000	Manchester

In the nomenclature 10BaseX, the number defines the data rate (10 Mbps), the term *Base* means baseband (digital) signal, and X approximately defines either the maximum size of the cable in 100 meters (for example 5 for 500 or 2 for 185 meters) or the type of the cable, T for unshielded twisted pair cable (UTP) and F for fiber-optic. The standard Ethernet uses a baseband signal, which means that the bits are changed to a digital signal and directly sent on the line. All implementations use a Manchester encoding, which we discuss in detail in Chapter 7.

5.5.3 Fast Ethernet (100 Mbps)

In the 1990s, some LAN technologies with transmission rates higher than 10 Mbps such as FDDI and Fiber Channel, appeared on the market. If the Standard Ethernet wanted to survive, it had to compete with these technologies. Ethernet made a big jump by increasing the transmission rate to 100 Mbps, and the new generation was called the Fast Ethernet. The designers of the Fast Ethernet needed to make it compatible with the Standard Ethernet. The MAC sublayer was left unchanged, which meant the frame format and the maximum and minimum size could also remain unchanged. By increasing the transmission rate, features of the Standard Ethernet that depend on the transmission rate, access method and implementation, had to be reconsidered.

Access Method

We remember that the proper operation of the CSMA/CD depends on the transmission rate, the minimum size of the frame, and the maximum network length. If we want to keep the minimum size of the frame, the maximum length of the network should be changed. In other words, if the minimum frame size is still 512 bits, and it is transmitted 10 times faster, the collision needs to be detected 10 times sooner, which means the maximum length of the network should be 10 times shorter (the propagation speed does not change). So the Fast Ethernet came with two solutions (it can work with either choice):

1. The first solution was to totally drop the bus topology and use a passive hub and star topology but make the maximum size of the network 250 meters instead of 2500 meters as in the Standard Ethernet. This approach is kept for compatibility with the Standard Ethernet.

2. The second solution is to use a link-layer switch (discussed later in the chapter) with buffer to store frames and full-duplex connection to each host to make the transmission medium private for each host. In this case, there is no need for CSMA/CD because the hosts are not competing with each other. The link-layer switch receives a frame from a source host and stores it in the buffer (queue) waiting for processing. It then checks the destination address and sends the frame out of the corresponding interface. Since the connection to the switch is full-duplex, the destination address can even send a frame to another station at the same time that it is receiving a frame. In other words, the shared medium is changed to many point-to-point media, and there is no need for contention.

Autonegotiation

A new feature added to Fast Ethernet is called *autonegotiation*. It allows a station or a hub a range of capabilities. Autonegotiation allows two devices to negotiate the mode or data rate of operation. It was designed particularly to allow incompatible devices to connect to one another. For example, a device with a maximum data rate of 10 Mbps can communicate with a device with a 100 Mbps data rate (but which can work at a lower rate).

Implementation

Fast Ethernet implementation at the physical layer can be categorized as either two-wire or four-wire. The two-wire implementation can be either shielded twisted pair (STP), which is called 100Base-TX, or fiber-optic cable, which is called 100Base-FX. The four-wire implementation is designed only for unshielded twisted pair (UTP), which is called 100Base-T4. Table 5.7 is a summary of the Fast Ethernet implementations. We discuss encoding in Chapter 7.

Table 5.7 Summary of Fast Ethernet implementations

Implementation	Medium	Medium Length	Wires	Encoding
100Base-TX	STP	100 m	2	4B5B + MLT-3
100Base-FX	Fiber	185 m	2	4B5B + NRZ-I
100Base-T4	UTP	100 m	4	Two 8B/6T

5.5.4 Gigabit Ethernet

The need for an even higher data rate resulted in the design of the Gigabit Ethernet Protocol (1000 Mbps). The IEEE committee calls the Standard 802.3z. The goals of the Gigabit Ethernet were to upgrade the data rate to 1 Gbps, but keep the address length, the frame format, and the maximum and minimum frame length the same.

MAC Sublayer

A main consideration in the evolution of Ethernet was to keep the MAC sublayer untouched. However, to achieve a data rate of 1 Gbps, this was no longer possible. Gigabit Ethernet has two distinctive approaches for medium access: half-duplex and full-duplex. Almost all implementations of Gigabit Ethernet follow the full-duplex approach, so we ignore the half-duplex mode. In full-duplex mode, there is a central switch connected to all computers or other switches. In this mode, for each input port, each switch has buffers in which data are stored until they are transmitted. Since the switch uses the

destination address of the frame and sends a frame out of the port connected to that particular destination, there is no collision. This means that CSMA/CD is not used. Lack of collision implies that the maximum length of the cable is determined by the signal attenuation in the cable, not by the collision detection process.

Implementation

Table 5.8 is a summary of the Gigabit Ethernet implementations. S-W and L-W mean short wave and long wave respectively. We discuss the encoding in Chapter 7.

Table 5.8 Summary of Gigabit Ethernet implementations

Implementation	Medium	Medium Length	Wires	Encoding
1000Base-SX	Fiber S-W	550 m	2	8B/10B + NRZ
1000Base-LX	Fiber L-W	5000 m	2	8B/10B + NRZ
1000Base-CX	STP	25 m	2	8B/10B + NRZ
1000Base-T4	UTP	100 m	4	4D-PAM5

5.5.5 10-Gigabit Ethernet

In recent years, there has been another look into the Ethernet for use in metropolitan areas. The idea is to extend the technology, the data rate, and the coverage distance so that the Ethernet can be used as LAN and MAN (metropolitan area network). The IEEE committee created 10-Gigabit Ethernet and called it Standard 802.3ae. The goals of the 10-Gigabit Ethernet design can be summarized as upgrading the data rate to 10 Gbps, keeping the same frame size and format, and allowing the interconnection of LANs, MANs, and WAN possible. This data rate is possible only with fiber-optic technology at this time. The standard defines two types of physical layers: LAN PHY and WAN PHY. The first is designed to support existing LANs; the second actually defines a WAN with links connected through SONET OC-192 (discussed later).

Implementation

10-Gigabit Ethernet operates only in full-duplex mode, which means there is no need for contention; CSMA/CD is not used in 10-Gigabit Ethernet. Four implementations are the most common: 10GBase-SR, 10GBase-LR, 10GBase-EW, and 10GBase-X4. Table 5.9 shows a summary of the 10-Gigabit Ethernet implementations. We discuss the encoding in Chapter 7.

Table 5.9 Summary of 10-Gigabit Ethernet implementations

Implementation	Medium	Medium Length	Number of wires	Encoding
10GBase-SR	Fiber 850 nm	300 m	2	64B66B
10GBase-LR	Fiber 1310 nm	10 Km	2	64B66B
10GBase-EW	Fiber 1350 nm	40 Km	2	SONET
10GBase-X4	Fiber 1310 nm	300 m to 10 Km	2	8B10B

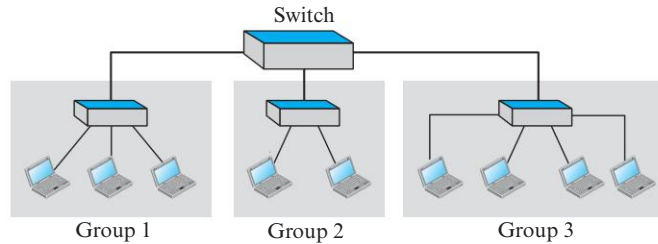
5.5.6 Virtual LANs

A station is considered part of a LAN if it physically belongs to that LAN. The criterion of membership is geographic. What happens if we need a virtual connection between two stations belonging to two different physical LANs? We can roughly define a **virtual**

local area network (VLAN) as a local area network configured by software, not by physical wiring.

Let us use an example to elaborate on this definition. Figure 5.58 shows a switched LAN in an engineering firm in which ten stations are grouped into three LANs that are connected by a switch.

Figure 5.58 *A switch connecting three LANs*

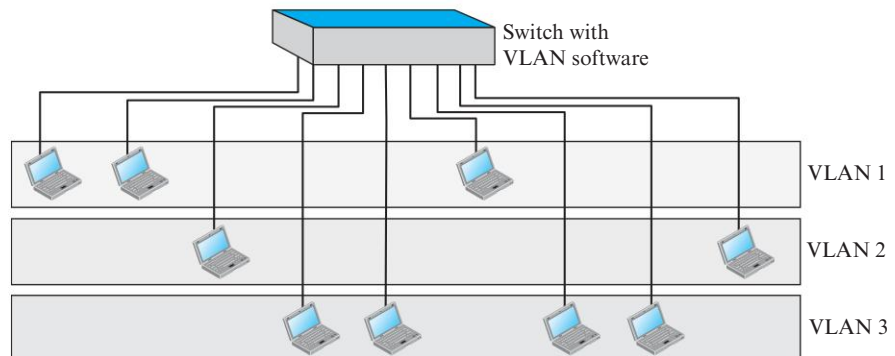


The first three engineers work together as the first group, the next two engineers work together as the second group, and the last four engineers work together as the third group. The LAN is configured to allow this arrangement.

But what would happen if the administrators needed to move two engineers from the first group to the third group, to speed up the project being done by the third group? The LAN configuration would need to be changed. The network technician must rewire. The problem is repeated if, in another week, the two engineers move back to their previous group. In a switched LAN, changes in the work group mean physical changes in the network configuration.

Figure 5.59 shows the same switched LAN divided into VLANs. The whole idea of VLAN technology is to divide a LAN into logical, instead of physical, segments. A LAN can be divided into several logical LANs called VLANs. Each VLAN is a work group in the organization. If a person moves from one group to another, there is no need to change the physical configuration. The group membership in VLANs is

Figure 5.59 *A switch using VLAN software*

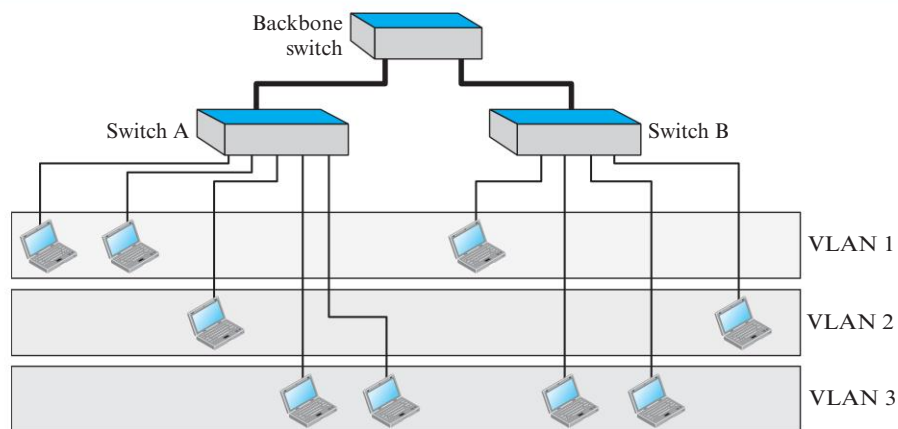


defined by software, not hardware. Any station can be logically moved to another VLAN. All members belonging to a VLAN can receive broadcast messages sent to that particular VLAN. This means if a station moves from VLAN 1 to VLAN 2, it receives broadcast messages sent to VLAN 2, but no longer receives broadcast messages sent to VLAN 1.

It is obvious that the problem in our previous example can easily be solved by using VLANs. Moving engineers from one group to another through software is easier than changing the configuration of the physical network.

VLAN technology even allows the grouping of stations connected to different switches in a VLAN. Figure 5.60 shows a backbone local area network with two switches and three VLANs. Stations from switches A and B belong to each VLAN.

Figure 5.60 Two switches in a backbone using VLAN software



This is a good configuration for a company with two separate buildings. Each building can have its own switched LAN connected by a backbone. People in the first building and people in the second building can be in the same work group even though they are connected to different physical LANs.

From these three examples, we can see that a VLAN defines broadcast domains. VLANs group stations belonging to one or more physical LANs into broadcast domains. The stations in a VLAN communicate with one another as though they belonged to a physical segment.

Membership

What characteristic can be used to group stations in a VLAN? Vendors use different characteristics such as interface number, port numbers, MAC addresses, IP addresses, IP multicast addresses, or a combination of two or more of these.

Interface Numbers

Some VLAN vendors use switch interface numbers as a membership characteristic. For example, the administrator can define that stations connecting to ports 1, 2, 3, and 7 belong to VLAN 1, stations connecting to ports 4, 10, and 12 belong to VLAN 2, and so on.

MAC Addresses

Some VLAN vendors use the 48-bit MAC address as a membership characteristic. For example, the administrator can stipulate that stations having MAC addresses E2:13:42:A1:23:34 and F2:A1:23:BC:D3:41 belong to VLAN 1.

IP Addresses

Some VLAN vendors use the 32-bit IP address (see Chapter 4) as a membership characteristic. For example, the administrator can stipulate that stations having IP addresses 181.34.23.67, 181.34.23.72, 181.34.23.98, and 181.34.23.112 belong to VLAN 1.

Multicast IP Addresses

Some VLAN vendors use the multicast IP address (see Chapter 4) as a membership characteristic. Multicasting at the IP layer is now translated to multicasting at the data-link layer.

Combination

Recently, the software available from some vendors allows all these characteristics to be combined. The administrator can choose one or more characteristics when installing the software. In addition, the software can be reconfigured to change the settings.

Configuration

How are the stations grouped into different VLANs? Stations are configured in one of three ways: manual, semiautomatic, and automatic.

Manual Configuration

In a manual configuration, the network administrator uses the VLAN software to manually assign the stations into different VLANs at setup. Later migration from one VLAN to another is also done manually. Note that this is not a physical configuration; it is a logical configuration. The term *manually* here means that the administrator types the port numbers, the IP addresses, or other characteristics, using the VLAN software.

Automatic Configuration

In an automatic configuration, the stations are automatically connected or disconnected from a VLAN using criteria defined by the administrator. For example, the administrator can define the project number as the criterion for being a member of a group. When a user changes projects, he or she automatically migrates to a new VLAN.

Semiautomatic Configuration

A semiautomatic configuration is somewhere between a manual configuration and an automatic configuration. Usually, the initializing is done manually, with migrations done automatically.

Communication between Switches

In a multi-switched backbone, each switch must know not only which station belongs to which VLAN, but also the membership of stations connected to other switches. For example, in Figure 5.60, switch A must know the membership status of stations connected to switch B, and switch B must know the same about switch A. Three methods have been devised for this purpose: table maintenance, frame tagging, and time-division multiplexing.

Table Maintenance

In this method, when a station sends a broadcast frame to its group members, the switch creates an entry in a table and records station membership. The switches send their tables to one another periodically for updating.

Frame Tagging

In this method, when a frame is traveling between switches, an extra header is added to the MAC frame to define the destination VLAN. The frame tag is used by the receiving switches to determine the VLANs to be receiving the broadcast message.

Time-Division Multiplexing (TDM)

In this method, the connection (trunk) between switches is divided into time-shared channels (see TDM in Chapter 7). For example, if the total number of VLANs in a backbone is five, each trunk is divided into five channels. The traffic destined for VLAN 1 travels in channel 1, the traffic destined for VLAN 2 travels in channel 2, and so on. The receiving switch determines the destination VLAN by checking the channel from which the frame arrived.

IEEE Standard

In 1996, the IEEE 802.1 subcommittee passed a standard called 802.1Q that defines the format for frame tagging. The standard also defines the format to be used in multi-switched backbones and enables the use of multivendor equipment in VLANs. IEEE 802.1Q has opened the way for further standardization in other issues related to VLANs. Most vendors have already accepted the standard.

Advantages

There are several advantages to using VLANs.

Cost and Time Reduction

VLANs can reduce the migration cost of stations going from one group to another. Physical reconfiguration takes time and is costly. Instead of physically moving one station to another segment or even to another switch, it is much easier and quicker to move it by using software.

Creating Virtual Work Groups

VLANs can be used to create virtual work groups. For example, in a campus environment, professors working on the same project can send broadcast messages to one another without the necessity of belonging to the same department. This can reduce traffic if the multicasting capability of IP was previously used.

Security

VLANs provide an extra measure of security. People belonging to the same group can send broadcast messages with the guaranteed assurance that users in other groups will not receive these messages.

5.6 OTHER WIRED NETWORKS

As we discussed in Chapter 1, the networks that we encounter in the Internet are either LANs or WANs. However, sometimes the terminology is under dispute. For example, some access networks such as dial-up connection or cable connection are called WANs

by some people and MANs (Metropolitan Area Networks) by others. In this section, we discuss some of these networks that are used, directly or indirectly, in the Internet without calling them MAN or WAN; we just call them networks.

5.6.1 Point-to-Point Networks

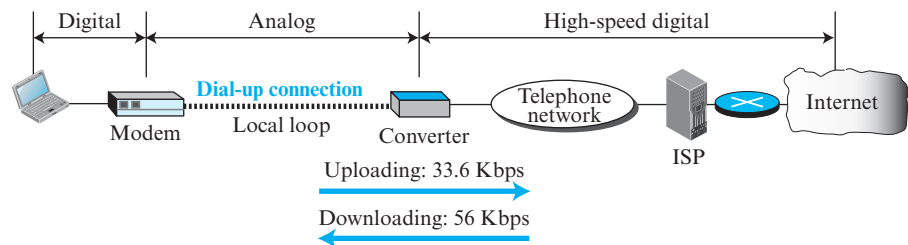
Some point-to-point networks, such as dial-up, DSL, and cable are used to provide internet access from Internet user premises. Since these networks use a dedicated connection between the two devices, they do not use media access control (MAC). The only protocol that is needed is PPP, as we discussed before.

Dial-up

A dial-up network or connection uses the services provided by the telephone networks to transmit data. The telephone network had its beginnings in the late 1800s. The entire network, which is referred to as the *plain old telephone system (POTS)*, was originally an analog system to transmit voice. With the advent of the computer era, the network, in the 1980s, began to carry data in addition to voice. During the last decade, the telephone network has undergone many technical changes. In fact, most of the telephone network is now digital. The only section that is analog is the line that connects the subscriber to the telephone network. The need to communicate digital data resulted in the invention of the dial-up modem.

The term **modem** is a composite word that refers to the two functional entities that make up the device: a signal *modulator* and a signal *demodulator*. A **modulator** creates an analog signal from digital data. A **demodulator** recovers the digital data from the modulated signal. These connections may be used only if one party is using digital signaling (such as through an Internet provider). They are asymmetric in that the downloading rate (flow of data from the Internet service provider to the PC) is a maximum of 56 kbps, while the uploading rate (flow of data from the PC to the Internet provider) can be a maximum of 33.6 kbps, as shown in Figure 5.61.

Figure 5.61 Dial-up network to provide Internet access



In *uploading*, the analog signal must still be sampled before entering into the high-speed digital telephone network. In this direction, quantization noise (as we see in Chapter 7) is introduced into the signal, which reduces the rate to 33.6 kbps. However, there is no sampling in the *downloading*. The signal is not affected by quantization noise. The individual user who accesses the Internet normally needs a higher speed for downloading than for uploading. These users, for example, want to download a large file more often than they want to upload one, so the asymmetric speed normally remain

unnoticed. We need to mention, however, that when a telephone line is used for Internet access, it cannot be used for voice communication at the same time.

One may wonder how we arrive at the 56-kbps speed. The telephone companies sample 8000 times per second with 8 bits per sample. One of the bits in each sample is used for control purposes, which means each sample is 7 bits. The rate is therefore 8000×7 , or 56,000 bps or 56 kbps.

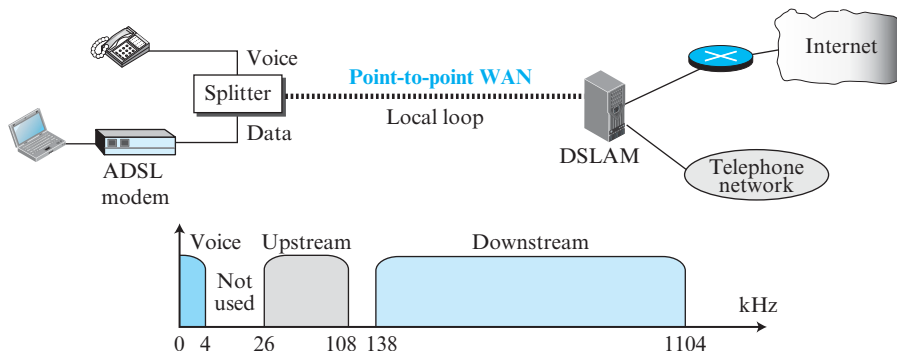
Digital Subscriber Line (DSL)

After traditional modems reached their peak data rate, telephone companies developed another technology, DSL, to provide higher-speed access to the Internet. **Digital subscriber line (DSL)** technology is one of the most promising for supporting high-speed digital communication over the existing telephone. DSL technology is a set of technologies, each differing in the first letter (ADSL, VDSL, HDSL, and SDSL). The set is often referred to as *xDSL*, where *x* can be replaced by A, V, H, or S. We just discuss the first, ADSL. The first technology in the set is *asymmetric DSL (ADSL)*. ADSL, like a 56K modem, provides higher speed (bit rate) in the downstream direction (from the Internet to the resident) than in the upstream direction (from the resident to the Internet). That is the reason it is called asymmetric. Unlike the asymmetry in 56K modems, the designers of ADSL specifically divided the available bandwidth of the local loop unevenly for the residential customer. The service is not suitable for business customers who need a large bandwidth in both directions.

Using Existing Local Loops

One interesting point is that ADSL uses the existing telephone lines (local loop). But how does ADSL reach a data rate that was never achieved with traditional modems? The answer is that the twisted-pair cable used in telephone lines is actually capable of handling bandwidths up to 1.1 MHz, but the filter installed at the end office of the telephone company where each local loop terminates limits the bandwidth to 4 kHz (sufficient for voice communication). If the filter is removed, however, the entire 1.1 MHz is available for data and voice communications. Typically, an available bandwidth of 1.104 MHz is divided into a voice channel, an upstream channel, and a downstream channel, as shown in Figure 5.62.

Figure 5.62 ADSL point-to-point network



ADSL allows the subscriber to use the voice channel and the data channel at the same time. The rate for the upstream can reach 1.44-Mbps. However, the data rate is normally below 500 kbps because of the high-level noise in this channel. The downstream data rate can reach 13.4 Mbps. However, the data rate is normally below 8 Mbps because of noise in this channel. A very interesting point is that the telephone company in this case serves as the ISP, so services such as email or Internet access are provided by the telephone company itself.

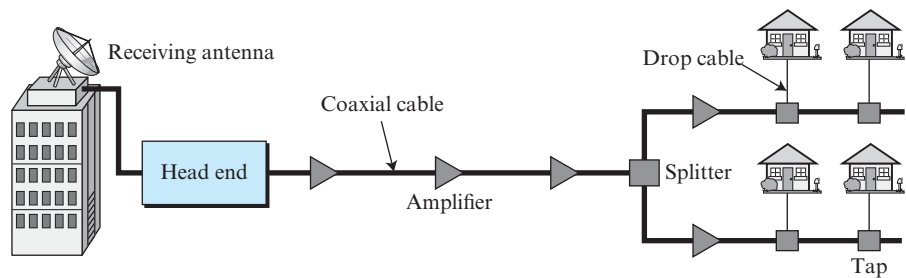
Cable

Cable networks were originally created to provide access to TV programs for those subscribers who had no reception because of natural obstructions such as mountains. Later the cable networks became popular with people who just wanted a better signal. In addition, cable networks enabled access to remote broadcasting stations via microwave connections. Cable TV also found a good market in Internet access provision, using some of the channels originally designed for video. After discussing the basic structure of cable networks, we discuss how cable modems can provide a high-speed connection to the Internet.

Traditional Cable Networks

Cable TV started to distribute broadcast video signals to locations with poor or no reception in the late 1940s. It was called **community antenna TV (CATV)** because an antenna at the top of a tall hill or building received the signals from the TV stations and distributed them, via coaxial cables, to the community. Figure 5.63 shows a schematic diagram of a traditional cable TV network.

Figure 5.63 Traditional cable TV network



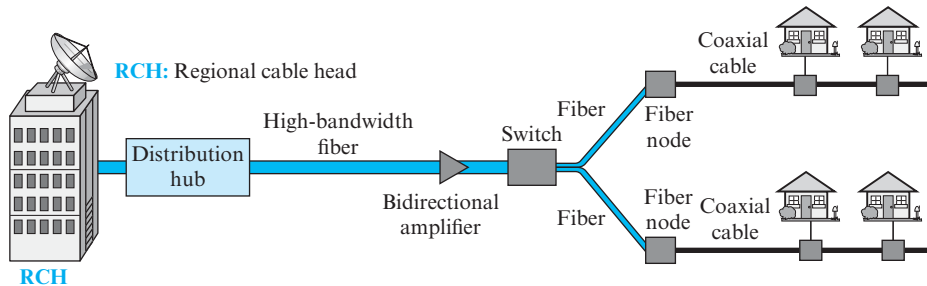
The cable TV office, called the *head end*, receives video signals from broadcasting stations and feeds the signals into coaxial cables. The signals became weaker and weaker with distance, so amplifiers were installed throughout the network to renew the signals. There could be up to 35 amplifiers between the head end and the subscriber premises. At the other end, splitters split the cable, and taps and drop cables make the connections to the subscriber premises.

The traditional cable TV system used coaxial cable end to end. Due to attenuation of the signals and the use of a large number of amplifiers, communication in the traditional network was unidirectional (one-way). Video signals were transmitted downstream, from the head end to the subscriber premises.

Hybrid Fiber-Coaxial (HFC) Network

The second generation of cable networks is called a **hybrid fiber-coaxial (HFC) network**. The network uses a combination of fiber-optic and coaxial cable. The transmission medium from the cable TV office to a box, called the *fiber node*, is optical fiber; from the fiber node through the neighborhood and into the house is still coaxial cable. Figure 5.64 shows a schematic diagram of an HFC network.

Figure 5.64 Hybrid fiber-coaxial (HFC) network



The *regional cable head (RCH)* normally serves up to 400,000 subscribers. The RCHs feed the *distribution hubs*, each of which serves up to 40,000 subscribers. The distribution hub plays an important role in the new infrastructure. Modulation and distribution of signals are done here; the signals are then fed to the fiber nodes through fiber-optic cables. The fiber node splits the analog signals so that the same signal is sent to each coaxial cable. Each coaxial cable serves up to 1000 subscribers. The use of fiber-optic cable reduces the need for amplifiers down to eight or less.

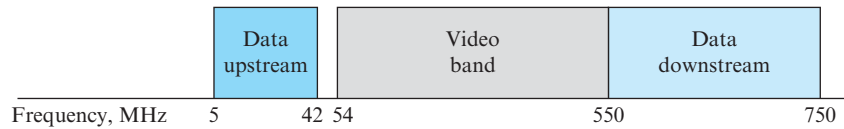
One reason for moving from traditional to hybrid infrastructure is to make the cable network bidirectional (two-way).

Cable TV for Data Transfer

Cable companies are now competing with telephone companies for the residential customer who wants high-speed data transfer. DSL technology provides high-data-rate connections for residential subscribers over the local loop. However, DSL uses the existing unshielded twisted-pair cable, which is very susceptible to interference. This imposes an upper limit on the data rate. A solution is the use of the cable TV network. In this section, we briefly discuss this technology.

Even in an HFC system, the last part of the network, from the fiber node to the subscriber premises, is still a coaxial cable. This coaxial cable has a bandwidth that ranges from 5 to 750 MHz (approximately). To provide Internet access, the cable company has divided this bandwidth into three bands: video, downstream data, and upstream data, as shown in Figure 5.65.

The *video band* occupies frequencies from 54 to 550 MHz. Since each TV channel occupies 6 MHz, this can accommodate more than 80 channels. The downstream data (from the Internet to the subscriber premises) occupies the upper band, from 550 to 750 MHz. This band is also divided into 6-MHz channels.

Figure 5.65 Division of coaxial cable band by CATV

The upstream data (from the subscriber premises to the Internet) occupies the lower band, from 5 to 42 MHz. This band is also divided into 6-MHz channels. There are 2 bits/ baud in QPSK. The standard specifies 1 Hz for each baud; this means that, theoretically, upstream data can be sent at 12 Mbps (2 bits/Hz $\times$ 6 MHz). However, the data rate is usually less than 12 Mbps.

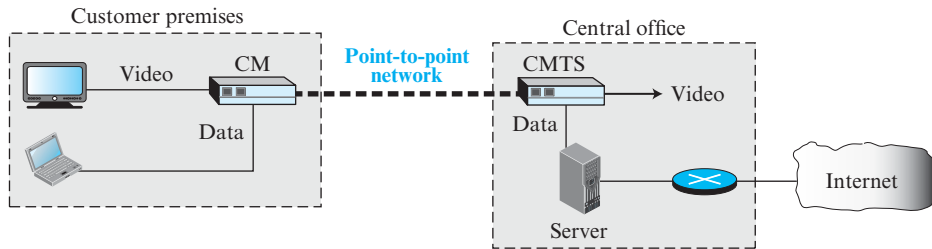
Sharing

Both upstream and downstream bands are shared by the subscribers. The upstream data bandwidth is 37 MHz. This means that there are only six 6-MHz channels available in the upstream direction. A subscriber needs to use one channel to send data in the upstream direction. The question is, “How can six channels be shared in an area with 1000, 2000, or even 100,000 subscribers?” The solution is time-sharing. The band is divided into channels; these channels must be shared between subscribers in the same neighborhood. The cable provider allocates one channel, statically or dynamically, for a group of subscribers. If one subscriber wants to send data, she or he contends for the channel with others who want access; the subscriber must wait until the channel is available.

We have a similar situation in the downstream direction. The downstream band has 33 channels of 6 MHz. A cable provider probably has more than 33 subscribers; therefore, each channel must be shared between a group of subscribers. However, the situation is different for the downstream direction; here we have a multicasting situation. If there are data for any of the subscribers in the group, the data are sent to that channel. Each subscriber is sent the data. But since each subscriber also has an address registered with the provider; the cable modem for the group matches the address carried with the data to the address assigned by the provider. If the address matches, the data are kept; otherwise, they are discarded.

CM and CMTS

To use a cable network for data transmission, we need two key devices: a **cable modem (CM)** and a **cable modem transmission system (CMTS)**. The **cable modem** is installed on the subscriber premises. The **cable modem transmission system** is installed inside the cable company. It receives data from the Internet and sends them to the subscriber. The CMTS also receives data from the subscriber and passes them to the Internet. It is similar to an ADSL modem. Figure 5.66 shows the location of these two devices. Like DSL technology, the cable company needs to become an ISP and provide Internet services to the subscriber. At the subscriber premises, the CM separates the video from data and sends them to the television set or the computer.

Figure 5.66 Cable modem transmission system (CMTS)

5.6.2 SONET

In this section, we introduce a high-speed network, SONET, that is used as a transport network to carry loads from other networks. We first discuss SONET as a protocol, and we then show how SONET networks can be constructed from the standards defined in the protocol.

The high bandwidths of fiber-optic cable are suitable for today's high-data-rate technologies (such as video conferencing) and for carrying large numbers of lower-data-rate technologies at the same time. For this reason, the importance of fiber-optics grows in conjunction with the development of technologies requiring high data rates or wide bandwidths for transmission. With their prominence came a need for standardization. The United States (ANSI) and Europe (ITU-T) have responded by defining standards that, though independent, are fundamentally similar and ultimately compatible. The ANSI standard is called the **Synchronous Optical Network (SONET)**, which we discuss here.

Architecture

SONET is a synchronous network using synchronous TDM multiplexing. All clocks in the system are locked to a master clock. We first introduce the architecture of a SONET system: signals, devices, and connections.

Signals

SONET defines a hierarchy of electrical signaling levels called **synchronous transport signals (STSs)**. Each STS level (STS-1 to STS-192) supports a certain data rate, specified in megabits per second (see Table 5.10). The corresponding optical signals are called **optical carriers (OCs)**.

Table 5.10 SONET rates

STS	OC	Rate (Mbps)	STS	OC	Rate (Mbps)
STS-1	OC-1	51.840	STS-24	OC-24	1244.160
STS-3	OC-3	155.520	STS-36	OC-36	1866.230
STS-9	OC-9	466.560	STS-48	OC-48	2488.320
STS-12	OC-12	622.080	STS-96	OC-96	4976.640
STS-18	OC-18	933.120	STS-192	OC-192	9953.280

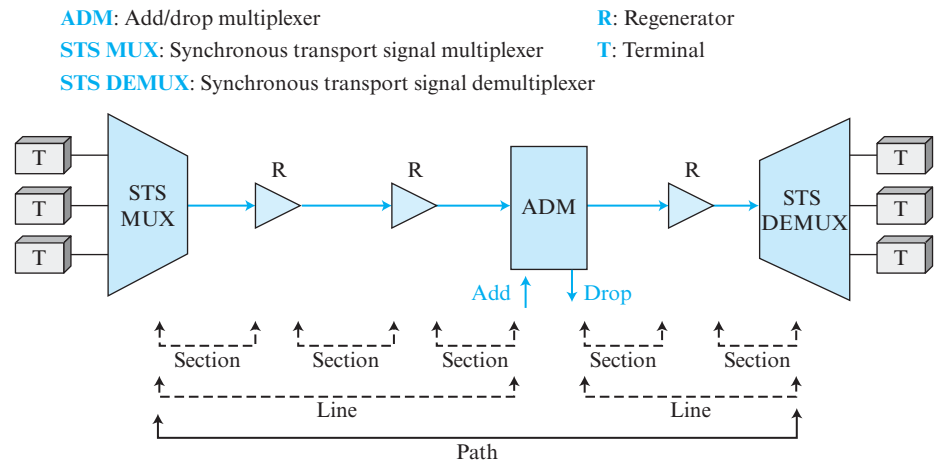
A glance through Table 5.10 reveals some interesting points. First, the lowest level in this hierarchy has a data rate of 51.840 Mbps, which is greater than that of

the DS-3 service (44.736 Mbps, explained in Chapter 7). In fact, the STS-1 is designed to accommodate data rates equivalent to those of the DS-3. The difference in capacity is provided to handle the overhead needs of the optical system. Second, the STS-3 rate is exactly three times the STS-1 rate; and the STS-9 rate is exactly one-half the STS-18 rate. These relationships mean that 18 STS-1 channels can be multiplexed into one STS-18, six STS-3 channels can be multiplexed into one STS-18, and so on.

SONET Devices

Figure 5.67 shows a simple link using SONET devices. SONET transmission relies on three basic devices: STS multiplexers/demultiplexers, regenerators, add/drop multiplexers and terminals.

Figure 5.67 A simple network using SONET equipment



STS Multiplexer/Demultiplexer STS multiplexers/demultiplexers mark the beginning points and endpoints of a SONET link. They provide the interface between an electrical tributary network and the optical network. An *STS multiplexer* multiplexes signals from multiple electrical sources and creates the corresponding OC signal. An *STS demultiplexer* demultiplexes an optical OC signal into corresponding electric signals.

Regenerator Regenerators extend the length of the links. A *regenerator* is a repeater (discussed later) that takes a received optical signal (OC-*n*), demodulates it into the corresponding electric signal (STS-*n*), regenerates the electric signal, and finally modulates the electric signal into its corresponding OC-*n* signal. A SONET regenerator replaces some of the existing overhead information (header information) with new information.

Add/Drop Multiplexer Add/drop multiplexers allow insertion and extraction of signals. An *add/drop multiplexer (ADM)* can add STSs coming from different sources into a given path or can remove a desired signal from a path and redirect it without demultiplexing the entire signal. Instead of relying on timing and bit positions, add/drop

multiplexers use header information such as addresses and pointers (described later in this section) to identify individual streams.

Terminals A *terminal* is a device that uses the services of a SONET network. For example, in the Internet, a terminal can be a router that needs to send packets to another router at the other side of a SONET network.

Connections

The devices defined in the previous section are connected using *sections*, *lines*, and *paths*.

Sections

A *section* is the optical link connecting two neighbor devices: multiplexer to multiplexer, multiplexer to regenerator, or regenerator to regenerator.

Lines

A *line* is the portion of the network between two multiplexers.

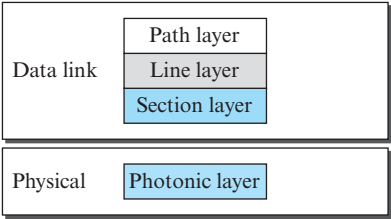
Paths

A *path* is the end-to-end portion of the network between two STS multiplexers. In a simple SONET of two STS multiplexers linked directly to each other, the section, line, and path are the same.

SONET Layers

The SONET standard includes four functional layers: the photonic, the section, the line, and the path layer. They correspond to both the physical and data-link layers. The headers added to the frame at the various layers are discussed later in this chapter. Figure 5.68 shows the layers and the relation between the layers and devices described before.

Figure 5.68 SONET layers compared with OSI or the Internet layers



Path Layer

The *path layer* is responsible for the movement of a signal from its optical source to its optical destination. At the optical source, the signal is changed from an electronic form into an optical form, multiplexed with other signals, and encapsulated in a frame. At the optical destination, the received frame is demultiplexed, and the individual optical signals are changed back into their electronic forms. Path-layer overhead is added at this layer. STS multiplexers provide path-layer functions.

Line Layer

The *line layer* is responsible for the movement of a signal across a physical line. Line-layer overhead is added to the frame at this layer. STS multiplexers and add/drop multiplexers provide line-layer functions.

Section Layer

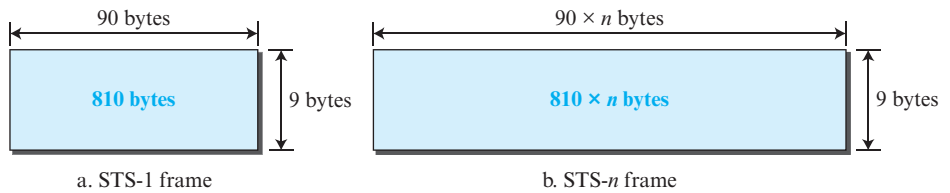
The *section layer* is responsible for the movement of a signal across a physical section. It handles framing, scrambling, and error control. Section-layer overhead is added to the frame at this layer.

Photonic Layer

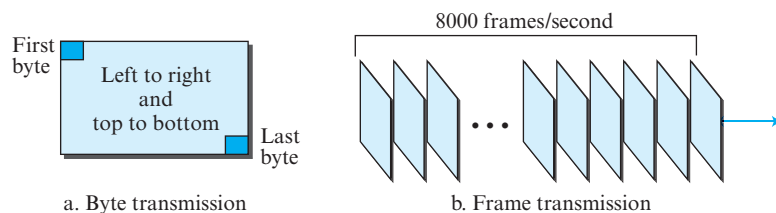
The *photonic layer* corresponds to the physical layer. It includes physical specifications for the optical fiber channel, the sensitivity of the receiver, multiplexing functions, and so on. SONET uses NRZ encoding (See Chapter 7) with the presence of light representing 1 and the absence of light representing 0.

SONET Frames

Each synchronous transfer signal STS- n is composed of 8000 frames. Each frame is a two-dimensional matrix of bytes with 9 rows by $90 \times n$ columns. For example, an STS-1 frame is 9 rows by 90 columns (810 bytes), and an STS-3 is 9 rows by 270 columns (2430 bytes). Figure 5.69 shows the general format of an STS-1 and an STS- n .

Figure 5.69 An STS-1 and an STS- n frame**Frame, Byte, and Bit Transmission**

One of the interesting points about SONET is that each STS- n signal is transmitted at a fixed rate of 8000 frames per second. This is the rate at which voice is digitized (see Chapter 7). For each frame the bytes are transmitted from the left to the right, top to the bottom. For each byte, the bits are transmitted from the most significant to the least significant (left to right). Figure 5.70 shows the order of frame and byte transmission.

Figure 5.70 STS-1 frames in transition

A SONET STS- n signal is transmitted at 8000 frames per second.

If we sample a voice signal and use 8 bits (1 byte) for each sample, we can say that each byte in a SONET frame can carry information from a digitized voice channel. In other words, an STS-1 signal can carry 774 voice channels simultaneously (810 minus 36 required bytes for overhead).

Each byte in a SONET frame can carry a digitized voice channel.

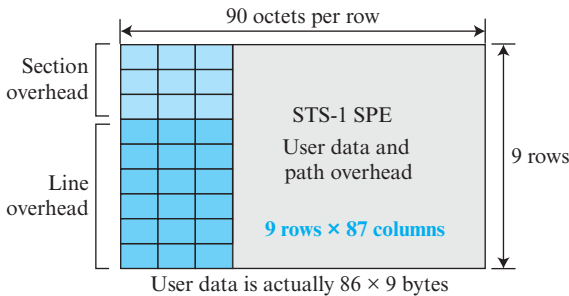
Note that in SONET, there is an exact relationship between the data rates of different STS signals. We could have found the data rate of STS-3 by using the data rate of STS-1 (multiply the latter by 3).

In SONET, the data rate of an STS-*n* signal is *n* times the data rate of an STS-1 signal.

STS-1 Frame Format

The basic format of an STS-1 frame is shown in Figure 5.71. As we said before, a SONET frame is a matrix of 9 rows of 90 bytes (octets) each, for a total of 810 bytes.

Figure 5.71 STS-1 frame overheads



STS Multiplexing

In SONET, frames of a lower rate can be synchronously time-division multiplexed into a higher-rate frame. For example, three STS-1 signals (channels) can be combined into one STS-3 signal (channel), four STS-3s can be multiplexed into one STS-12, and so on.

Multiplexing is synchronous TDM, and all clocks in the network are locked to a master clock to achieve synchronization.

In SONET, all clocks in the network are locked to a master clock.

We need to mention that multiplexing can also take place at the higher data rates. For example, four STS-3 signals can be multiplexed into an STS-12 signal. However, the STS-3 signals need to first be demultiplexed into 12 STS-1 signals, and then these twelve signals need to be multiplexed into an STS-12 signal. The reason for this extra work will be clear after our discussion on byte interleaving.

Add/Drop Multiplexer

Multiplexing of several STS-1 signals into an STS- n signal is done at the STS multiplexer (at the path layer). Demultiplexing of an STS- n signal into STS-1 components is done at the STS demultiplexer. In between, however, SONET uses add/drop multiplexers that can replace a signal with another one. We need to know that this is not demultiplexing/multiplexing in the conventional sense. An add/drop multiplexer operates at the line layer. An add/drop multiplexer does not create section, line, or path overhead. It almost acts as a switch; it removes one STS-1 signal and adds another one. The type of signal at the input and output of an add/drop multiplexer is the same (both STS-3 or both STS-12, for example). The add/drop multiplexer (ADM) only removes the corresponding bytes and replaces them with the new bytes (including the bytes in the section and line overhead).

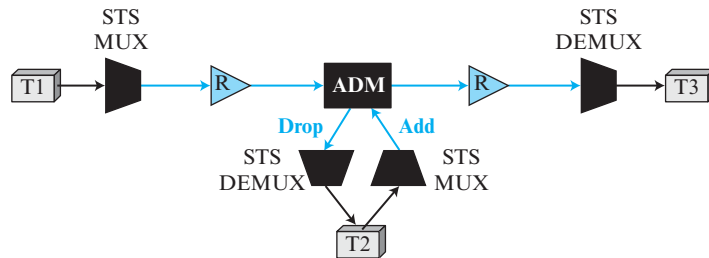
SONET Networks

Using SONET equipment, we can create a SONET network that can be used as a high-speed backbone carrying loads from other networks such as ATM or IP. We can roughly divide SONET networks into three categories: linear, ring, and mesh networks.

Linear Network

A linear network is normally made of STS multiplexers and demultiplexers, some generators and some add/drop multiplexers, as shown in Figure 5.72.

Figure 5.72 A linear SONET network



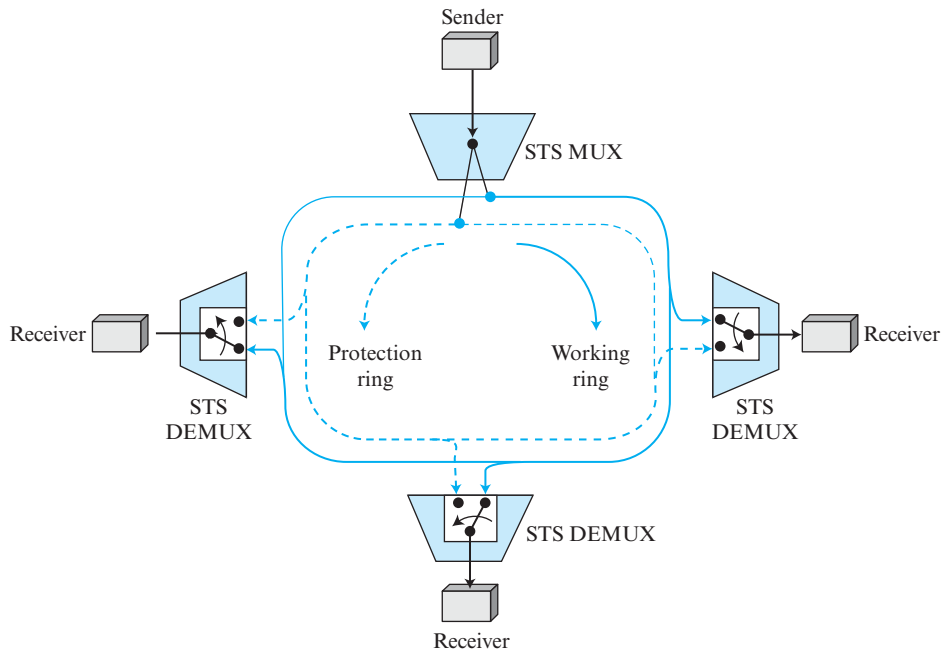
Ring Networks

ADMs make it possible to have SONET ring networks. SONET rings can be used in either a unidirectional or a bidirectional configuration. In each case, we can add extra rings to make the network self-healing, capable of self-recovery from line failure. Figure 5.73 shows a ring network.

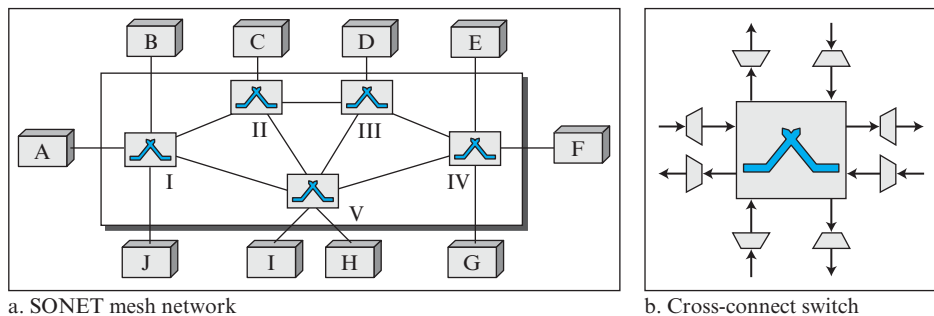
Although we have chosen one sender and three receivers in the figure, there can be many other configurations. The sender uses a two-way connection to send data to both rings simultaneously; the receiver uses selecting switches to select the ring with better signal quality. We have used one STS multiplexer and three STS demultiplexers to emphasize that nodes operate on the path layer.

Mesh Networks

One problem with ring networks is the lack of scalability. When the traffic in a ring increases, we need to upgrade not only the lines, but also the ADMs. In this situation, a

Figure 5.73 *A unidirectional path switching ring*

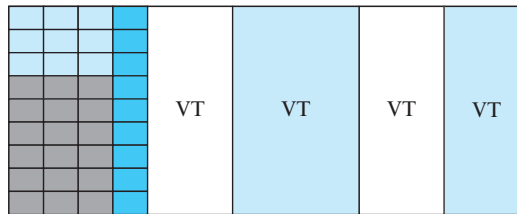
mesh network with switches probably gives better performance. A switch in a network mesh is called a cross-connect. A cross-connect, like other switches we have seen, has input and output ports. In an input port, the switch takes an OC- n signal, changes it to an STS- n signal, demultiplexes it into the corresponding STS-1 signals, and sends each STS-1 signal to the appropriate output port. An output port takes STS-1 signals coming from different input ports, multiplexes them into an STS- n signal, and makes an OC- n signal for transmission. Figure 5.74 shows a mesh SONET network, and the structure of a switch.

Figure 5.74 *A mesh SONET network*

Virtual Tributaries

SONET is designed to carry broadband payloads. Current digital hierarchy data rates (DS-1 to DS-3), however, are lower than STS-1. To make SONET backward-compatible with the current hierarchy, its frame design includes a system of **virtual tributaries (VTs)** (see Figure 5.75). A virtual tributary is a partial payload that can be inserted into an STS-1 and combined with other partial payloads to fill out the frame. Instead of using all 86 payload columns of an STS-1 frame for data from one source, we can subdivide the SPE and call each component a VT.

Figure 5.75 Virtual tributaries



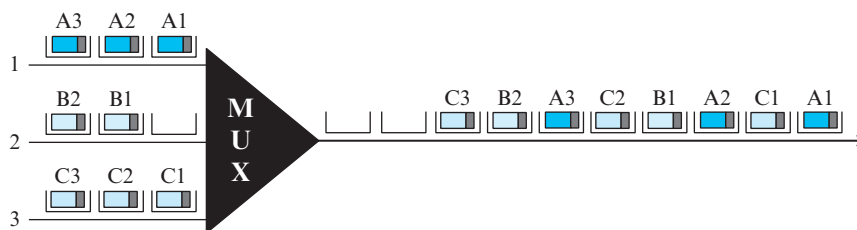
Four types of VTs have been defined to accommodate existing digital hierarchies. Notice that the number of columns allowed for each type of VT can be determined by doubling the type identification number (VT1.5 gets three columns, VT2 gets four columns, etc.). VT1.5 accommodates the U.S. DS-1 service (1.544 Mbps). VT2 accommodates the European CEPT-1 service (2.048 Mbps). VT3 accommodates the DS-1C service (fractional DS-1, 3.152 Mbps). VT6 accommodates the DS-2 service (6.312 Mbps).

When two or more tributaries are inserted into a single STS-1 frame, they are interleaved column by column. SONET provides mechanisms for identifying each VT and separating them without demultiplexing the entire stream. Discussion of these mechanisms and the control issues behind them is beyond the scope of this book.

5.6.3 Switched Network: ATM

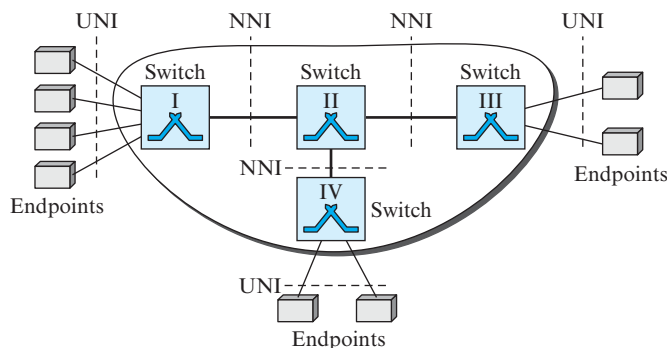
Asynchronous Transfer Mode (ATM) is a switched wide area network based on the *cell relay* protocol designed by the ATM forum and adopted by the ITU-T. The combination of ATM and SONET will allow high-speed interconnection of all the world's networks. In fact, ATM can be thought of as the “highway” of the information super-highway.

ATM uses statistical (asynchronous) time-division multiplexing—that is why it is called Asynchronous Transfer Mode—to multiplex cells coming from different channels. It uses fixed-size slots (size of a cell). ATM multiplexers fill a slot with a cell from any input channel that has a cell; the slot is empty if none of the channels has a cell to send. Figure 5.76 shows how cells from three inputs are multiplexed. At the first tick of the clock, channel 2 has no cell (empty input slot), so the multiplexer fills the slot with a cell from the third channel. When all the cells from all the channels are multiplexed, the output slots are empty.

Figure 5.76 ATM multiplexing

Architecture

ATM is a cell-switched network. The user access devices, called the endpoints, are connected through a **user-to-network interface (UNI)** to the switches inside the network. The switches are connected through **network-to-network interfaces (NNIs)**. Figure 5.77 shows an example of an ATM network.

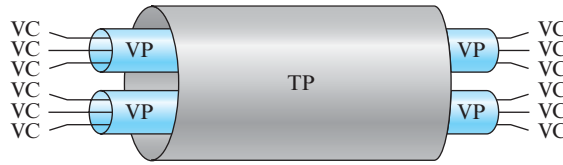
Figure 5.77 Architecture of an ATM network

Virtual Connection

Connection between two endpoints is accomplished through transmission paths (TPs), virtual paths (VPs), and virtual circuits (VCs). A **transmission path (TP)** is the physical connection (wire, cable, satellite, and so on) between an endpoint and a switch or between two switches. Think of two switches as two cities. A transmission path is the set of all highways that directly connect the two cities.

A transmission path is divided into several virtual paths. A **virtual path (VP)** provides a connection or a set of connections between two switches. Think of a virtual path as a highway that connects two cities. Each highway is a virtual path; the set of all highways is the transmission path.

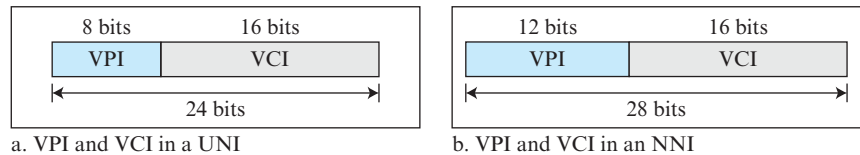
Cell networks are based on **virtual circuits (VCs)**. All cells belonging to a single message follow the same virtual circuit and remain in their original order until they reach their destination. Think of a virtual circuit as the lanes of a highway (virtual path). Figure 5.78 shows the relationship between a transmission path (a physical connection), virtual paths (a combination of virtual circuits that are bundled together

Figure 5.78 TP, VPs, and VCs

because parts of their paths are the same), and virtual circuits that logically connect two points.

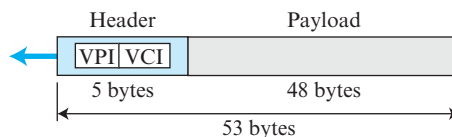
Identifiers In a virtual circuit network, to route data from one endpoint to another, the virtual connections need to be identified. For this purpose, the designers of ATM created a hierarchical identifier with two levels: a virtual-path identifier (VPI) and a virtual-circuit identifier (VCI). The VPI defines the specific VP, and the VCI defines a particular VC inside the VP. The VPI is the same for all virtual connections that are bundled (logically) into one VP.

The lengths of the VPIs for UNIs and NNIs are different. In a UNI, the VPI is 8 bits, whereas in an NNI, the VPI is 12 bits. The length of the VCI is the same in both interfaces (16 bits). We therefore can say that a virtual connection is identified by 24 bits in a UNI and by 28 bits in an NNI (see Figure 5.79).

Figure 5.79 Virtual connection identifiers in UNIs and NNIs

The whole idea behind dividing a virtual circuit identifier into two parts is to allow hierarchical routing. Most of the switches in a typical ATM network are routed using VPIs. The switches at the boundaries of the network, those that interact directly with the endpoint devices, use both VPIs and VCIs.

Cells The basic data unit in an ATM network is called a cell. A cell is only 53 bytes long with 5 bytes allocated to the header and 48 bytes carrying the payload (user data may be less than 48 bytes). Most of the header is occupied by the VPI and VCI that define the virtual connection through which a cell should travel from an endpoint to a switch or from a switch to another switch. Figure 5.80 shows the cell structure.

Figure 5.80 An ATM cell

Connection Establishment and Release

ATM uses two types of connections: PVC and SVC.

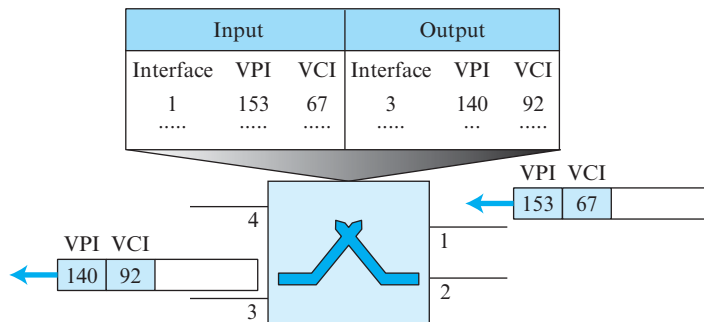
PVC A *permanent virtual-circuit connection (PVC)* is established between two endpoints by the network provider. The VPIs and VCIs are defined for the permanent connections, and the values are entered for the tables of each switch.

SVC In a *switched virtual-circuit connection (SVC)*, each time an endpoint wants to make a connection with another endpoint, a new virtual circuit must be established. ATM cannot do the job by itself, but needs the network-layer addresses and the services of another protocol (such as IP). The signaling mechanism of this other protocol makes a connection request by using the network-layer addresses of the two endpoints. The actual mechanism depends on the network-layer protocol.

Switching

ATM uses switches to route the cell from a source endpoint to the destination endpoint. A switch routes the cell using both the VPIs and the VCIs. The routing requires the whole identifier. Figure 5.81 shows how a PVC switch routes the cell. A cell with a VPI of 153 and VCI of 67 arrives at switch interface (port) 1. The switch checks its switching table, which stores six pieces of information per row: arrival interface number, incoming VPI, incoming VCI, corresponding outgoing interface number, the new VPI, and the new VCI. The switch finds the entry with the interface 1, VPI 153, and VCI 67 and discovers that the combination corresponds to output interface 3, VPI 140, and VCI 92. It changes the VPI and VCI in the header to 140 and 92, respectively, and sends the cell out through interface 3.

Figure 5.81 Routing with a switch

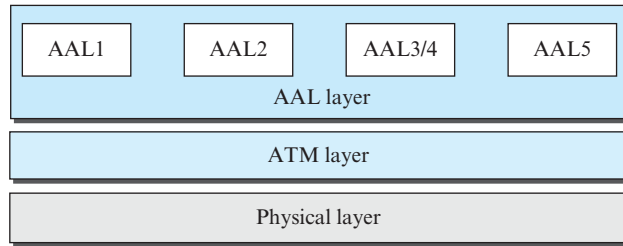


ATM Layers

The ATM standard defines three layers. They are, from top to bottom, the application adaptation layer, the ATM layer, and the physical layer (see Figure 5.82). The endpoints use all three layers while the switches use only the two bottom layers.

AAL Layer The **application adaptation layer (AAL)** was designed to enable two ATM concepts. First, ATM must accept any type of payload, both data frames and streams of bits. A data frame can come from an upper-layer protocol that creates a clearly defined frame to be sent to a carrier network such as ATM. A good example is

Figure 5.82 ATM layers



the Internet. ATM must also carry multimedia payloads. It can accept continuous bit streams and break them into chunks to be encapsulated into a cell at the ATM layer. AAL uses two sublayers to accomplish these tasks.

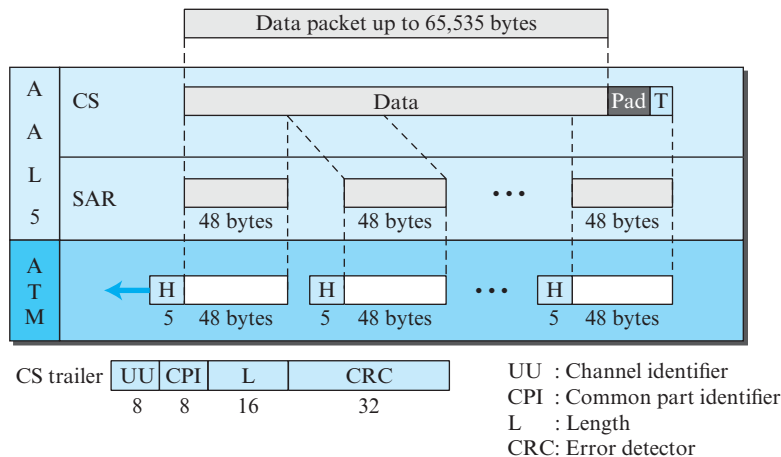
Whether the data are a data frame or a stream of bits, the payload must be segmented into 48-byte segments to be carried by a cell. At the destination, these segments need to be reassembled to recreate the original payload. The AAL defines a sublayer, called a **segmentation and reassembly (SAR)** sublayer, to do so. Segmentation is at the source; reassembly, at the destination.

Before data are segmented by SAR, they must be prepared to guarantee the integrity of the data. This is done by a sublayer called the **convergence sublayer (CS)**.

ATM defines four versions of the AAL: *AAL1*, *AAL2*, *AAL3/4*, and *AAL5*. We discuss only *AAL5* here, which is used in the Internet today.

For Internet applications, the *AAL5* sublayer was designed. It is also called the **simple and efficient adaptation layer (SEAL)**. *AAL5* assumes that all cells belonging to a single message travel sequentially and that control functions are included in the upper layers of the sending application. Figure 5.83 shows the *AAL5* sublayer.

Figure 5.83 AAL5



The packet at the CS layer uses a trailer with four fields. The UU is the user-to-user identifier. The CPI is the common part identifier. The L field defines the length of the original data. The CRC field is a two-byte error-checking field for the entire data.

ATM Layer The *ATM layer* provides routing, traffic management, switching, and multiplexing services. It processes outgoing traffic by accepting 48-byte segments from the AAL sublayers and transforming them into 53-byte cells by the addition of a 5-byte header.

Physical Layer Like Ethernet and wireless LANs, ATM cells can be carried by any physical-layer carrier.

Congestion Control and Quality of Service

ATM has a very developed congestion control and quality of service.

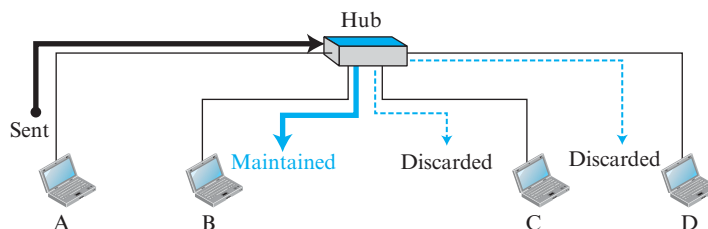
5.7 CONNECTING DEVICES

Hosts and networks do not normally operate in isolation. We use connecting devices to connect hosts together to make a network or to connect networks together to make an internet. Connecting devices can operate in different layers of the Internet model. We discuss three kinds of *connecting devices*: repeaters (or hubs), link-layer switches (or two-layer switches), and routers (or three-layer switches). Repeaters and hubs operate in the first layer of the Internet model. Link-layer switches and two-layer switches operate in the first two layers. Routers and three-layer switches operate in the first three layers.

5.7.1 Repeaters or Hubs

A **repeater** is a device that operates only in the physical layer. Signals that carry information within a network can travel a fixed distance before attenuation endangers the integrity of the data. A repeater receives a signal and, before it becomes too weak or corrupted, *regenerates* and *retimes* the original bit pattern. The repeater then sends the refreshed signal. In the past, when Ethernet LANs were using bus topology, a repeater was used to connect two segments of a LAN to overcome the length restriction of the coaxial cable. Today, however, Ethernet LANs use star topology. In a star topology, a repeater is a multiport device, often called a *hub*, that can be used to serve as the connecting point and at the same time function as a repeater. Figure 5.84 shows that when a packet from station A to station B arrives at the hub, the signal representing

Figure 5.84 Repeater or hub



the frame is regenerated to remove any possible corrupting noise, but the hub forwards the packet from all outgoing ports except the one from which the signal has received. In other words, the frame is broadcast. All stations in the LAN receive the frame, but only station B keeps it. The rest of the stations discard it. Figure 5.84 shows the role of a repeater or a hub in a switched LAN.

The figure definitely shows that a hub does not have a filtering capability; it does not have the intelligence to find from which port the frame should be sent out.

A repeater has no filtering capability.

A hub or a repeater is a physical-layer device. They do not have a link-layer address and they do not check the link-layer address of the received frame. They just regenerate the corrupted bits and send them out from every port.

5.7.2 Link-Layer Switches

A *link-layer switch* operates in both the physical and the data-link layers. As a physical-layer device, it regenerates the signal it receives. As a link-layer device, the link-layer switch can check the MAC addresses (source and destination) contained in the frame.

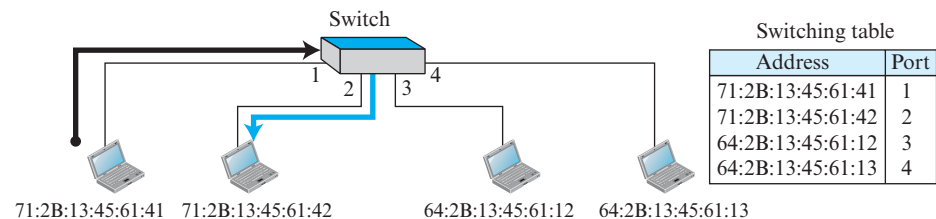
Filtering

One may ask what is the difference in functionality between a link-layer switch and a hub. A link-layer switch has *filtering* capability. It can check the destination address of a frame and can decide from which outgoing port the frame should be sent.

A link-layer switch has a table used in filtering decisions.

Let us give an example. In Figure 5.85, we have a LAN with four stations that are connected to a link-layer switch. If a frame destined for station 71:2B:13:45:61:42 arrives at port 1, the link-layer switch consults its table to find the departing port.

Figure 5.85 Link-Layer Switch



According to its table, frames for 71:2B:13:45:61:42 should be sent out only through port 2; therefore, there is no need for forwarding the frame through other ports.

A link-layer switch does not change the link-layer (MAC) addresses in a frame.

Transparent Switches

A *transparent switch* is a switch in which the stations are completely unaware of the switch's existence. If a switch is added or deleted from the system, reconfiguration of the stations is unnecessary. According to the IEEE 802.1d specification, a system equipped with transparent switches must meet three criteria:

- ❑ Frames must be forwarded from one station to another.
- ❑ The forwarding table is automatically made by learning frame movements in the network.
- ❑ Loops in the system must be prevented.

Forwarding

A transparent switch must correctly forward the frames, as discussed in the previous section.

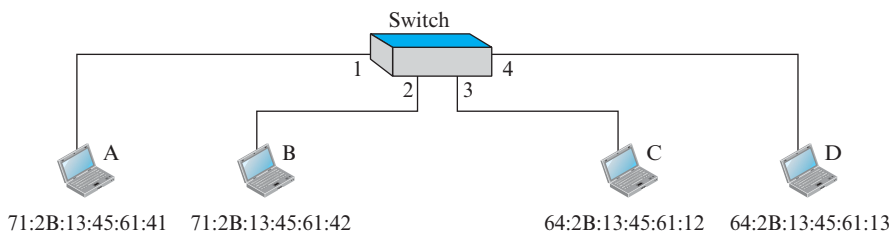
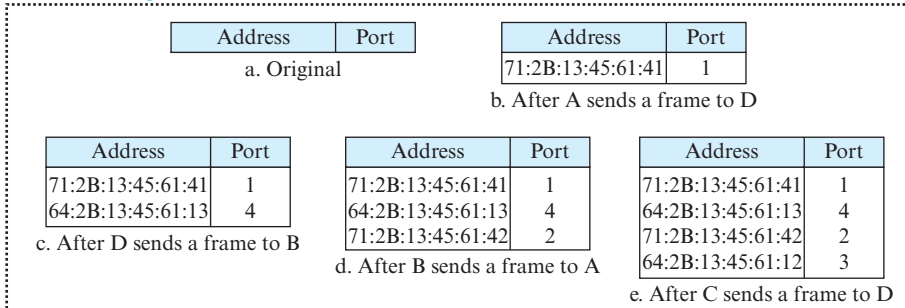
Learning

The earliest switches had switching tables that were static. The system administrator would manually enter each table entry during switch setup. Although the process was simple, it was not practical. If a station was added or deleted, the table had to be modified manually. The same was true if a station's MAC address changed, which is not a rare event. For example, putting in a new network card means a new MAC address.

A better solution to the static table is a dynamic table that maps addresses to ports (interfaces) automatically. To make a table dynamic, we need a switch that gradually learns from the frame movements. To do this, the switch inspects both the destination and the source addresses. The destination address is used for the forwarding decision (table lookup); the source address is used for adding entries to the table and for updating purposes. Let us elaborate on this process using Figure 5.86.

Figure 5.86 Learning switch

Gradual building of Table



1. When station A sends a frame to station D, the switch does not have an entry for either D or A. The frame goes out from all three ports; the frame floods the network. However, by looking at the source address, the switch learns that station A must be connected to port 1. This means that frames destined for A, in the future, must be sent out through port 1. The switch adds this entry to its table. The table has its first entry now.
2. When station D sends a frame to station B, the switch has no entry for B, so it floods the network again. However, it adds one more entry to the table related to station D.
3. The learning process continues until the table has information about every port.

However, note that the learning process may take a long time. For example, if a station does not send out a frame (a rare situation), the station will never have an entry in the table.

5.7.3 Routers

We discussed routers in Chapter 4. In this chapter, we mention routers to compare them with a two-layer switch and a hub. A *router* is a three-layer device; it operates in the physical, data-link, and network layers. As a physical-layer device, it regenerates the signal it receives. As a link-layer device, the router checks the physical addresses (source and destination) contained in the packet. As a network-layer device, a router checks the network-layer addresses.

A router is a three-layer (physical, data-link, and network) device.

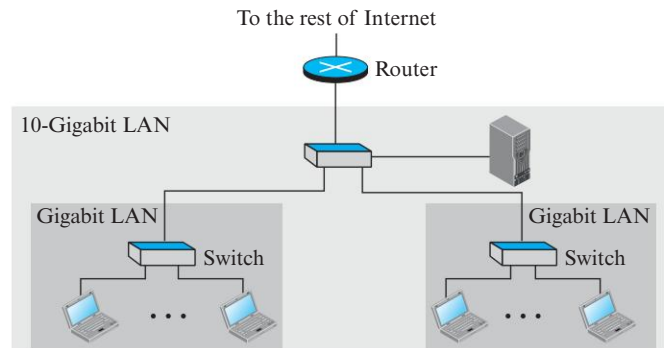
A router can connect networks. In other words, a router is an internetworking device; it connects independent networks to form an internetwork. According to this definition, two networks connected by a router become an internetwork or an internet.

There are three major differences between a router and a repeater or a switch.

1. A router has a physical and logical (IP) address for each of its interfaces.
2. A router acts only on those packets in which the link-layer destination address matches the address of the interface at which the packet arrives.
3. A router changes the link-layer address of the packet (both source and destination) when it forwards the packet.

Let us give an example. In Figure 5.87, assume an organization has two separate buildings with a Gigabit Ethernet LAN installed in each building. The organization uses switches in

Figure 5.87 Routing example



each LAN. The two LANs can be connected to form a larger LAN using 10-Gigabit Ethernet technology that speeds up the connection to the Ethernet and the connection to the organization server. A router then can connect the whole system to the Internet.

A router, as we saw in Chapter 4, will change the MAC address it receives because the MAC addresses have only local jurisdictions.

A router changes the link-layer addresses in a packet.

5.8 END-CHAPTER MATERIALS

5.8.1 Recommended Reading

For more details about subjects discussed in this chapter, we recommend the following books. The items in brackets [...] refer to the reference list at the end of the text.

Books

Several excellent books discuss link-layer issues. Among them we recommend [Ham 80], [Zar 02], [Ror 96], [Tan 03], [GW 02], [For 03], [KMK 04], [Sta 04], [Kes 02], [PD 03], [Kei 02], [Spu 00], [KCK 98], [Sau 98], [Izz 00], [Per 00], and [WV 00].

RFCs

A discussion of the use of the checksum in the Internet can be found in RFC 1141.

5.8.2 Key Terms

10-Gigabit Ethernet	data link control (DLC)
Address Resolution Protocol (ARP)	dataword
ALOHA	demodulator
application adaptation layer (AAL)	digital subscriber line (DSL)
asynchronous balanced mode (ABM)	Fast Ethernet
Asynchronous Transfer Mode (ATM)	Gigabit Ethernet
bit stuffing	Hamming distance
burst error	High-level Data Link Control (HDLC)
byte stuffing	hybrid-fiber coaxial (HFC) network
carrier sense multiple access (CSMA)	Internet Protocol Control Protocol (IPCP)
carrier sense multiple access with collision avoidance (CSMA/CA)	Link Control Protocol (LCP)
carrier sense multiple access with collision detection (CSMA/CD)	logical link control (LLC)
Challenge Handshake Authentication Protocol (CHAP)	media access control (MAC)
channelization	modem
codeword	modulator
Community Antenna TV (CATV)	network interface card (NIC)
contention	network-to-network interface (NNI)
controlled access	one's complement
convergence sublayer (CS)	optical carrier (OC)
cyclic redundancy check (CRC)	parity-check code
	Point-to-Point Protocol (PPP)
	polling
	Project 802

pure ALOHA	synchronous transport signal (STS)
random access	syndrome
repeater	time-division multiple access (TDMA)
segmentation and reassembly (SAR)	token passing
simple and efficient adaptation layer (SEAL)	transmission path (TP)
single-bit error	user-to-network interface (UNI)
slotted ALOHA	virtual circuit (VC)
Standard Ethernet	virtual local area network (VLAN)
switch	virtual path (VP)
Synchronous Optical Network (SONET)	virtual tributaries (VT)

5.8.3 Summary

We can consider the data-link layer as two sublayers. The upper sublayer is responsible for data link control, and the lower sublayer is responsible for resolving access to the shared media. Data link control (DLC) deals with the design and procedures for communication between two adjacent nodes: node-to-node communication. This sublayer is responsible for framing and error control. Error control deals with data corruption during transmission. We discussed two link-layer protocols in this chapter: HDLC and PPP. High-level Data Link Control (HDLC) is a bit-oriented protocol for communication over point-to-point and multipoint links. However, the most common protocol for point-to-point access is the Point-to-Point Protocol (PPP), which is a byte-oriented protocol.

Many formal protocols have been devised to handle access to a shared link. We categorize them into three groups: random access protocols, controlled access protocols, and channelization protocols. In random access or contention methods, no station is superior to another station and none is assigned the control over another. In controlled access, the stations consult one another to find which station has the right to send. Channelization is a multiple-access method in which the available bandwidth of a link is shared in time, frequency, or through code, between different stations.

At the data-link layer, we use link-layer addressing. The system normally finds the link-layer address of the next node using the Address Resolution Protocol.

Ethernet is the most widely used local area network protocol. The data-link layer of Ethernet consists of the LLC sublayer and the MAC sublayer. The MAC sublayer is responsible for the operation of the CSMA/CD access method and framing. Each station on an Ethernet network has a unique 48-bit address imprinted on its network interface card (NIC). A virtual local area network (VLAN) is configured by software, not by physical wiring. Membership in a VLAN can be based on port numbers, MAC addresses, IP addresses, IP multicast addresses, or a combination of these features. VLANs are cost- and time-efficient, can reduce network traffic, and provide an extra measure of security.

The need to communicate digital data resulted in the invention of the dial-up modem. Because of the need for high-speed downloading and uploading, the telephone companies added a new technology, the *digital subscriber line* (DSL). Cable networks were originally created to provide better access to TV programs. Cable TV also found a good market in Internet access provision using some of the channels originally designed for video. Synchronous Optical Network (SONET) is a standard developed by ANSI for fiber-optic networks. Asynchronous Transfer Mode (ATM) is a cell relay

protocol that, in combination with SONET, allows high-speed connections. A cell is a small, fixed-size block of information. The ATM data packet is a cell composed of 53 bytes. The ATM standard defines three layers: Application adaptation layer (AAL), ATM layer, and physical layer.

A repeater is a connecting device that operates in the physical layer of the Internet model. A switch is a connecting device that operates in the physical and data-link layers of the Internet model. A transparent switch can forward and filter frames and automatically build its forwarding table. A router is a connecting device that operates in the first three layers of the TCP/IP suite.

5.9 PRACTICE SET

5.9.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check his/her understanding of the materials before continuing with the practice set.

5.9.2 Questions

- Q5-1.** How does a single-bit error differ from a burst error?
- Q5-2.** What is the definition of a linear block code?
- Q5-3.** In a block code, a dataword is 20-bits and the corresponding codeword is 25-bits. What are the values of k , r , and n according to the definitions in the text? How many redundant bits are added to each dataword?
- Q5-4.** In a codeword, we add two redundant bits to each 8-bit data word. Find the number of
 - a.** valid codewords
 - b.** invalid codewords
- Q5-5.** Distinguish between communication at the network layer and communication at the data-link layer.
- Q5-6.** Distinguish between a point-to-point link and a broadcast link.
- Q5-7.** Explain why flags are needed when we use variable-size frames.
- Q5-8.** Explain why we cannot bit-stuff in character-oriented framing to change a flag byte appearing in the text.
- Q5-9.** A category of error detecting (and correcting) code, called the Hamming code, is a code in which $d_{\min} = 3$. This code can detect up to two errors (or correct one single error). In this code, the values of n , k , and r are related as: $n = 2^r - 1$ and $k = n - r$. Find the number of bits in the dataword and the codewords if r is 3.
- Q5-10.** In CRC, if the dataword is 5-bits and the codeword is 8-bits, how many 0s need to be added to the dataword to make the dividend? What is the size of the remainder? What is the size of the divisor?
- Q5-11.** What is the minimum Hamming distance?
- Q5-12.** If we want to be able to detect two-bit errors, what should be the minimum Hamming distance?

- Q5-13.** In CRC, we have chosen the generator 1100101. What is the probability of detecting a burst error of length
 a. 5? b. 7? c. 10?
- Q5-14.** Assume that we are sending data items of 16-bit length. If two data items are swapped during transmission, can the traditional checksum detect this error? Explain.
- Q5-15.** In CRC, which of the following generators (divisors) guarantees the detection of a single bit error?
 a. 101 b. 100 c. 1
- Q5-16.** In CRC, which of the following generators (divisors) guarantees the detection of an odd number of errors?
 a. 10111 b. 101101 c. 111
- Q5-17.** Stations in a pure Aloha network send frames of size 1000-bits at the rate of 1 Mbps. What is the vulnerable time for this network?
- Q5-18.** In a pure Aloha network with $G = 1/2$, how is the throughput affected in each of the following cases?
 a. G is increased to 1 b. G is decreased to $1/4$
- Q5-19.** To understand the uses of K in Figure 5.40, find the probability that a station can send immediately in each of the following cases:
 a. After one failure b. After three failures
- Q5-20.** Based on Figure 5.30, how do we interpret *success* in an Aloha network?
- Q5-21.** Assume that the propagation delay in a broadcast network is $5\ \mu\text{s}$ and the frame transmission time is $10\ \mu\text{s}$.
 a. How long does it take for the first bit to reach the destination?
 b. How long does it take for the last bit to reach the destination after the first bit has arrived?
 c. How long is the network involved with this frame (vulnerable to collision)?
- Q5-22.** Assume that the propagation delay in a broadcast network is $3\ \mu\text{s}$ and the frame transmission time is $5\ \mu\text{s}$. Can the collision be detected no matter where it occurs?
- Q5-23.** Can the value of a traditional checksum be all 0s (in binary)? Defend your answer.
- Q5-24.** Show how the Fletcher algorithm (Figure 5.18) attaches weights to the data items when calculating the checksum.
- Q5-25.** Explain why there is only one address field (instead of two) in an HDLC frame.
- Q5-26.** Which of the following is a random-access protocol?
 a. CSMA/CD b. Polling c. TDMA
- Q5-27.** Do we need a multiple access protocol when we use the DSL service provided by the telephone company to access the Internet? Why?
- Q5-28.** Is the size of the ARP packet fixed? Explain.

- Q5-29.** What is the size of an ARP packet when the protocol is IPv4 and the hardware is Ethernet?
- Q5-30.** What is the size of an Ethernet frame carrying an ARP packet of size 28 bytes?
- Q5-31.** Can two hosts in two different networks have the same link-layer address?
- Q5-32.** Explain why collision is an issue in random access protocols but not in controlled access or channelizing protocols.
- Q5-33.** Why is there no need for CSMA/CD on a full-duplex Ethernet LAN?
- Q5-34.** Compare the data rates for Standard Ethernet, Fast Ethernet, Gigabit Ethernet, and Ten-Gigabit Ethernet.
- Q5-35.** What are the common Standard Ethernet implementations?
- Q5-36.** What are the common Gigabit Ethernet implementations?
- Q5-37.** What is dial-up modem technology? List some of the common modem standards discussed in this chapter and give their data rates.
- Q5-38.** Compare and contrast a traditional cable network with a hybrid fiber-coaxial network.
- Q5-39.** Four stations are connected to a hub in a transitional Ethernet network. The distances between the hub and the stations are 300 m, 400 m, 500 m, and 700 m, respectively. What is the length of this network when we need to calculate T_p ?
- Q5-40.** What do we mean when we say that a link-layer switch can filter traffic? Why is filtering important?
- Q5-41.** How is the preamble field different from the SFD field in an Ethernet frame?
- Q5-42.** What is the purpose of a NIC?
- Q5-43.** Why is SONET called a synchronous network?
- Q5-44.** Discuss the functions of each SONET layer.
- Q5-45.** How does a VLAN save a company time and money?
- Q5-46.** How does a VLAN reduce network traffic?
- Q5-47.** In ATM, what is the relationship between TPs, VPs, and VCs?

5.9.3 Problems

- P5-1.** What is the maximum effect of a 2 ms burst of noise on data transmitted at the following rates?
 a. 1500 bps b. 12 kbps c. 100 kbps d. 100 Mbps
- P5-2.** Assume that the probability that a bit in a data unit is corrupted during transmission is p . Find the probability that x number of bits are corrupted in an n -bit data unit for each of the following cases:
 a. $n = 8, x = 1, p = 0.2$
 b. $n = 16, x = 3, p = 0.3$
 c. $n = 32, x = 10, p = 0.4$

P5-3. Byte-stuff the following frame payload in which E is the escape byte, F is the flag byte, and D is a data byte other than an escape or a flag character.

D	E	D	D	F	D	D	E	E	D	F	D
---	---	---	---	---	---	---	---	---	---	---	---

P5-4. Bit-stuff the following frame payload:

00011111111001111101000111111111110000111

P5-5. Using the code in Table 5.2, what is the dataword if one of the following codewords is received?

- a. 01011 b. 11111 c. 00000 d. 11011

P5-6. Prove that the code represented by the following codewords is not linear. You need to find only one case that violates the linearity.

{(00000), (01011), (10111), (11111)}

P5-7. Exclusive-OR (XOR) is one of the most used operations in the calculation of codewords. Apply the exclusive-OR operation on the following pair of patterns. Interpret the results.

- a. (10001) ⊕ (10001) b. (11100) ⊕ (00000) c. (10011) ⊕ (11111)

P5-8. In Table 5.1, the sender sends dataword 10. A 3-bit burst error corrupts the codeword. Can the receiver detect the error? Defend your answer.

P5-9. A simple parity check bit, which is normally added at the end of the word (changing a 7-bit ASCII character to a byte), cannot detect even numbers of errors. For example, two, four, six, or eight errors cannot be detected in this way. A better solution is to organize the characters in a table and create row and column parities. The bit in the row parity is sent with the byte, the column parity is sent as an extra byte (Figure 5.88).

Figure 5.88 Problem P5-9

	C1	C2	C3	C4	C5	C6	C7	
R1	1	1	0	0	1	1	1	1
R2	1	0	1	1	1	0	1	1
R3	0	1	1	1	0	0	1	0
R4	0	1	0	1	0	0	1	1
	0	1	0	1	0	1	0	1
	Column parities							

Rn: Row *n*
Cm: Column *m*

Row parities

Show how the following errors can be detected:

- a. An error at (R3, C3).
 - b. Two errors at (R3, C4) and (R3, C6).
 - c. Three errors at (R2, C4), (R2, C5), and (R3, C4).
 - d. Four errors at (R1, C2), (R1, C6), (R3, C2), and (R3, C6).
- P5-10.** Given the dataword 101001111 and the divisor 10111, show the generation of the CRC codeword at the sender site (using binary division).
- P5-11.** What is the Hamming distance for each of the following codewords:
- a. $d(10000, 00000)$
 - b. $d(10101, 10000)$
 - c. $d(00000, 11111)$
 - d. $d(00000, 00000)$
- P5-12.** Although it can be formally proved that the code in Table 5.3 is both linear and cyclic, use only two tests to partially prove the fact:
- a. Test the cyclic property on codeword 0101100.
 - b. Test the linear property on codewords 0010110 and 1111111.
- P5-13.** Referring to the CRC-8 in Table 5.4, answer the following questions:
- a. Does it detect a single error? Defend your answer.
 - b. Does it detect a burst error of size 6? Defend your answer.
 - c. What is the probability of detecting a burst error of size 9?
 - d. What is the probability of detecting a burst error of size 15?
- P5-14.** Assuming even parity, find the parity bit for each of the following data units.
- a. 1001011
 - b. 0001100
 - c. 1000000
 - d. 1110111
- P5-15.** This problem shows a special case in checksum handling. A sender has two data items to send: $(4567)_{16}$ and $(BA98)_{16}$. What is the value of the checksum?
- P5-16.** Manually simulate the Fletcher algorithm (Figure 5.18) to calculate the checksum of the following bytes: $(2B)_{16}$, $(3F)_{16}$, $(6A)_{16}$, and $(AF)_{16}$. Also show that the result is a weighted checksum.
- P5-17.** Assume that a packet is made only of four 16-bit words $(A7A2)_{16}$, $(CABF)_{16}$, $(903A)_{16}$, and $(A123)_{16}$. Manually simulate the algorithm in Figure 5.17 to find the checksum.
- P5-18.** Traditional checksum calculation needs to be done in one's complement arithmetic. Computers and calculators today are designed to do calculations in two's complement arithmetic. One way to calculate the traditional checksum is to add the numbers in two's complement arithmetic, find the quotient and remainder of dividing the result by 2^{16} , and add the quotient and the remainder to get the sum in one's complement. The checksum can be found by subtracting the sum from $2^{16} - 1$. Use the above method to find the checksum of the following four numbers: 43,689, 64,463, 45,112, and 59,683.
- P5-19.** An ISBN-13 code, a new version of ISBN-10, is another example of a weighted checksum with 13 digits, in which there are 12 decimal digits

defining the book and the last digit is the checksum digit. The code, $D_1D_2D_3D_4D_5D_6D_7D_8D_9D_{10}D_{11}D_{12}C$, satisfies the following.

$$[(1 \times D_1) + (3 \times D_2) + (1 \times D_3) + \dots + (3 \times D_{12}) + (1 \times C)] \bmod 10 = 0$$

In other words, the weights are 1 and 3, alternatively. Using the above description, calculate the check digit for ISBN-13: **978-0-07-296775-C**.

- P5-20.** To formulate the performance of a multiple-access network, we need a mathematical model. When the number of stations in a network is very large, the Poisson distribution, $p[x] = (e^{-\lambda} \times \lambda^x)/(x!)$, is used. In this formula, $p[x]$ is the probability of generating x number of frames in a period of time and λ is the average number of generated frames during the same period of time. Using the Poisson distribution:
- Find the probability that a pure Aloha network generates x number of frames during the vulnerable time. Note that the vulnerable time for this network is two times the frame transmission time (T_{fr}).
 - Find the probability that a slotted Aloha network generates x number of frames during the vulnerable time. Note that the vulnerable time for this network is equal to the frame transmission time (T_{fr}).
- P5-21.** In the previous problem, we used the Poisson distribution to find the probability of generating x number of frames, in a certain period of time, in a pure or slotted Aloha network as $p[x] = (e^{-\lambda} \times \lambda^x)/(x!)$. In this problem, we want to find the probability that a frame in a such a network reaches its destination without colliding with other frames. For this purpose, it is simpler to think that we have G stations, each sending an average of one frame during the frame transmission time (instead of having N frames, each sending an average of G/N frames during the same time). Then, the probability of success for a station is the probability that no other station sends a frame during the vulnerable time.
- Find the probability that a station in a pure Aloha network can successfully send a frame during a vulnerable time.
 - Find the probability that a station in a slotted Aloha network can successfully send a frame during a vulnerable time.
- P5-22.** In the previous problem, we found that the probability of a station (in a G -station network) successfully sending a frame in a vulnerable time is $P = e^{-2G}$ for a pure Aloha and $P = e^{-G}$ for a slotted Aloha network. In this problem, we want to find the throughput of these networks, which is the probability that any station (out of G stations) can successfully send a frame during the vulnerable time.
- Find the throughput of a pure Aloha network.
 - Find the throughput of a slotted Aloha network.
- P5-23.** In the previous problem, we showed that the throughput is $S = Ge^{-2G}$ for a pure Aloha network and $S = Ge^{-G}$ for a slotted Aloha network. In this problem, we want to find the value of G in each network that makes the throughput maximum and find the value of the maximum throughput. This can be

done if we find the derivative of S with respect to G and set the derivative to zero.

- a. Find the value of G that makes the throughput maximum, and find the value of the maximum throughput for a pure Aloha network.
- b. Find the value of G that makes the throughput maximum, and find the value of the maximum throughput for a slotted Aloha network.

P5-24. A multiple access network with a large number of stations can be analyzed using the Poisson distribution. When there is a limited number of stations in a network, we need to use another approach for this analysis. In a network with N stations, we assume that each station has a frame to send during the frame transmission time (T_{fr}) with probability p . In such a network, a station is successful in sending its frame if the station has a frame to send during the vulnerable time and no other station has a frame to send during this period of time.

- a. Find the probability that a station in a pure Aloha network can successfully send a frame during the vulnerable time.
- b. Find the probability that a station in an slotted Aloha network can successfully send a frame during the vulnerable time.

P5-25. In the previous problem, we found the probability of success for a station to send a frame successfully during the vulnerable time. The throughput of a network with a limited number of stations is the probability that any station (out of N stations) can send a frame successfully. In other words, the throughput is the sum of N success probabilities.

- a. Find the throughput of a pure Aloha network.
- b. Find the throughput of a slotted Aloha network.

P5-26. In the previous problem, we found the throughputs of a pure and a slotted Aloha network as $S = Np(1-p)^{2(N-1)}$ and $S = Np(1-p)^{(N-1)}$, respectively. In this problem we want to find the maximum throughput with respect to p .

- a. Find the value of p that maximizes the throughput of a pure Aloha network, and calculate the maximum throughput when N is a very large number.
- b. Find the value of p that maximizes the throughput of a slotted Aloha network, and calculate the maximum throughput when N is a very large number.

P5-27. There are only three active stations in a slotted Aloha network: A, B, and C. Each station generates a frame in a time slot with the corresponding probabilities $p_A = 0.2$, $p_B = 0.3$, and $p_C = 0.4$, respectively.

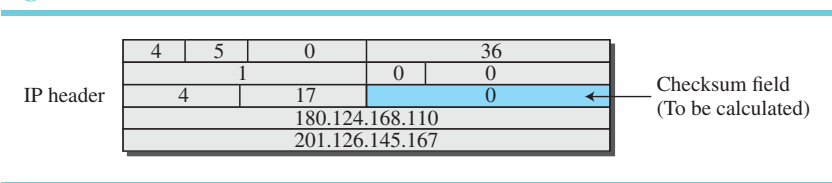
- a. What is the throughput of each station?
- b. What is the throughput of the network?

P5-28. There are only three active stations in a slotted Aloha network: A, B, and C. Each station generates a frame in a time slot with the corresponding probabilities $p_A = 0.2$, $p_B = 0.3$, and $p_C = 0.4$, respectively.

- a. What is the probability that any station can send a frame in the first slot?
- b. What is the probability that station A can successfully send a frame for the first time in the second slot?

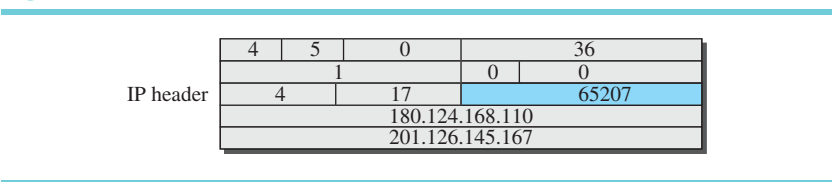
- c. What is the probability that station C can successfully send a frame for the first time in the third slot?
- P5-29.** Manually simulate the Adler algorithm (Figure 5.19) to calculate the checksum of the following words: $(\text{FBFF})_{16}$ and $(\text{EFAA})_{16}$. Also show that the result is a weighted checksum.
- P5-30.** As a practical example, find the value of the checksum field for a small datagram as shown in Figure 5.89. Since the checksum for an IP datagram is calculated only for the header, we have shown only the values of these fields.

Figure 5.89 Problem P5-30



- P5-31.** A slotted Aloha network is working with maximum throughput.
- a. What is the probability that a slot is empty?
- b. How many slots, n , on an average, should pass before getting an empty slot?
- P5-32.** One of the useful parameters in a LAN is the number of bits that can fit in one meter of the medium ($n_{b/m}$). Find the value of $n_{b/m}$ if the data rate is 100 Mbps and the medium propagation speed is 2×10^8 m/s.
- P5-33.** Another useful parameter in a LAN is the bit length of the medium (L_b), which defines the number of bits that the medium can hold at any time. Find the bit length of a LAN if the data rate is 100 Mbps and the medium length in meters (L_m) for communication between two stations is 200 m. Assume that the propagation speed in the medium is 2×10^8 m/s.
- P5-34.** We have defined the parameter a as the number of frames that can fit the medium between two stations as $a = (T_p)/(T_{fr})$. Another way to define this parameter is $a = L_b/F_b$, in which L_b is the bit length of the medium and F_b is the frame length of the medium. Show that the two definitions are equivalent.
- P5-35.** Assume that a datagram with the header in Figure 5.90 has arrived. The value of the checksum is 65,207, as shown. Check if the header is corrupted in transition.

Figure 5.90 Problem P5-35



- P5-36.** One of the examples of a weighted checksum is the ISBN-10 code we see printed on the back cover of some books. In ISBN-10, there are 9 decimal digits that define the country, the publisher, and the book. The tenth (rightmost) digit is a checksum digit. The code, $D_1D_2D_3D_4D_5D_6D_7D_8D_9C$, satisfies the following.

$$[(10 \times D_1) + (9 \times D_2) + (8 \times D_3) + \dots + (2 \times D_9) + (1 \times C)] \bmod 11 = 0$$

In other words, the weights are 10, 9, ..., 1. If the calculated value for C is 10, one uses the letter X instead. By replacing each weight w with its complement in modulo 11 arithmetic ($11 - w$), it can be shown that the check digit can be calculated as shown below.

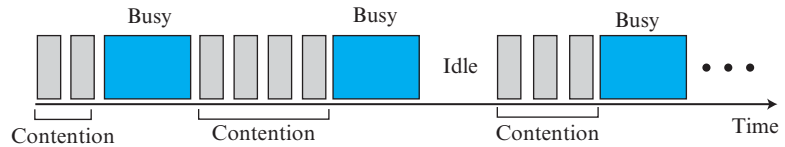
$$C = [(1 \times D_1) + (2 \times D_2) + (3 \times D_3) + \dots + (9 \times D_9)] \bmod 11$$

Calculate the check digit for ISBN-10: **0-07-296775-C**.

- P5-37.** There are only two stations, A and B, in a bus 1-persistence CSMA/CD network with $T_p = 25.6 \mu s$ and $T_{fr} = 51.2 \mu s$. Station A has a frame to send to station B. The frame is unsuccessful two times and succeeds on the third try. Draw a timeline diagram for this problem. Assume that the R is 1 and 2, respectively and ignore the time for sending a jamming signal (see Figure 5.40).
- P5-38.** To understand why we need to have a minimum frame size $T_{fr} = 2 \times T_p$ in a CDMA/CD network, assume we have a bus network with only two stations, A and B, in which $T_{fr} = 40 \mu s$ and $T_p = 25 \mu s$. Station A starts sending a frame at time $t = 0.0 \mu s$ and station B starts sending a frame at $t = 23.0 \mu s$. Answer the following questions:
- Do frames collide?
 - If the answer to part *a* is yes, does station A detect collision?
 - If the answer to part *a* is yes, does station B detect collision?
- P5-39.** In a bus 1-persistence CSMA/CD with $T_p = 50 \mu s$ and $T_{fr} = 120 \mu s$, there are two stations A and B. Both stations start sending frames to each other at the same time. Since the frames collide, each station tries to retransmit. Station A comes out with $R = 0$ and station B with $R = 1$. Ignore any other delay including the delay for sending jamming signals. Do the frames collide again? Draw a timeline diagram to prove your claim. Does the generation of a random number help avoid collision in this case?
- P5-40.** The random variable R (Figure 5.40) is designed to give stations different delays when a collision has occurred. To alleviate the collision, we expect that different stations generate different values of R . To show the point, find the probability that the value of R is the same for two stations after
- the first collision
 - the second collision
- P5-41.** Assume that we have a slotted CSMA/CD network. Each station in this network uses a contention period, in which the station contends for access to the shared channel before being able to send a frame. We assume that the contention period is made of contention slots. At the beginning of each slot, the station senses the channel. If the channel is free, the station sends its frame; if the channel is busy, the station refrains from sending and waits until the beginning

of the next slot. In other words, the station waits, on an average, for k slots before sending its frame, as shown in Figure 5.91. Note that the channel is either in the contention state, the transmitting state, or the idle state (when no station has a frame to send). However, if N is a very large number, the idle state actually disappears.

Figure 5.91 Problem P5-41



- a. What is the probability of a free slot (P_{free}) if the number of stations is N and each station has a frame to send with probability p ?
 - b. What is the maximum of this probability when N is a very large number?
 - c. What is the probability that the j th slot is free?
 - d. What is the average number of slots, k , that a station should wait before getting a free slot?
 - e. What is the value of k when N (the number of stations) is very large?
- P5-42.** Although the throughput calculation of a CDMA/CD is really involved, we can calculate the maximum throughput of a slotted CDMA/CD with the specification we described in the previous problem. We found that the average number of contention slots a station needs to wait is $k = e$ slots. With this assumption, the throughput of a slotted CDMA/CD is

$$S = (T_{\text{fr}})/(\text{time the channel is busy for a frame})$$

The time the channel is busy for a frame is the time to wait for a free slot plus the time to transmit the frame plus the propagation delay to receive the good news about the lack of collision. Assume that the duration of a contention slot is $2 \times (T_p)$ and $a = (T_p)/(T_{\text{fr}})$. Note that the parameter a is the number of frames that occupy the transmission media. Find the throughput of a slotted CDMA/CD in terms of the parameter a .

- P5-43.** In a bus CSMD network with a data rate of 10 Mbps, a collision occurs 20 μs after the first bit of the frame leaves the sending station. What should the length of the frame be so that the sender can detect the collision?
- P5-44.** Assume that there are only two stations, A and B, in a bus CSMA/CD network. The distance between the two stations is 2000 m and the propagation speed is 2×10^8 m/s. If station A starts transmitting at time t_1 :
- a. Does the protocol allow station B to start transmitting at time $t_1 + 8 \mu\text{s}$? If the answer is yes, what will happen?

- b. Does the protocol allow station B to start transmitting at time $t_1 + 11 \mu\text{s}$? If the answer is yes, what will happen?
- P5-45.** If an Ethernet destination address is 07:01:02:03:04:05, what is the type of the address (unicast, multicast, or broadcast)?
- P5-46.** There are two stations, A and B, in a CSMA/CD network. Assume that stations A and B each have a frame to send in the collision interval ($2 \times t_p$) with probability p_1 and p_2 , respectively. Answer the following questions if p_1 is 0.3 and p_2 is 0.4:
- What is the probability of success for station A?
 - What is the probability of success for station B?
 - What is the probability of success for a frame?
- P5-47.** We have a pure ALOHA network with a data rate of 10 Mbps. What is the maximum number of 1000-bit frames that can be successfully sent by this network.
- P5-48.** In a CDMA/CD network with a data rate of 10 Mbps, the minimum frame size is found to be 512-bits for the correct operation of the collision detection process. What should be the minimum frame size if we keep the size of the network constant, but increase the data rate to
- 100 Mbps?
 - 1 Gbps?
 - 10 Gbps?
- P5-49.** What is the hexadecimal equivalent of the following Ethernet address?
- 01011010 00010001 01010101 00011000 10101010 00001111
- P5-50.** How does the Ethernet address 1A:2B:3C:4D:5E:6F appear on the line in binary?
- P5-51.** Suppose the length of a 10Base5 cable is 2500 m. If the speed of propagation in a thick coaxial cable is 200,000,000 m/s, how long does it take for a bit to travel from the beginning to the end of the network? Assume that there is a $10 \mu\text{s}$ delay in the equipment.
- P5-52.** What type of topology is used when customers in an area use DSL modems for data transfer purposes? Explain.
- P5-53.** A router with IPv4 address 125.45.23.12 and Ethernet address 23:45:AB:4F:67:CD has received a packet for a host destination with IP address 125.11.78.10. Show the entries in the ARP request packet sent by the router. Also encapsulate the packet in an Ethernet frame.
- P5-54.** In Figure 5.50, show the activity at Bob's site when the response is sent from Bob to Alice.
- P5-55.** An Ethernet MAC sublayer receives 42 bytes of data from the upper layer. How many bytes of padding must be added to the data?
- P5-56.** What is the ratio of useful data to the entire packet for the smallest Ethernet frame?
- P5-57.** A link-layer switch uses a filtering table; a router uses a forwarding table. Can you explain the difference?

5.10 SIMULATION EXPERIMENTS

5.10.1 Applets

We have created some Java applets to show some of the main concepts discussed in this chapter. It is strongly recommended that students activate these applets on the book website and carefully examine the protocols in action.

5.10.2 Lab Assignments

In this section, we use Wireshark to simulate two protocols: Ethernet and ARP. Full descriptions of these lab assignments are on the book website.

Lab5-1. In this lab, we need to examine the contents of a frame sent by the data-link layer. We want to find the value of different fields such as destination and source MAC addresses, the value of CRC, the value of the protocol field which shows which payload is being carried by the frame, and so on.

Lab5-2. In this lab, we need to examine the contents of an ARP packet. We want to capture the ARP request and ARP replay packets. The interesting fields to examine are those that show what type of source and destination addresses are used in each packet.

5.11 PROGRAMMING ASSIGNMENTS

For each of the following assignments, write a program in the programming language you are familiar with.

Prg5-1. Write and test a program that simulates the byte stuffing and byte unstuffing as shown in Figure 5.5.

Prg5-2. Write and test a program that simulates the bit stuffing and bit unstuffing as shown in Figure 5.7.

Prg5-3. Write and test a program to simulate the flow diagram in Figure 5.17.

Prg5-4. Write and test a program to simulate the flow diagram in Figure 5.18.

Prg5-5. Write and test a program to simulate the flow diagram in Figure 5.19.